



**Hochschule
Bonn-Rhein-Sieg**

*University
of Applied Sciences*

Fachbereich Informatik
Department of Computer Sciences

Abschlussarbeit

Studiengang Bachelor Informatik

Entwurf und Evaluation eines fehlertoleranten Job-Processing in Java

von

Max Urfei

Erstprüfer: Prof. Dr. Andreas Hackelöer

Zweitprüfer: M. Sc. Moritz Balg

eingereicht am TT.MM.YYYY

Inhaltsverzeichnis

Abkürzungsverzeichnis	iv
Abbildungsverzeichnis	v
Tabellenverzeichnis	v
Listings	vi
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung	3
1.4 Hypothesen	4
1.5 Aufbau der Arbeit	4
2 Grundlagen	6
2.1 Grundlagen der Hintergrundverarbeitung	6
2.1.1 Asynchrone Verarbeitung und Queueing	6
2.1.2 Ausführungssemantiken	8
2.1.3 Background-Job-Processing	9
2.2 Konsistenz und konkurrierende Verarbeitung	10
2.2.1 ACID	11
2.2.2 Transaktionsgrenzen	12
2.2.3 Synchronisationsverfahren und Isolationsstufen	13
2.3 Fehlertoleranz und Wiederherstellung	14
2.3.1 Fehlerklassifikation	14
2.3.2 Retry-/Backoff-Strategien	15
2.3.3 Recovery	17
2.3.4 Idempotenz	18
2.4 Stand der Technik	19
2.4.1 Brokerbasierte Ansätze	19
2.4.2 Datenbankbasierte Ansätze	20
2.4.3 Einordnung und Handlungsbedarf	20
3 Methodik	22
3.1 Literaturrecherche	22
3.2 Systementwurf	22
3.2.1 Anforderungserhebung und Ableitung der Zusicherungen	23
3.2.2 Architekturentwurf und prototypische Implementierung	23
3.3 Evaluation	24
3.3.1 Operationalisierung der Zusicherungen	24
3.3.2 Test- und Nachweismethoden	25

3.3.3	Fehlerinjektion und Testdurchführung	26
3.3.4	Reproduzierbarkeit und Auswertung	26
4	Anforderungen	27
4.1	Systemkontext	27
4.2	Abgrenzung des betrachteten Fehlermodells	28
4.3	Zusicherungen	30
4.3.1	Joberstellung	31
4.3.2	Jobverarbeitung	31
4.3.3	Retries und Recovery	32
5	Umsetzung	34
5.1	Architektur und Design	34
5.1.1	Architekturentwurf	34
5.1.2	Zustandsmodell	36
5.1.3	Entwurf der Datenbank	37
5.1.4	Joberstellung	39
5.1.5	Exklusive Verarbeitung und Transaktionsmodell	40
5.1.6	Retry-Strategie	42
5.1.7	Recovery	44
5.1.8	Idempotente Ergebniserzeugung	46
5.2	Implementierung	47
5.2.1	Auswahl verwendeter Technologien	47
5.2.2	Persistenz und Datenbankszugriff	48
5.2.3	Joberstellung	50
5.2.4	Claiming und Verarbeitung	51
5.2.5	Abschluss einer Jobverarbeitung	54
5.2.6	Retries	55
5.2.7	Recovery	57
6	Evaluation	60
6.1	Versuchsaufbau	60
6.2	Fehlerszenarien und Operationalisierung	61
6.3	Durchführung	62
6.3.1	Joberstellung und Idempotenz (Z-1, Z-2)	62
6.3.2	Retry-Verhalten bei Infrastrukturfehlern (Z-6)	63
6.3.3	Atomarität der Ergebnisspeicherung (Z-4)	64
6.3.4	Nebenläufigkeit beim Claiming und Abschluss (Z-3, Z-5)	64
6.3.5	Recovery nach Worker-Ausfall (Z-7, Z-8, Z-9)	65
6.4	Ergebnisse	66
6.5	Bewertung der Hypothesen	70
7	Diskussion und Fazit	71
7.1	Gewonnene Erkenntnisse	71

7.2	Einschränkungen und Grenzen der Arbeit	71
	Literaturverzeichnis	74
8	Anhang	76
8.1	Testzuordnung	76
8.2	Testlogs	76

Abkürzungsverzeichnis

ACID Atomicity, Consistency, Isolation, Durability. 4, 11, 12, 14, 20, 34, 37, 42, 63, 67, 69

API Application Programming Interface. 35, 40, 42, 61

CRUD Create, Read, Update, Delete. 2

DTO Data Transfer Object. 50, 51

REST Representational State Transfer. 9, 27, 31, 34–36, 39, 40, 43, 47, 50, 55, 57, 60

UUID Universally Unique Identifier. 18, 19, 48, 50, 52

Abbildungsverzeichnis

2.1	Vergleich synchroner und asynchroner Kommunikation	7
2.2	Queueing (Microsoft Corporation (Hrsg.) 2026b)	7
2.3	Verhalten bei transienten Fehlern	16
4.1	Systemkontext	27
5.1	Abstrakte Architektur des entwickelten Systems	36
5.2	Zustandsmodell eines Jobs	37
5.3	Datenbankschema des entwickelten Job-Processing-Systems	38
5.4	Ablauf der Jobannahme	39
5.5	Transaktionsmodell	41
5.6	Klassendiagramm der Retry-Komponente	43
5.7	Mehrfachausführung mit Seiteneffekten	46

Tabellenverzeichnis

4.2	Übersicht betrachteter Fehlerklassen	29
4.4	Systemanforderungen	31
6.1	Testmatrix: Zusicherungen, Fehlerinjektion und Prüfkriterien	62
8.1	Zuordnung von Zusicherungen zu den jeweiligen Testmethoden	76

Listings

5.1	Claiming eines Jobs im <code>JobRepository</code>	49
5.2	Auswählen eines Jobs zur Bearbeitung	49
5.3	Finden verwaister Jobs	49
5.4	POST eines Jobs	50
5.5	POST Endpoint	51
5.6	Initialisierung des Worker-Pools	51
5.7	Ausführungsschleife eines Workers	52
5.8	Beanspruchen eines Jobs	53
5.9	Abschluss einer Jobverarbeitung	54
5.10	Generischer <code>RetryExecutor</code>	55
5.11	Beispiel einer <code>RetryPolicy</code>	56
5.12	Nutzung des <code>RetryExecutors</code>	57
5.13	Periodischer Recovery-Zyklus	58
5.14	Transaktionale Wiederherstellung eines Jobs	58

1 Einleitung

1.1 Motivation

Ein Zahlungsauftrag, der nach einem Verbindungsabbruch versehentlich mehrfach ausgeführt wird, belastet einen Kunden doppelt. Ein Compliance-Report, dessen Erstellung durch den Absturz des ausführenden Prozesses unbemerkt verloren geht, fehlt zum gesetzlich vorgeschriebenen Stichtag. So unterschiedlich beide Fälle wirken, beruhen sie auf derselben grundlegenden Anforderung an ein Background-Job-Processing-System: Ein Job darf im Fehlerfall weder mehrfach noch gar nicht wirken und vor allem nicht verloren gehen. Was sich zunächst als Frage der internen Konsistenz eines Systems liest, kann damit unmittelbar finanzielle und regulatorische Konsequenzen haben, die weit über das zugrundeliegende technische Problem hinausreichen.

In modernen Softwaresystemen stellt die Verarbeitung von Hintergrundjobs einen zentralen Bestandteil vieler Backend-Architekturen dar (Kleppmann 2017, Kap. 10). Zahlreiche Geschäftsprozesse, darunter die Generierung von Reports, der Import großer Datenmengen, periodische Abrechnungsläufe oder das Versenden asynchroner Benachrichtigungen, erfordern eine Verarbeitung, die entkoppelt vom synchronen Request-Response-Zyklus stattfindet. Insbesondere in industrienahen und sicherheitskritischen Anwendungen müssen solche Prozesse nicht nur funktional korrekt ablaufen, sondern darüber hinaus zuverlässig, nachvollziehbar und fehlertolerant sein. Aber auch in nicht-kritischen Systemen ist Zuverlässigkeit zum einen von Bedeutung für die Nutzerfreundlichkeit der Anwendung, zum anderen aber auch um Produktivitäts- und Gewinneinbußen in Produktivsystemen in Folge von Fehlern zu vermeiden (Kleppmann 2017, Kap. 1 Abschnitt „How Important Is Reliability?“).

1.2 Problemstellung

In der Praxis können bei der Verarbeitung von Hintergrundjobs jederzeit Teilausfälle auftreten. Prozesse können während der Verarbeitung unerwartet terminieren, Datenbankverbindungen können kurzzeitig unterbrochen werden, identische Anfragen infolge von clientseitigen Wiederholungsversuchen mehrfach an das System übermittelt werden oder Ergebnisse bzw. Antworten können verloren gehen (Kleppmann 2017, Kap. 8). Insbesondere Netzwerkpartitionierungsfehler tauchen dabei in der Praxis als häufige Ausfallursachen in Cloud-Systemen auf (Alquraan u. a. 2018), wodurch die im Folgenden beschriebenen Konsistenzprobleme keine seltenen Randfälle, sondern einen im laufenden Betrieb regelmäßig zu erwartenden Fall darstellen. Ohne geeignete Mechanismen führen derartige Szenarien zu inkonsistenten Systemzuständen oder dem Verlorengangen einer Nachricht. Eine naheliegende Reaktion auf letzteren Fall besteht darin, betroffene Operationen automatisch zu wiederholen. Werden jedoch viele zuvor fehlgeschlagene oder unterbrochene Operationen ohne geeignete Steuerung gleichzeitig erneut versucht, etwa weil eine Datenbank nach einem kurzzeitigen Ausfall wieder verfügbar wird und sämtliche

zuvor blockierten Anfragen nahezu zeitgleich erneut auf sie zugreifen, kann dies selbst zu einer erheblichen, kurzfristigen Lastspitze führen, die das gerade erst wiederhergestellte System erneut überlastet. Dieses in der Praxis als *Thundering-Herd-Problem* bezeichnete Phänomen zeigt, dass naive Fehlerbehandlungsstrategien ein bereits bestehendes Problem unter Umständen sogar verschärfen können (Newman 2026, Kap. 4).

Bestehende Frameworks und Bibliotheken, etwa Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026b), stellen umfangreiche Funktionalitäten für die Hintergrundverarbeitung bereit. Hierzu zählen unter anderem Scheduling, Wiederholungsmechanismen sowie Verfahren zur Fehlerbehandlung und Wiederaufnahme. Sie verfolgen dabei jedoch jeweils eigene Architekturansätze und abstrahieren die zugrunde liegenden Mechanismen weitgehend hinter ihren Programmierschnittstellen. Dadurch eignen sie sich nur eingeschränkt, um systematisch zu untersuchen, welche Architekturentscheidungen für bestimmte Konsistenz- und Verarbeitungszusicherungen tatsächlich erforderlich sind.

Die beschriebene Problematik ist von hoher praktischer und industrieller Relevanz. Nahezu jedes Backend-System, das über einfache Create, Read, Update, Delete (CRUD)-Operationen hinausgeht, muss Aufgaben zuverlässig im Hintergrund verarbeiten und dabei konsistente Datenbestände gewährleisten (Richardson 2018, Kap. 4). Fehler in diesem Bereich führen zu Datenverlust, inkonsistenten Geschäftszuständen sowie unnötigem manuellem Eingriff und können insbesondere in regulierten Domänen, wie beispielsweise der Luftfahrt, Compliance-Verstöße nach sich ziehen. Auch im Kontext von Cloud-nativen Architekturen ist dieses Thema relevant. Horizontale Skalierung, Container-Neustarts und kurzzeitige Netzwerkunterbrechungen gehören dort zum regulären Betriebsverhalten und müssen bereits beim Entwurf eines Systems berücksichtigt werden. Daher bilden die in dieser Arbeit behandelten Konzepte auch die Grundlage für den zuverlässigen Betrieb von Backend-Systemen in Cloud-Umgebungen.

Zur Umsetzung einer solchen zuverlässigen Hintergrundverarbeitung hat sich in der Praxis der Einsatz dedizierter Message Broker wie RabbitMQ oder Azure Service Bus etabliert, die eine robuste Entkopplung zwischen Auftragserstellung und -verarbeitung ermöglichen (Kleppmann 2017, Kap. 11). Dieser Ansatz erfordert jedoch den zusätzlichen Betrieb und die Überwachung einer eigenständigen Infrastrukturkomponente neben der ohnehin für die fachlichen Daten benötigten Datenbank. Zusätzlich braucht es eine Konsistenzsicherung, etwa mittels des Transactional-Outbox-Patterns, um Datenbank und Broker konsistent zu halten (Newman 2026, Kap. 8 Abschnitt „Pattern: Transactional Outbox“). Für viele klein- und mittelgroße Systeme steht dieser zusätzliche Betriebsaufwand in keinem angemessenen Verhältnis zum tatsächlichen Bedarf. Daraus ergibt sich als leitende Überlegung dieser Arbeit, ob sich die angestrebten Zuverlässigkeitsgarantien auch allein mit einer ohnehin bereits vorhandenen Datenbank erreichen lassen, ohne den Betrieb einer zusätzlichen Broker-Infrastruktur.

Obwohl die zugrundeliegenden Konzepte wie Fehlertoleranz, Recovery, Idempotenz, Nebenläufigkeitskontrolle und Transaktionsmanagement in der Literatur umfassend beschrieben werden, fehlen Arbeiten, die diese Aspekte zu einer durchgängigen datenbankbasierten Architektur für Background-Job-Processing zusammenführen und deren Verhalten unter definierten Fehlerszenarien systematisch evaluieren. Ebenso implementie-

ren bestehende Frameworks diese Mechanismen häufig als interne Abstraktionen, wodurch deren Zusammenspiel und Auswirkungen auf die Zuverlässigkeit des Gesamtsystems für Entwickler nur eingeschränkt nachvollziehbar sind. Daraus ergibt sich die dieser Arbeit zugrundeliegende Hauptforschungsfrage:

Welche Architekturmechanismen und Entwurfsentscheidungen sind notwendig, um in einem Java/Spring-basierten Job-Processing-Backend definierte Konsistenzzusicherungen unter typischen Teilausfallszenarien nachweisbar einzuhalten?

Zur Beantwortung dieser Frage werden folgende Teilfragen untersucht:

- TF1: Welche Zusicherungen muss das System gewährleisten, damit Jobs stets korrekt und ohne Inkonsistenzen verarbeitet werden?
- TF2: Welchen Beitrag leisten Idempotenz, Retry-Strategien und Recovery-Mechanismen zur Robustheit unter definierten Fehlerszenarien?
- TF3: Wie müssen Transaktionen und Locking-Strategien genutzt werden, damit bei paralleler Arbeit mehrerer Worker keine Doppelverarbeitung oder inkonsistenten Zustände entstehen?
- TF4: Wie lässt sich sicherstellen, dass Jobs nach einem unerwarteten Prozessabbruch korrekt wiederaufgenommen werden, ohne dass Aufträge verloren gehen?

1.3 Zielsetzung

Ziel dieser Arbeit ist der Entwurf, die prototypische Implementierung und die Evaluation eines fehlertoleranten Job-Processing-Backends auf Basis einer relationalen Datenbank in Kombination mit Java und Spring. Dabei soll herausgearbeitet werden, welche Architekturmechanismen erforderlich sind, damit Hintergrundjobs auch bei Teilausfällen zuverlässig und korrekt verarbeitet werden und definierte Konsistenzzusicherungen eingehalten werden.

Konkret verfolgt die Arbeit folgende Teilziele:

- TZ1: Definition von Zusicherungen: Es sollen präzise Korrektheitsbedingungen formuliert werden, die das System einhalten muss, unabhängig davon, welche Fehler auftreten. Dazu zählt insbesondere, dass kein Job doppelt erfolgreich abgeschlossen wird und dass kein Job dauerhaft verloren geht.
- TZ2: Architekturentwurf: Es soll eine Architektur entworfen werden, die die hergeleiteten Zusicherungen umsetzt. Architekturentscheidungen sollen explizit dokumentiert und begründet werden.
- TZ3: Prototypische Implementierung: Die entworfene Architektur soll als funktionsfähiger Prototyp umgesetzt werden, der die definierten Zusicherungen unter realistischen Bedingungen einhält.
- TZ4: Evaluation unter Fehlerbedingungen: Es soll nachgewiesen werden, dass das System seine Zusicherungen auch bei gezielt herbeigeführten Teilausfällen nicht verletzt.

1.4 Hypothesen

Auf Basis der Forschungsfrage und der Zielsetzung werden folgende Hypothesen formuliert, deren Überprüfung im Rahmen der Evaluation erfolgt.

- H1: Durch die Kombination eines persistenten Statusmodells mit transaktionsbasiertem Locking und Idempotenz-Mechanismen lässt sich ein System entwerfen, das auch bei unerwarteten Prozessabbrüchen und Teilausfällen keine Jobs verliert und keine doppelten Ergebnisse erzeugt.
- H2: Retry-Strategien mit exponentiellem Backoff in Verbindung mit einer Fehlerklassifikation (transient vs. permanent) erhöhen die Robustheit des Systems gegenüber kurzzeitigen Infrastrukturausfällen, ohne dabei die definierten Zusicherungen zu verletzen.
- H3: Die definierten Zusicherungen lassen sich durch automatisierte Tests mit gezielter Fehlerinjektion reproduzierbar überprüfen und nachweisen.

1.5 Aufbau der Arbeit

Kapitel 2 legt die theoretischen Grundlagen der Arbeit. Es führt zunächst die Grundkonzepte asynchroner Verarbeitung und des Background-Job-Processings ein, bevor die für Nebenläufigkeit und Fehlertoleranz zentralen Konzepte wie Atomicity, Consistency, Isolation, Durability (ACID), Synchronisationsverfahren, Fehlerklassifikation, Retry-Strategien, Recovery und Idempotenz vorgestellt werden. Das Kapitel schließt mit einer Einordnung des Stands der Technik, aus der sich die in Kapitel 1.2 skizzierte Lücke ergibt, die die vorliegende Arbeit füllt.

Kapitel 3 beschreibt das methodische Vorgehen der Arbeit. Dazu zählt das Vorgehen bei der Literaturrecherche, die Erfassung der Anforderungen sowie die daraus schrittweise Ableitung von Zusicherungen, als auch die Methodik der späteren Evaluation. Dort werden insbesondere das Vorgehen zur Operationalisierung der Zusicherungen und die verwendeten Test- und Nachweismethoden betrachtet.

Kapitel 4 legt die Anforderungen an das zu entwickelnde System dar. Nach einer Beschreibung des Systemkontexts wird das betrachtete Fehlermodell explizit abgegrenzt. Darauf aufbauend werden die Zusicherungen Z-1 bis Z-9 hergeleitet, die als verbindlicher Maßstab für Entwurf und Evaluation dienen.

Kapitel 5 beschreibt den Entwurf und die prototypische Implementierung des Systems. Es begründet die zentralen Architekturentscheidungen, darunter die Verwendung einer relationalen Datenbank als Job-Store, das Zustandsmodell sowie die Mechanismen für Nebenläufigkeitskontrolle, Retry und Recovery und dokumentiert deren spezifische Umsetzung in Java und Spring.

Kapitel 6 evaluiert das implementierte System. Ausgehend von der in Tabelle 6.1 dokumentierten Testmatrix werden gezielt Fehler in das System injiziert und die resultierenden Systemzustände gegen die definierten Prüfkriterien geprüft. Die Ergebnisse werden anschließend zur Bewertung der in Kapitel 1.4 formulierten Hypothesen herangezogen.

Kapitel 7 diskutiert die gewonnenen Erkenntnisse sowie die Grenzen und Einschränkungen der Arbeit, insbesondere hinsichtlich des in Kapitel 4.2 bewusst eingeschränkten Fehlermodells. Des Weiteren gibt das Kapitel einen Ausblick auf mögliche Erweiterungen, besonders hinsichtlich Skalierbarkeit und Performance unter höherer Last.

2 Grundlagen

[TODO: Unterschriften von Abbildungen etc. sollten selbstlesend sein -> nochmal in gesamter Arbeit überarbeiten] Dieses Kapitel vermittelt die theoretischen Grundlagen, die für das Verständnis der im weiteren Verlauf entwickelten Architektur erforderlich sind. Zunächst werden die grundlegenden Konzepte der Hintergrundverarbeitung sowie die Anforderungen an zuverlässige Background-Job-Processing-Systeme erläutert. Darauf aufbauend werden die für diese Arbeit relevanten Mechanismen zur Gewährleistung von Konsistenz bei paralleler Verarbeitung sowie Konzepte zur Fehlertoleranz, Wiederherstellung und idempotenten Verarbeitung eingeführt. Die dargestellten Grundlagen bilden die Basis für den in Kapitel 5.1 vorgestellten Architekturentwurf und dessen anschließende Implementierung und Evaluation.

2.1 Grundlagen der Hintergrundverarbeitung

Hintergrundverarbeitung dient dazu, zeitintensive oder ressourcenintensive Aufgaben vom synchronen Request-Response-Zyklus zu entkoppeln. Dadurch können Anfragen frühzeitig beantwortet werden, ohne den Aufrufer lange zu blockieren, während die eigentliche Verarbeitung unabhängig vom Lebenszyklus der ursprünglichen Anfrage fortgesetzt wird. Gleichzeitig entstehen daraus jedoch auch einige Herausforderungen: In verteilten Umgebungen sind Teilausfälle, Timeouts und kurzzeitige Unterbrechungen üblich, sodass ohne entsprechende Mechanismen nicht davon ausgegangen werden kann, dass eine angestoßene Verarbeitung vollständig und genau einmal ausgeführt wird. Insbesondere kann bei Kommunikationsabbrüchen oder Wiederholungsversuchen unklar bleiben, ob eine Operation bereits verarbeitet wurde oder ob sie erneut angestoßen werden muss (Kleppmann 2017, Kap. 8). Vor diesem Hintergrund werden in den folgenden Abschnitten grundlegende Konzepte vorgestellt, die eine Entkopplung der Verarbeitung ermöglichen und die Basis für robuste Hintergrundverarbeitung bilden.

2.1.1 Asynchrone Verarbeitung und Queueing

In Backend-Systemen lässt sich Kommunikation und Verarbeitung bezüglich der zeitlichen Kopplung zwischen synchronem und asynchronem Verhalten unterscheiden. Der Unterschied der Funktionsweise wird in Abbildung 2.1 deutlich. Bei synchroner Verarbeitung nach dem Request-Response-Stil wartet der Aufrufer auf eine Antwort auf die Anfrage und blockiert gegebenenfalls, bevor er fortfährt. Bei asynchroner Verarbeitung wird die Antwort vom Zeitpunkt der Auftragserstellung entkoppelt. Der Aufrufer initiiert die Arbeit, blockiert jedoch nicht und kann auch während des Wartens auf eine Antwort weiter arbeiten (Richardson 2018, Kap. 3.1.1).

Zur praktischen Umsetzung asynchroner Verarbeitung werden häufig Puffer eingesetzt. Dieses Prinzip wird als *Queueing* bezeichnet. Dadurch können Nachrichtenverluste aufgrund von Überlastung reduziert und Lastspitzen ausgeglichen werden, indem der Puffer erhaltene Anfragen zur späteren Ausführung zwischenspeichert (Kleppmann 2017,

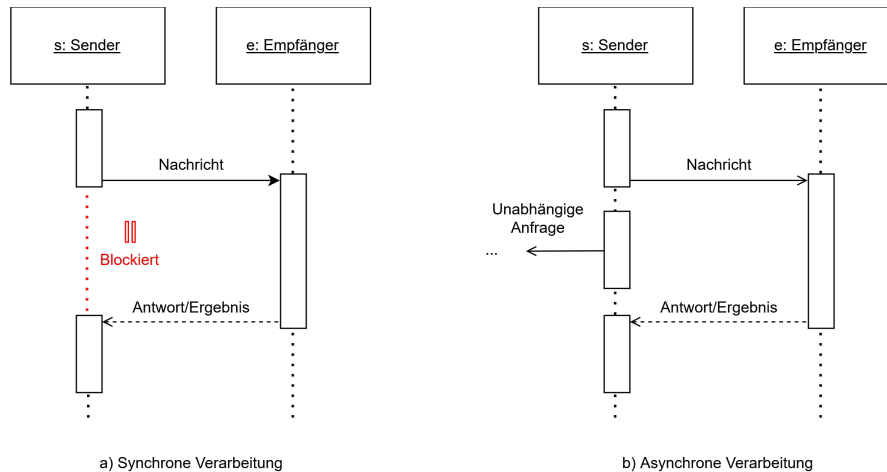


Abbildung 2.1: Vergleich synchroner und asynchroner Kommunikation

Kap. 11, Abschnitt „Message brokers“). Das grundlegende Prinzip ist in Abbildung 2.2 schemenhaft dargestellt. Ein solcher Puffer kann verschiedene Ausprägungen annehmen,

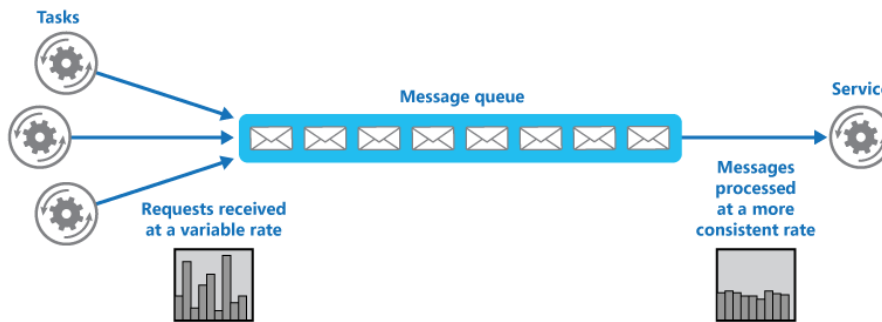


Abbildung 2.2: Queueing (Microsoft Corporation (Hrsg.) 2026b)

beispielsweise als In-Memory beim Empfänger, als Datenbank oder in Form eines Message Brokers. Ein Produzent sendet eine Nachricht an einen Zwischenspeicher und wartet in der Regel lediglich auf das Acknowledgment, dass die Nachricht zwischengespeichert wurde, nicht jedoch auf die Verarbeitung oder das Ergebnis (Kleppmann 2017, Kap. 11, Abschnitt „Messaging Systems“). Die so erreichte Entkopplung ermöglicht es, Anfragen unabhängig von ihrer späteren Verarbeitung zunächst zwischenzuspeichern. Produzent und Konsument müssen daher weder gleichzeitig aktiv sein noch während der gesamten Verarbeitung miteinander kommunizieren. Auf dieser Entkopplung basieren auch Message-Broker-Systeme. Falls ein Konsument temporär nicht verfügbar ist oder während der Verarbeitung ausfällt, können Message Broker trotzdem für die At-Least-Once-Semantik (vgl. Kapitel 2.1.2) sorgen (Kleppmann 2017, Kap. 11, Abschnitt „Acknowledgments and redelivery“). Des Weiteren müssen Verarbeitungskapazitäten in Folge von Queueing nicht auf die maximal zu erwartende Last ausgelegt werden, sondern können sich an der

durchschnittlichen Auslastung orientieren. Kurzzeitige Lastspitzen werden dabei durch die Zwischenspeicherung der Aufgaben in der Queue ausgeglichen (Microsoft Corporation (Hrsg.) 2026b). Die synchrone Request-Response-Kommunikation bietet demgegenüber den Vorteil einer unmittelbaren Rückmeldung über Erfolg oder Fehler einer Operation, setzt jedoch voraus, dass Client und Server während der gesamten Interaktion verfügbar sind und die Kommunikation bis zur Übermittlung der Antwort aufrechterhalten wird (Richardson 2018, Kap. 3.4).

Die asynchrone Verarbeitung bildet damit die Grundlage für die Hintergrundverarbeitung, indem die Entkopplung genutzt wird, um Aufgaben unabhängig vom ursprünglichen Aufruf auszuführen und gleichzeitig die Robustheit und Skalierbarkeit des Systems zu erhöhen (Newman 2026, Kap. 8, Abschnitt „Why Use A Message Broker?“).

2.1.2 Ausführungssemantiken

In verteilten Systemen führen potenzielle Prozessabstürze und Teilausfälle dazu, dass Aufträge verloren gehen oder der Verarbeitungszustand einer Aufgabe unklar sein kann. Um zu beschreiben, was Systeme aufgrund ihres Entwurfs und ihrer Mechanismen zur Fehlertoleranz bezüglich Auslieferung beziehungsweise Ausführung garantieren, werden grundsätzlich drei fundamentale Ausführungssemantiken unterschieden, die Newman (2026, Kap. 8 Abschnitt „Types of Delivery Guarantees“) vor dem Hintergrund der Auslieferung wie folgt definiert. Im Kontext von Job-Processing sind diese Semantiken zur Nachrichtenzustellung analog auf die Ausführung der Jobs anwendbar.

At-Least-Once

Ein Konsument erhält die Nachricht mindestens einmal. Bei der Auslieferung bestätigt der Konsument über ein Acknowledge das Ankommen der Nachricht. Sofern das Acknowledgement beim Absender eingeht, ist für diesen garantiert, dass die Nachricht angekommen ist. Bleibt diese Bestätigung innerhalb eines definierten Zeitfensters jedoch aus, z.B. weil die Nachricht oder das Acknowledgement verloren geht, wird die Nachricht erneut zugestellt. Dieser Fall wird *Redelivery* genannt. Dieses Verfahren verhindert Datenverlust, birgt jedoch das Risiko von Duplikaten, für den Fall, dass die Nachricht ankommt, aber das Acknowledge verloren geht und es infolgedessen zur Redelivery kommt. Einschränkend sei hier gesagt, dass die Zustellung nicht garantiert werden kann, wenn der Konsument bei keinem Zustellversuch verfügbar ist.

At-Most-Once

Ein Konsument erhält die Nachricht maximal einmal. Der Produzent schickt die Nachricht ab und erwartet keine Empfangsbestätigung. Eine erneute Zustellung findet unter keinen Umständen statt. Dadurch werden Duplikate strikt ausgeschlossen, jedoch muss ein potenzieller Datenverlust zugunsten besserer Latenzen hingenommen werden.

Exactly-Once

Diese Semantik garantiert, dass eine Nachricht genau einmal beim Konsumenten ankommt. In der Praxis ist das nur schwer zu realisieren, da beispielsweise nicht klar sein kann, ob ein Acknowledgement nur noch nicht angekommen, oder schon verloren gegangen ist. Viele Umsetzungen beruhen daher auf der Notwendigkeit, dass sowohl beim Produzenten, als auch Konsumenten entsprechende Mechanismen implementiert werden. Eine Variante zur Umsetzung von Exactly-Once ist die Approximation über eine Kombination der Implementierung von At-Least-Once beim Produzent und Idempotenz beim Konsument. Die Garantie bezieht sich somit nicht auf die tatsächliche Anzahl der Zustellungen einer Nachricht, sondern auf die Berücksichtigung durch den Konsumenten. Eine Nachricht kann mehrfach zugestellt werden, solange sie nur einmal angenommen wird.

2.1.3 Background-Job-Processing

Aufbauend auf den Konzepten der asynchronen Verarbeitung (vgl. Kapitel 2.1.1) bezeichnet das Background-Job-Processing die Ausführung von Aufgaben außerhalb des Request-Response-Zyklus. Hierbei werden fachliche Operationen als eigenständige Arbeitseinheiten (Jobs) modelliert und zeitlich entkoppelt verarbeitet.

In der Softwarearchitektur existieren verschiedene etablierte Entwurfsmuster zur Umsetzung einer solchen Infrastruktur, wie beispielsweise das Queue-Based Load Leveling Pattern (Microsoft Corporation (Hrsg.) 2026b). Dieses Muster besteht ähnlich des Konzepts aus Abbildung 2.2 aus drei funktionalen Kernkomponenten:

- **Der Produzent:** Erzeugt einen Job und gibt diesen weiter an den Job-Store. Das kann zum einen direkt durch einen Service geschehen, oder beispielsweise auch über den Aufruf eines entfernten Endpoints, der den Job über eine synchrone Schnittstelle, wie z.B. Representational State Transfer (REST), entgegennimmt. In letzterem Fall fungiert der Service hinter dem Endpoint als Produzent. Dessen Aufgabe besteht darin, den Auftrag zu validieren und an die Persistenzschicht zu übergeben, woraufhin der synchrone Request-Response-Zyklus für den Client sofort beendet wird. Jobs können aber nicht nur durch solche Event-driven Trigger ausgelöst werden, sondern auch durch Schedule-driven Trigger. Dabei erfolgt die Ausführung eines Jobs periodisch oder zu vordefinierten Zeitpunkten, z.B. durch einen Cron-Trigger (Microsoft Corporation (Hrsg.) 2026a).
- **Der Job-Store:** Speichert die Jobs für den Zeitraum von der Erstellung bis mindestens zur endgültigen Beendigung zwischen. Für die Umsetzung existieren verschiedene Varianten, beispielsweise als Message Broker oder relationale Datenbank. Diese Realisierungsmöglichkeiten bieten jeweils spezifische Vor- und Nachteile in der Nutzung. Die Auswahl des passenden Ansatzes sollte daher durch die Qualitätsanforderungen an das System, wie z.B. Durchsatz oder Akzeptanz einzelner verlorener Jobs, begründet werden (Kleppmann 2017, Kap. 11).
- **Der Konsument:** Autonome Komponente, die die Fachlogik des hinterlegten Jobs ausführt. Sobald der Konsument über freie Systemressourcen verfügt, kann er einen

wartenden Job aus dem Job-Store beanspruchen und führt diesen aus (Microsoft Corporation (Hrsg.) 2026b).

Durch eine solche Architektur können Background-Jobs als „fire and forget“-Operationen (Microsoft Corporation (Hrsg.) 2026a) implementiert werden. Sie laufen asynchron in einem vom Client isolierten Prozess, sodass deren Verarbeitungszeit keine unmittelbare Auswirkung auf die Reaktionszeit oder Verfügbarkeit des Produzenten hat (Microsoft Corporation (Hrsg.) 2026a). Außerdem ermöglicht dieses Konzept das Abfangen von Lastspitzen und flexiblere Skalierung (Microsoft Corporation (Hrsg.) 2026b).

Die Zuweisung von Jobs an Konsumenten kann grundsätzlich über unterschiedliche Modelle erfolgen. Beim Push-Modell weist eine zentrale Komponente Aufgaben aktiv freien Konsumenten zu. Beim Pull-Modell fragen Konsumenten eigenständig neue Aufgaben aus dem Job-Store ab. Beide Ansätze unterscheiden sich hinsichtlich Komplexität, Skalierbarkeit und Koordinationsaufwand (Kleppmann 2017, Kap. 11 Abschnitt „Transmitting Event Streams“).

Werden mehrere Konsumenten parallel betrieben, entsteht die Herausforderung der konkurrierenden Verarbeitung. Ohne geeignete Koordinationsmechanismen könnten mehrere Instanzen denselben Job gleichzeitig beanspruchen oder widersprüchliche Statusänderungen durchführen. Die hierfür relevanten Konzepte werden in Kapitel 2.2 behandelt.

Da die Verarbeitung zeitlich entkoppelt erfolgt, müssen Informationen über den Verarbeitungszustand über den Lebenszyklus einzelner Prozesse hinweg erhalten bleiben. Hierfür ist eine persistente Speicherung des Zustands erforderlich. Insbesondere in verteilten Systemen können Teilausfälle auftreten, bei denen einzelne Komponenten oder Prozesse bei der Ausführung ausfallen, während andere Teile des Systems weiterhin verfügbar bleiben. Damit eine unterbrochene Verarbeitung nach einem solchen Ausfall wieder aufgenommen werden kann, muss das System den zuletzt bekannten Zustand kennen oder anhand persistenter Informationen rekonstruieren können (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“).

Neben Prozessabstürzen können auch verschiedene Kommunikationsfehler auftreten, wie z.B. die vorübergehende Nichterreichbarkeit externer Dienste. Das System muss daher abhängig von der jeweiligen Fehlersituation entscheiden können, ob eine Verarbeitung erneut durchgeführt, fortgesetzt oder endgültig beendet werden soll (Microsoft Corporation (Hrsg.) 2026a). Die hierfür relevanten Konzepte zur Fehlerbehandlung und Wiederherstellung werden in Kapitel 2.3 vorgestellt.

Da sowohl Wiederholungsversuche als auch Wiederaufnahmen nach einem Ausfall dazu führen können, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden, muss darüber hinaus sichergestellt werden, dass solche Mehrfachausführungen keine inkonsistenten Ergebnisse verursachen. Hierzu werden Verfahren der idempotenten Verarbeitung eingesetzt, die in Kapitel 2.3.4 vorgestellt werden.

2.2 Konsistenz und konkurrierende Verarbeitung

Bei der Verarbeitung von Daten durch mehrere parallel arbeitende Prozesse entsteht die Herausforderung, dass konkurrierende Zugriffe auf gemeinsame Ressourcen zu inkon-

sistenten Zuständen führen können. Ohne geeignete Mechanismen können Änderungen verloren gehen, widersprüchliche Zustände entstehen oder mehrere Prozesse dieselben Daten gleichzeitig verändern (Saake, Heuer und Sattler 2018, Kap. 13.1.2).

Im Kontext von Background-Job-Processing tritt dieses Problem besonders dann auf, wenn mehrere Worker-Instanzen parallel auf denselben Jobbestand zugreifen. Versuchen sie ohne geeignete Synchronisationsmechanismen denselben Job zu beanspruchen oder konkurrierende Statusänderungen durchzuführen, können *Race Conditions* entstehen. Insbesondere bei Pull-basierten Architekturen, bei denen die Jobverteilung nicht durch eine Orchestrierungskomponente realisiert wird, muss eine solche ungewollte Doppelbeanspruchung explizit verhindert werden (Kleppmann 2017, Kap. 7 Abschnitt „Isolation“). Die Gewährleistung konsistenter Zustände stellt daher eine zentrale Anforderung an Systeme mit paralleler Verarbeitung dar. Die hierfür etablierten Konzepte und Mechanismen werden in den folgenden Abschnitten vorgestellt.

2.2.1 ACID

Zur Vereinfachung der Gewährleistung konsistenter Zustände bei parallelen Zugriffen existiert das Konzept der Transaktionen. Diese fassen mehrere Operationen zu einer logischen Einheit zusammen, deren Ausführung nach außen als zusammenhängender Vorgang erscheint (Kleppmann 2017, Kap. 7). Im Kontext persistenter Datenhaltung werden von Transaktionen typischerweise die vier zentralen Eigenschaften ACID gefordert (Saake, Heuer und Sattler 2018, Kap. 13.1.1):

Atomicity Die Atomarität stellt sicher, dass eine Transaktion entweder vollständig oder gar nicht ausgeführt wird. Tritt während der Ausführung ein Fehler auf, werden sämtliche bis dahin vorgenommenen Änderungen verworfen, sodass keine partiellen Zwischenzustände bestehen bleiben.

Consistency Die Konsistenz beschreibt die Eigenschaft, dass eine erfolgreich abgeschlossene Transaktion die Daten von einem gültigen Zustand in einen anderen gültigen Zustand überführt. Dabei müssen definierte Integritätsbedingungen erhalten bleiben. Kleppmann (2017, Kap. 7 Abschnitt „Consistency“) weist darauf hin, dass die Einhaltung solcher fachlichen Invarianten in der Verantwortung der Anwendung liegt und nicht wie die anderen Eigenschaften durch das Datenbanksystem garantiert werden kann.

Isolation Die Isolation gewährleistet, dass parallel ausgeführte Transaktionen sich nicht gegenseitig beeinflussen. Aus Sicht einer Transaktion soll die Ausführung so erscheinen, als würde sie allein auf dem Datenbestand arbeiten. Dadurch können Race Conditions und andere Nebenläufigkeitsprobleme vermieden werden.

Durability Die Dauerhaftigkeit garantiert, dass die Ergebnisse einer erfolgreich abgeschlossenen Transaktion auch nach Systemabstürzen oder anderen Fehlern dauerhaft erhalten bleiben.

Im Kontext von Background-Job-Processing sind insbesondere die Eigenschaften der Atomarität und Isolation von Bedeutung. Während die Atomarität verhindert, dass Statusänderungen eines Jobs nur teilweise durchgeführt werden, bildet die Isolation die Grundlage für die koordinierte Verarbeitung gemeinsamer Datenbestände durch mehrere parallel arbeitende Worker-Instanzen. Die relevanten Konzepte zum Erreichen dieser Eigenschaften werden in den folgenden Abschnitten näher betrachtet.

2.2.2 Transaktionsgrenzen

Während ACID die theoretischen Garantien einer Transaktion definiert, bestimmt die Festlegung von Transaktionsgrenzen, welche konkreten Datenbankoperationen des softwareseitigen Kontrollflusses zu einer unteilbaren, logischen Einheit zusammengefasst werden. Die Festlegung dieser Grenzen steuert, wann eine Datenbanksitzung eine Transaktion initiiert und zu welchem Zeitpunkt die Änderungen final festgeschrieben (Commit) oder im Fehlerfall vollständig verworfen (Rollback) werden (Kudraß 2015, S.250).

Die Wahl geeigneter Transaktionsgrenzen wird zusätzlich dadurch erschwert, dass fachliche und technische Transaktionen nicht zwangsläufig identisch sind. Eine fachliche Transaktion beschreibt eine zusammenhängende Geschäftsoperation, deren Ergebnis aus Sicht der Anwendung als Einheit betrachtet wird. Eine technische Transaktion hingegen bezeichnet die konkrete Transaktion auf der Datenbankebene, welche die ACID-Eigenschaften für die von ihr gekapselten Datenbankoperationen gewährleistet. Während eine fachliche Operation mehrere technische Transaktionen umfassen kann, können innerhalb einer technischen Transaktion auch nur Teilaspekte einer fachlichen Operation verarbeitet werden. Werden die Grenzen falsch gesetzt, können unvollständige Ergebnisse festgeschrieben werden, die bei einem späteren Fehlschlagen der fachlichen Transaktion nicht mehr durch ein Rollback rückgängig gemacht werden können. Zudem können im Mehrbenutzerbetrieb verschiedene Parallelitätsanomalien auftreten, wenn die Zugriffe der verschiedenen Transaktionen nicht passend koordiniert werden (Saake, Heuer und Sattler 2018, Kap. 13.1.2). Ein verbreiteter Ansatz zur Vermeidung dieser Anomalien ist die Verwendung von Sperren, auf die in Kapitel 2.2.3 näher eingegangen wird. Wichtig ist an dieser Stelle, dass die gewählten Transaktionsgrenzen direkten Einfluss auf die Dauer von Sperren und somit auch auf parallel ausgeführte Transaktionen haben.

Eine Möglichkeit zur Definition von Transaktionsgrenzen besteht darin, jede Datenbankoperation in einer eigenen Transaktion auszuführen. Dadurch werden Sperren nur für sehr kurze Zeit gehalten, was die Parallelität und den Durchsatz des Systems erhöht. Allerdings können zusammengehörige Änderungen nicht mehr atomar ausgeführt werden. Tritt zwischen mehreren aufeinanderfolgenden Operationen ein Fehler auf, können bereits festgeschriebene Änderungen nicht mehr gemeinsam zurückgesetzt werden. Die betroffenen Daten können dadurch in einen fachlich falschen Zustand überführt werden. Als naiver Gegenentwurf können die Transaktionsgrenzen entsprechend der fachlichen Transaktion gesetzt werden. Diese Strategie kann jedoch zu langlaufenden Transaktionen führen, welche die Leistungsfähigkeit des Datenspeichers beeinträchtigt. Zum einen halten Transaktionen im Beispiel des 2-Phasen-Sperrprotokolls sämtliche während ihrer Laufzeit akquirierten Sperren bis zum endgültigen Commit oder Rollback aufrecht (Kleppmann

2017, Kap. 7 Abschnitt „Two-Phase Locking (2PL)“). Dadurch steigt die Wahrscheinlichkeit, dass konkurrierende Transaktionen aufeinander warten müssen. Zum anderen können langlaufende Transaktionen die Bereinigung alter Datenversionen verzögern, da diese weiterhin für aktive Transaktionen sichtbar bleiben müssen (The PostgreSQL Global Development Group (Hrsg.) 2026, Kap. 24.1.2). Dies erhöht den Ressourcenbedarf des Datenbanksystems und kann dessen Leistungsfähigkeit beeinträchtigen.

Im Software-Entwurf erfordert das Setzen der Transaktionsgrenzen daher eine kontinuierliche Abwägung zwischen der Systemperformance und dem Schutz vor Datenanomalien. Weit gefasste Transaktionsgrenzen reduzieren das Risiko der Entstehung von Inkonsistenzen, blockieren jedoch Systemressourcen und Datensätze. Eng gefasste Grenzen erhöhen den Durchsatz, können jedoch die atomare Ausführung fachlicher Abläufe aufbrechen und so Datenanomalien begünstigen.

2.2.3 Synchronisationsverfahren und Isolationsstufen

Zur Umsetzung der Isolationseigenschaft von Transaktionen müssen konkurrierende Zugriffe auf gemeinsame Daten koordiniert werden. Eine verbreitete Strategie zur Durchsetzung der Isolation ist die pessimistische Synchronisation. Sie basiert auf der Annahme, dass konkurrierende Zugriffe auftreten können und verhindert diese bereits im Vorfeld durch Sperrmechanismen. Bevor eine Transaktion einen Datensatz verändert, wird dieser für andere konkurrierende Transaktionen gesperrt. Der Vorteil pessimistischer Verfahren besteht darin, dass Konflikte bereits vor ihrer Entstehung verhindert werden. Dadurch eignen sie sich insbesondere für Szenarien mit häufigen konkurrierenden Zugriffen. Nachteilig wirken sich jedoch zusätzliche Wartezeiten aus, da Transaktionen auf die Freigabe benötigter Sperren warten müssen. Bei hoher Auslastung kann dies die Parallelität und den Gesamtdurchsatz eines Systems reduzieren. Außerdem entsteht die Gefahr von Deadlocks (Kudraß 2015, S. 253–257).

Alternative Ansätze sind nicht sperrende Verfahren, beispielsweise die optimistische Synchronisation oder das Zeitstempelverfahren. Anstatt Daten vor der Bearbeitung zu sperren, werden bei der optimistischen Synchronisation Transaktionen zunächst ausgeführt und erst beim Schreiben geprüft, ob ein Konflikt entstanden ist. Ist es zwischen zwei oder mehreren Transaktionen zum Konflikt gekommen, müssen einzelne Transaktionen zurückgesetzt und wiederholt werden. Bei dem Zeitstempelverfahren werden Zeitstempel für jede Transaktionen, sowie der Zeitpunkt des letzten Lese- und Schreibzugriffs für jedes Datenbankobjekt verwaltet. Anhand dieser Zeitstempel kann geprüft werden, ob zwei Transaktionen in Konflikt zueinander stehen und ob eine der Transaktionen zurückgesetzt werden muss. Nicht sperrende Verfahren vermeiden die durch Sperren verursachten Wartezeiten und ermöglichen dadurch eine hohe Parallelität. Treten jedoch Konflikte auf, müssen betroffene Operationen wiederholt werden. Sie eignen sich daher insbesondere für Systeme, in denen konkurrierende Änderungen selten vorkommen (Kudraß 2015, S. 257–259).

Welche Synchronisationsstrategie geeignet ist, hängt von den Anforderungen des jeweiligen Systems und der Häufigkeit erwarteter Konflikte ab. Während pessimistische Verfahren Konflikte präventiv verhindern, akzeptieren optimistische Verfahren mögliche Konflikte

und behandeln diese nachträglich. Zusätzlich können Datenbanksysteme die strenge Isolationsforderung von ACID durch unterschiedliche Isolationsstufen abschwächen. Diese Stufen legen fest, in welchem Umfang Transaktionen die Änderungen parallel laufender Transaktionen beobachten und welche Parallelitätsanomalien dadurch im System auftreten können (Kudraß 2015, S. 251). Je nach Anwendungskontext kann es sein, dass nicht alle dieser Anomalien auftreten können, beispielsweise wenn nur Leseoperationen ausgeführt werden, oder dass deren Auftreten in Kauf genommen werden kann, um eine bessere Performance zu erhalten. Höhere Isolationsstufen bieten stärkere Konsistenzgarantien, können jedoch die Parallelität des Systems verringern (Saake, Heuer und Sattler 2018, Kap. 13.1.3).

2.3 Fehlertoleranz und Wiederherstellung

Verteilte Softwaresysteme sind zwangsläufig mit einer Vielzahl potenzieller Fehlerquellen konfrontiert. Prozesse können unerwartet terminieren, Netzwerkverbindungen unterbrochen werden oder externe Dienste zeitweise nicht erreichbar sein (Newman 2026, Kap. 1 Abschnitt „The Three Golden Rules Of Distributed Systems“). Beispielsweise kann es bei einzelnen Teilausfällen dazu kommen, dass unklar bleibt, ob ein Verarbeitungsschritt erfolgreich abgeschlossen wurde oder erneut ausgeführt werden muss. Da sich solche Fehler in komplexen Systemlandschaften nicht vollständig vermeiden lassen, muss ihre Existenz als grundlegende Systemeigenschaft betrachtet werden (Newman 2026, Kap. 1 Abschnitt „Failure Is Inevitable“).

Die Zuverlässigkeit eines Systems ergibt sich daher nicht primär daraus, Fehler vollständig zu verhindern, sondern daraus, wie gut mit ihrem Auftreten umgegangen wird. Fehlertoleranz bezeichnet in diesem Zusammenhang die Fähigkeit eines Systems, seine wesentlichen Funktionen trotz auftretender Störungen weiterhin bereitzustellen. Darüber hinaus muss ein System in der Lage sein, nach einem Fehler einen konsistenten Zustand wiederherzustellen und die Verarbeitung kontrolliert fortzusetzen, beziehungsweise neuzustarten (Newman 2026, Kap. 1 „What Is Resilience?“). Die folgenden Abschnitte stellen die hierfür relevanten Konzepte und Mechanismen vor.

2.3.1 Fehlerklassifikation

Da Fehler in verteilten Systemen unvermeidbar sind, stellt sich nicht die Frage, ob sie auftreten, sondern wie mit ihnen umgegangen werden soll. Die Wirksamkeit von Fehlertoleranzmechanismen hängt dabei wesentlich davon ab, die Ursache und Eigenschaften eines Fehlers korrekt einzuordnen. Nicht jeder Fehler kann durch dieselben Maßnahmen behandelt werden. Während einige Fehler durch eine erneute Ausführung einer Operation behoben werden können, sind andere dauerhaft und erfordern eine Anpassung der Eingabedaten oder eine manuelle Intervention. Die Klassifikation von Fehlern bildet daher die Grundlage für Entscheidungen über Retry-Strategien, Fehlerbehandlung und Wiederherstellungsmechanismen (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Darüber hinaus können nicht alle denkbaren Fehler mit vertretbarem Aufwand behandelt

werden. Während kurzzeitige Infrastrukturprobleme, Prozessabstürze oder Kommunikationsfehler häufig durch geeignete Architekturmechanismen abgefangen werden können, liegen andere Ereignisse, etwa großflächige Naturkatastrophen oder der vollständige Ausfall eines Rechenzentrums, gegebenenfalls außerhalb des Fehlerannahmemodells einer Anwendung. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Entscheidung darüber, welche Fehlerarten berücksichtigt werden und welche Risiken akzeptiert werden müssen (Kleppmann 2017, Kap. 1 Abschnitt „Reliability“).

Eine grundlegende Einteilung erfolgt in *transiente* und *permanente* Fehler. Transiente Fehler treten zeitlich begrenzt auf und verschwinden nach kurzer Zeit wieder. Ursachen liegen häufig in der technischen Infrastruktur eines Systems, beispielsweise in vorübergehenden Netzwerkstörungen, kurzzeitigen Überlastungen von Diensten, nicht erreichbaren Datenbanken oder dem Neustart einzelner Systemkomponenten (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Permanente Fehler hingegen bestehen unabhängig von der Anzahl der Wiederholungsversuche fort. Ihre Ursache liegt häufig in der Anwendung selbst oder in den zu verarbeitenden Daten. Beispiele hierfür sind ungültige Eingabedaten oder fehlende Berechtigungen. Newman (2026, Kap. 3 Abschnitt „Should You Always Retry?“) verdeutlicht dies anhand von HTTP-Fehlercodes wie 404 Not Found oder 403 Forbidden. Die Abgrenzung zwischen beiden Fehlerarten ist insbesondere für die automatisierte Fehlerbehandlung relevant. Während transiente Fehler häufig eine erneute Ausführung rechtfertigen, sollten permanente Fehler möglichst früh erkannt und gesondert behandelt werden.

Eine besondere Rolle spielen zudem Timeout-Fehler. In verteilten Systemen werden Timeouts verwendet, um potenzielle Ausfälle zu erkennen. Dabei kann jedoch nicht eindeutig unterschieden werden, ob tatsächlich ein Fehler vorliegt oder die Antwort lediglich ungewöhnlich lange braucht. Ein Timeout signalisiert daher nur, dass eine erwartete Antwort nicht innerhalb eines definierten Zeitfensters eingetroffen ist und lässt somit keine Aussage über eine Ursache zu (Bass 2021, Kap. 17.2 Abschnitt „Failure in the Cloud“).

2.3.2 Retry-/Backoff-Strategien

Die in Kapitel 2.3.1 eingeführte Unterscheidung zwischen transienten und permanenten Fehlern bildet die Grundlage für den Einsatz von Wiederholungsmechanismen. Da transiente Fehler nur zeitweise die Verarbeitung behindern, kann in solchen Fällen eine erneute, gegebenenfalls verzögerte Ausführung erfolgreich sein, obwohl der ursprüngliche Versuch fehlgeschlagen ist (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“). Retry-Strategien verfolgen damit also das Ziel, die Erfolgswahrscheinlichkeit einer Verarbeitung zu erhöhen, ohne dass ein manueller Eingriff erforderlich wird. Retries sollten ausschließlich für Fehler durchgeführt werden, deren Ursache außerhalb der eigentlichen Geschäftslogik liegt und deren Fortbestand nicht dauerhaft erwartet wird. Permanente Fehler sollten dagegen unmittelbar als endgültig fehlgeschlagen behandelt werden, da eine erneute Ausführung die Erfolgswahrscheinlichkeit nicht erhöht und das System unnötig belastet. Ein typisches Szenario, in dem Retries häufig sinnvoll sind, ist das erneute Verschicken von verloren gegangenen Nachrichten (Newman 2026, Kap. 3

Abschnitt „Should You Always Retry?“; Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability considerations“).

Da die Ursache eines transienten Fehlers potenziell auch für eine gewisse Zeit nach dem Auftreten des Fehlers weiterhin besteht, werden Wiederholungen üblicherweise nicht unmittelbar durchgeführt. Stattdessen wird zwischen den einzelnen Versuchen eine Wartezeit eingefügt. Dieses Vorgehen wird als *Backoff* bezeichnet und soll verhindern, dass wiederholte Fehlversuche zusätzliche Last auf bereits beeinträchtigte Systeme erzeugen. Bekommt eine Anfrage beispielsweise ein Timeout, weil der Server zu viele Anfragen erhält, würden direkte Retries den Server noch weiter mit Anfragen verstopfen. Eine einfache Variante besteht darin, zwischen allen Wiederholungsversuchen dieselbe Wartezeit zu verwenden. In verteilten Systemen wird jedoch häufig ein exponentieller Backoff eingesetzt, bei dem sich das Warteintervall nach jedem fehlgeschlagenen Versuch schrittweise erhöht. Dadurch erhält das betroffene System mehr Zeit, sich von einer temporären Störung zu erholen, da die Dichte der Wiederholungsversuche reduziert wird (Newman 2026, Kap. 3 Abschnitt „Delays Between Retries“).

Neben der zeitlichen Staffellung von Wiederholungsversuchen muss deren Anzahl begrenzt werden. Andernfalls besteht die Gefahr, dass ein fehlerhafter Job unendlich oft ausgeführt wird und so dauerhaft Ressourcen belegt. In der Praxis wird daher eine maximale Anzahl von Wiederholungsversuchen definiert. Wird diese überschritten, gilt die Verarbeitung als endgültig fehlgeschlagen und der Job wird in einen Fehlerzustand überführt (Newman 2026, Kap. 3 Abschnitt „How Many Retries Are Appropriate?“). Abbildung 2.3 visualisiert das oben beschriebene Vorgehen.

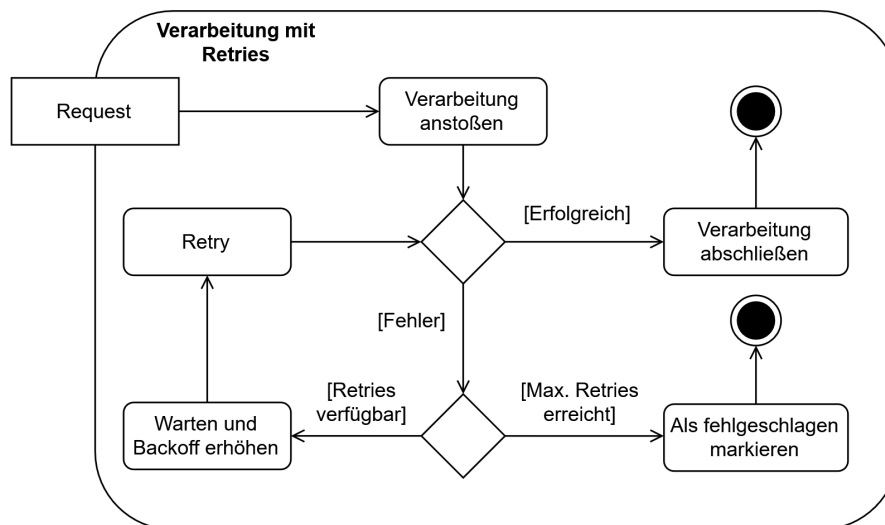


Abbildung 2.3: Verhalten bei transienten Fehlern

Retry- und Backoff-Strategien bilden damit einen wesentlichen Baustein zur Behandlung transienter Fehler. Sie ermöglichen die automatische Wiederholung fehlgeschlagener Verarbeitungen und versuchen gleichzeitig die Belastung der betroffenen Systeme durch das Vermeiden unmittelbar aufeinanderfolgender Wiederholungsversuche möglichst gering zu

halten.

2.3.3 Recovery

Neben transienten Fehlern, die häufig durch Retry-Mechanismen behandelt werden können, können auch Fehler auftreten, die den Ausführungsprozess einer Verarbeitung vollständig unterbrechen. Da einzelne Komponenten unabhängig voneinander ausfallen können, während andere Teile des Systems weiterhin funktionieren, entstehen sogenannte Teilausfälle (Kleppmann 2017, Kap. 8 Abschnitt „Faults and Partial Failures“). Diese treten nichtdeterministisch auf, wodurch Unsicherheit über den tatsächlichen Verarbeitungszustand einer Aufgabe entstehen kann. Gegebenenfalls ist nicht eindeutig feststellbar, ob eine Verarbeitung bereits erfolgreich abgeschlossen wurde, aber die Bestätigung darüber fehlt, ob sie kurz vor ihrem Abschluss steht oder aufgrund eines Fehlers vorzeitig abgebrochen wurde (Kleppmann 2017, Kap. 8). Besondere Relevanz besitzen Ausfälle ausführender Komponenten während einer laufenden Bearbeitung. In einem solchen Szenario kann kein lokaler Retry mehr initiiert werden, da die ausführende Komponente selbst nicht mehr verfügbar ist. Recovery bezeichnet in diesem Kontext die Fähigkeit eines Systems, eine derart unterbrochene Verarbeitung zu erkennen und geeignete Maßnahmen zur Wiederaufnahme einzuleiten (Bass 2021, Kap. 4.2).

Voraussetzung für eine Wiederaufnahme ist zunächst die Erkennung unterbrochener Verarbeitungen. Wird eine Verarbeitung nach ihrer Übernahme durch eine ausführende Komponente unterbrochen, kann ein Zustand entstehen, in dem die Aufgabe aus Sicht des Systems weiterhin als aktiv gilt, obwohl tatsächlich keine Bearbeitung mehr stattfindet. Zur Erkennung solcher Situationen werden zusätzlich zu einem Statusmodell Heartbeat- oder Lease-/Timeout-Mechanismen benötigt (Bass 2021, Kap. 17.2 Abschnitt „Failure in the Cloud“). Bei Heartbeats sendet die ausführende Komponente in regelmäßigen Abständen Signale, anhand derer ihre Aktivität überwacht werden kann. Bleiben diese Signale über einen definierten Zeitraum aus, wird ein Ausfall vermutet und die Verarbeitung als potenziell unterbrochen betrachtet (Bass 2021, Kap. 4.2 Abschnitt „Detect Faults“). Lease- und Timeout-Mechanismen definieren lediglich eine Zeitspanne bzw. einen Zeitpunkt, bis zu dem eine Verarbeitung erlaubt ist. Beendet die ausführende Komponente die Verarbeitung nicht innerhalb dieses Rahmens, wird der Job als fehlgeschlagen interpretiert. Ist die Zeitspanne nicht passend konfiguriert, werden daher auch Jobs abgebrochen, die zu lange gebraucht haben, auch wenn bei deren Verarbeitung kein Fehler aufgetreten ist.

Nach der Erkennung eines Ausfalls muss die Verarbeitung erneut angestoßen werden. Hierfür wird die betroffene Aufgabe durch Zurücksetzen des Status wieder freigegeben. Die nächste Ausführungskomponente mit freien Ressourcen kann diesen Job wieder beanspruchen. Dadurch entsteht jedoch die Möglichkeit, dass der betroffene Job mehrfach ausgeführt und gespeichert wird, falls die ursprüngliche Ausführungskomponente nicht ausgefallen ist, sondern nur lange braucht. Weitere Details werden in Kapitel 2.3.4 erläutert.

Für die Wiederaufnahme existieren unterschiedliche Ansätze. Dabei wird zwischen flüchtigem und persistentem Zustand unterschieden. Während lokal im Arbeitsspeicher gehaltene

Informationen beim Absturz eines Prozesses verloren gehen, kann persistent gespeicherter Zustand auch nach einem Ausfall weiterhin genutzt werden (Kleppmann 2017, Kap. 11 Abschnitt „Rebuilding state after a failure“). Die Verarbeitung auf Basis eines Zwischenzustands wieder aufzunehmen ist unter Umständen jedoch nicht trivial. Eine verbreitete Wiederherstellungsstrategie besteht daher darin, die für eine Verarbeitung relevanten Eingabedaten bis zum erfolgreichen Abschluss des Jobs zu speichern und eine unterbrochene Verarbeitung bei Bedarf auf Basis dieser Ursprungsdaten erneut auszuführen. Dies ist häufig einfacher und robuster als der Versuch, den exakten Laufzeitzustand eines abgestürzten Prozesses zu rekonstruieren und die Verarbeitung dort fortzusetzen (Bass 2021, Kap. 17.3). Welcher Ansatz gewählt wird, hängt unter anderem von der Komplexität der Verarbeitung sowie vom Aufwand einer vollständigen Neuausführung ab. Unabhängig von Fortsetzung oder Neustart darf ein Ausfall nicht zu inkonsistenten Zuständen führen, die eine spätere Wiederaufnahme verhindern bzw. das Ergebnis beeinflussen. Daher muss sichergestellt werden, dass keine falschen oder unvollständigen Ergebnisse und Zustandsinformationen persistiert werden und dass die Speicherung immer atomar geschieht (Bass 2021, Kap. 4.2 Abschnitt „Recover from Faults“).

Die in diesem Kapitel beschriebene Wiederaufnahme führt zwangsläufig dazu, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden können. Da dabei gegebenenfalls nicht eindeutig festgestellt werden kann, ob eine vorherige Ausführung bereits teilweise oder vollständig erfolgreich war, müssen Systeme auf mögliche Mehrfachausführungen vorbereitet sein. Hieraus ergibt sich die Notwendigkeit idempotenter Verarbeitung, die im folgenden Abschnitt näher betrachtet wird.

2.3.4 Idempotenz

Die in den vorherigen Abschnitten beschriebenen Retry- und Wiederherstellungsmechanismen verfolgen eine At-Least-Once-Semantik und erhöhen dadurch die Zuverlässigkeit eines Systems, führen jedoch gleichzeitig dazu, dass einzelne Verarbeitungsschritte mehrfach ausgeführt werden können. Um trotz solcher Mehrfachausführungen konsistente Datenbestände sicherzustellen, wird das Konzept der Idempotenz eingesetzt. Eine Operation wird als idempotent bezeichnet, wenn ihre mehrmalige Ausführung denselben fachlichen Zustand erzeugt wie eine einmalige Ausführung (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Wiederholte Aufrufe verändern das Ergebnis somit nicht, wodurch Duplikate oder erneut ausgeführte Verarbeitungsschritte keine inkonsistenten Zustände verursachen. Wie in Kapitel 2.1.2 dargelegt, stellt Idempotenz damit eine wesentliche Voraussetzung dar, um auf Basis einer At-Least-Once-Semantik eine Exactly-Once-Semantik zu erreichen.

Idempotenz kann dabei auf unterschiedlichen Ebenen eines Systems umgesetzt werden. In Bezug auf Job-Processing muss bereits die Aufnahme eines Jobs in den Job-Store idempotent gestaltet werden, um zu verhindern, dass derselbe Job mehrfach im System erfasst wird. Ein verbreiteter Ansatz zur idempotenten Annahme von Anfragen ist die Implementierung eines idempotenten Receivers (Hohpe und Woolf 2005, Kap. 10 Abschnitt „Idempotent Receiver“) unter Verwendung eindeutiger Identifikatoren, wie beispielsweise eines Universally Unique Identifier (UUID). Bei einer möglichen Variante

wird jeder Anfrage vom Sender eine eindeutige Kennung hinzugefügt, die vom Empfänger zusammen mit der Anfrage persistent gespeichert wird. Geht dieselbe Anfrage erneut ein, kann das System anhand dieser UUID erkennen, dass die Anfrage bereits verarbeitet wurde und eine redundante Speicherung verhindern (Newman 2026, Kap. 3 Abschnitt „Client Request IDs“; Richardson 2018, Kap. 3.3.6).

Darüber hinaus muss auch die eigentliche Verarbeitung eines Jobs so gestaltet werden, dass wiederholte Ausführungen und Wiederaufnahmen keine inkonsistenten Ergebnisse erzeugen (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Um eine Idempotente Verarbeitung sicherzustellen, darf ein Ergebnis nur einmal erfolgreich persistiert werden. Außerdem muss das Speichern des Ergebnisses atomar ausgeführt werden. Datenänderungen müssen folglich über klar definierte Transaktionsgrenzen sowie ein Statusmodell abgesichert werden, um eine konsistente Weiterbearbeitung nach einem Crash zu gewährleisten (Kleppmann 2017, Kap. 7). Das Statusmodell garantiert, dass ausschließlich nicht fertige Jobs Ergebnisse speichern dürfen, während das Transaktionsmodell verhindert, dass nur Teile von Ergebnissen gespeichert werden.

Idempotenz bildet somit die Grundlage dafür, dass Retry- und Recovery-Mechanismen eingesetzt werden können, ohne die Konsistenz der verarbeiteten Daten zu gefährden. Sie stellt einen zentralen Baustein fehlertoleranter Background-Job-Processing-Systeme dar und ermöglicht die korrekte Verarbeitung trotz unvermeidbarer Mehrfachausführungen und Duplikate.

2.4 Stand der Technik

Die Verarbeitung von Hintergrundaufgaben ist ein etabliertes Problemfeld verteilter Systeme. Entsprechend existieren zahlreiche Frameworks und Architekturmuster, die eine asynchrone Hintergrundaufführung von Aufgaben ermöglichen. Die Ansätze unterscheiden sich insbesondere hinsichtlich der verwendeten Infrastruktur, der Persistenzmechanismen sowie der Umsetzung von Fehlertoleranz und Wiederherstellung. Im Folgenden werden die für diese Arbeit relevanten Lösungsansätze vorgestellt und hinsichtlich ihrer Eigenschaften eingeordnet.

2.4.1 Brokerbasierte Ansätze

Eine weit verbreitete Klasse von Background-Job-Processing-Systemen basiert auf dezentralisierten Message Brokern wie RabbitMQ (Broadcom Inc. (Hrsg.) 2026a) oder Apache Kafka (Apache Software Foundation (Hrsg.) 2026). Hierbei werden Aufgaben in Form von Nachrichten an einen Broker übergeben, der die Entkopplung zwischen Produzenten und Konsumenten übernimmt. Worker beziehen die Nachrichten über den Broker und führen die zugehörige Verarbeitung aus. Der Broker läuft als eigenständige Middleware-Komponente entweder nativ oder virtuell als Server. Der Vorteil dieses Ansatzes liegt in der hohen Skalierbarkeit und der klaren Trennung zwischen Anwendungslogik und Nachrichteninfrastruktur. Darüber hinaus stellen moderne Broker Mechanismen zur Fehlertoleranz, Lastverteilung und Ausfallsicherheit bereit. Gleichzeitig entsteht durch

brokerbasierte Architekturen zusätzliche betriebliche Komplexität, da neben der eigentlichen Anwendung weitere Infrastrukturkomponenten aufgesetzt, betrieben und überwacht werden müssen. Des Weiteren ergeben sich Herausforderungen bei der Konsistenz zwischen Anwendungsdatenbank und Message Broker. Problematisch ist beispielsweise die atomare Kopplung von Datenänderungen in der Datenbank und das Verschicken einer Nachricht. Da beide Systeme unabhängig voneinander arbeiten, müssen zusätzliche Mechanismen wie das Transactional Outbox Pattern eingesetzt werden, um Nachrichtenverlust oder Inkonsistenzen zu vermeiden (Richardson 2018, Kap. 3.3.7).

2.4.2 Datenbankbasierte Ansätze

Neben brokerbasierten Lösungen existieren Frameworks, die eine relationale Datenbank als zentralen Job-Store verwenden. Zu bekannten Vertretern dieses Ansatzes zählen unter anderem JobRunr (Rosoco BV (Hrsg.) 2026), Hangfire (Hangfire OÜ (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026b). Anstatt Aufgaben an einen separaten Broker zu übertragen, werden diese als persistente Datensätze in einer Datenbank gespeichert und von Workern direkt ausgelesen.

Die Nutzung einer relationalen Datenbank ermöglicht den Gebrauch der ACID-Eigenschaften, um das Speichern neuer Jobs sowie das Verwalten von Status- und fachlichen Datenänderungen konsistent innerhalb einer gemeinsamen Transaktion zu verarbeiten. In der Praxis verfolgen existierende Frameworks jedoch unterschiedliche Schwerpunkte und adressieren die Problemstellung jeweils aus einer anderen Perspektive. Hangfire garantiert nur eine At-Least-Once Ausführung und überlässt die Umsetzung von Idempotenz der Fachlogik. Während der Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) primär auf zeitgesteuertes Scheduling und clusteringbasierte Koordination fokussiert, stellt Spring Batch eine spezialisierte Infrastruktur für die blockweise Massenverarbeitung bereit, die auf periodische oder zeitgesteuerte Durchläufe ausgelegt ist. Modernere Java-Bibliotheken wie JobRunr und Hangfire integrieren zwar direkt ein automatisiertes Retry-Handling, kapseln die zugrundeliegenden Mechanismen jedoch als Blackbox, wodurch eine feingranulare Kontrolle und die präzise Analyse des Systemverhaltens bei Infrastrukturausfällen erschwert werden. Robuste und langlebige Workflow-Ausführungen wie Temporal (Temporal Technologies Inc. 2026) lösen die Frage nach Fehlertoleranz wiederum über eine dedizierte Engine, erzwingen damit jedoch einen eigenständigen Serverbetrieb und eine damit einhergehende infrastrukturelle Komplexität. Die konkrete Umsetzung der Nebenläufigkeitskontrolle, Wiederherstellung und Fehlerbehandlung ist stark von den jeweiligen Frameworks abhängig. Die zugrundeliegenden Prinzipien bleiben somit hinter der Schnittstelle der Frameworks verborgen und lassen sich nicht ohne Weiteres abstrahieren oder auf eigene Systementwürfe übertragen.

2.4.3 Einordnung und Handlungsbedarf

Die vorgestellten Ansätze zeigen, dass sowohl brokerbasierte als auch datenbankbasierte Architekturen zur Umsetzung von Background-Job-Processing-Systemen geeignet sind. Während bestehende Frameworks bereits umfangreiche Funktionalitäten für Scheduling,

Retry-Mechanismen und Fehlertoleranz bereitstellen, liegt ihr Fokus primär auf der praktischen Nutzung, wodurch die zugrundeliegenden transaktionalen Mechanismen und deren Verhalten in kritischen Ausfallszenarien für den Entwickler oft intransparent bleiben. Insbesondere die Frage, unter welchen Voraussetzungen eine Datenbank gleichzeitig als persistenter Job-Store und Koordinationsmechanismus eingesetzt werden kann, wird in der Literatur nur begrenzt behandelt. Konzepte wie Fehlertoleranz, Recovery, Idempotenz und Nebenläufigkeitskontrolle werden zwar häufig einzeln beschrieben, ihre Integration zu einer konsistenten Gesamtarchitektur bleibt jedoch oftmals implizit.

Die vorliegende Arbeit grenzt sich von bestehenden Frameworks ab, indem sie ein rein datenbankgestütztes Pull-Modell entwirft, die Architekturentscheidungen begründet darlegt und das System bezüglich der angestrebten Fehlertoleranz evaluiert. Gleichzeitig wird durch die Kombination aus einem Statusmodell und der PostgreSQL-Funktionalität `FOR UPDATE SKIP LOCKED` untersucht, wie die Koordination direkt auf der Datenbankebene orchestriert werden kann, ohne einen separaten Message Broker einzusetzen und somit die Systemkomplexität niedrig zu halten.

3 Methodik

Dieses Kapitel beschreibt das methodische Vorgehen zur Beantwortung der in Kapitel 1.2 formulierten Forschungs- und Teilfragen. Dazu wird zuerst das Vorgehen bei der Literaturrecherche systematisch dargelegt. Anschließend folgt eine Erläuterung zum Prozess der Anforderungserhebung, der Entwicklung der Systemarchitektur sowie deren prototypische Umsetzung. Schließlich wird dargestellt, wie im Rahmen der Evaluierungsstrategie untersucht wird, ob das entwickelte System die definierten Zusicherungen und Anforderungen einhält.

3.1 Literaturrecherche

[TODO: Datenbanken, Suchbegriffe/-string, Auswahlkriterien, Schneeballverfahren]

3.2 Systementwurf

Die Systementwicklung folgt einem schrittweisen, an den Anforderungen orientierten Vorgehen. Ausgehend von der Problemstellung werden in Kapitel 4 zunächst Anforderungen an das zu entwickelnde System erhoben und daraus konkrete Zusicherungen abgeleitet (vgl. TZ1). Diese Zusicherungen beschreiben die Eigenschaften, die das System auch unter Fehlerbedingungen einhalten soll. Auf dieser Grundlage werden in Kapitel 5 anschließend Architekturentscheidungen getroffen, die die Erfüllung der Zusicherungen ermöglichen (vgl. TZ2). Die ausgewählten Mechanismen werden anschließend in einem prototypischen System implementiert (vgl. TZ3).

Damit ergibt sich für den Systementwurf folgende methodische Reihenfolge:

1. Problemstellung
2. Anforderungserhebung
3. Zusicherungen
4. Architekturentwurf
5. prototypische Implementierung

Abschließend wird das implementierte System evaluiert. Die Evaluation überprüft anhand gezielt erzeugter Fehlersituationen, ob die aus den Anforderungen abgeleiteten Zusicherungen durch den entworfenen und implementierten Prototyp eingehalten werden (vgl. TZ4). Die konkrete Durchführung wird in Kapitel 6 beschrieben.

3.2.1 Anforderungserhebung und Ableitung der Zusicherungen

Der zentrale Aspekt des entwickelten Systems ist die Fehlertoleranz. Um die Anforderungen hierfür systematisch zu ermitteln, orientiert sich das Vorgehen an den Ermittlungsgegenständen des Requirements-Engineerings nach Rupp (2021). Anstelle klassischer Stakeholder-Befragungen fließen in dieser Arbeit spezifische Artefakte als primäre Anforderungsquellen in die Erhebung ein. Das Ziel ist dabei die Entwicklung eines Job-Processing Systems, das resilient gegenüber Fehlern und Teilausfällen in verteilten Systemen ist. Um den Rahmen der Anforderungen einzugrenzen, wird in Kapitel 4.1 der Systemkontext spezifiziert. Als zentrale Quelle für die Anforderungserhebung dienen schließlich die aus der Literatur abgeleiteten Problemstellungen verteilter Systeme bezüglich möglicher auftretender Fehler und das sich daraus ergebende Fehlermodell, dessen für diese Arbeit betrachtete Teilmenge in Kapitel 4.2 definiert wird. Aus diesen Artefakten werden schließlich die Anforderungen an das System erhoben.

Darauf folgend werden aus diesen Anforderungen konkrete Zusicherungen abgeleitet. Diese Verfeinerung hilft, um das zentrale Qualitätskriterium der Prüfbarkeit sicherzustellen. Eine Zusicherung beschreibt in diesem Sinne eine testbare Eigenschaft, die unabhängig von einem konkreten Ausführungsablauf beziehungsweise dem Auftreten eines Fehlers zwingend eingehalten werden soll. Die auf diese Weise formulierten Zusicherungen Z-1 bis Z-9 sind in Kapitel 4.3 vollständig spezifiziert. Sie dienen als präzise Korrektheitsbedingungen für das entwickelte System und bilden daher die Grundlage für den späteren Systementwurf und die Evaluation.

Durch dieses Vorgehen wird konstruktiv sichergestellt, dass die Erfüllung der Anforderungen später objektiv durch spezifische Prüfkriterien und automatisierte Testfälle beurteilt werden kann (vgl. Kapitel 3.3).

3.2.2 Architekturentwurf und prototypische Implementierung

Auf Grundlage der definierten Anforderungen und Zusicherungen werden die Architektur und die zentralen Verarbeitungsmechanismen des Systems entworfen. Um die Komplexität eines solchen Systems methodisch fundiert zu handhaben, orientiert sich der Entwurf an einem sichtenbasierten Ansatz. Die Fachliteratur empfiehlt, komplexe Systeme aus mehreren Blickwinkeln zu analysieren und zu dokumentieren (Rupp 2021, S.470). Die Bildung dieser spezifischen Sichten hilft das System strukturiert und schrittweise zu entwickeln (Rupp 2021, S.469). Das Vorgehen orientiert sich hierfür an dem Vier-Sichten-Modell nach Starke (2024, S.80 ff.). Dieses unterstützt nicht nur die Abstraktion und Kapselung, sondern hilft dabei, domänenspezifische Fragestellungen isoliert zu betrachten.

Die geforderten Sichten spiegeln sich in der Struktur dieser Arbeit wie folgt wider: Die Kontextabgrenzung findet bereits im Rahmen der Anforderungserhebung statt und ist entsprechend in Kapitel 4.1 thematisiert. Die Bausteinsicht wird in Abbildung 5.1 im Rahmen des Architekturentwurfs dargestellt, während einzelne Laufzeitsichten in den Kapitel 5.1 untergeordneten Kapiteln zu finden sind. Durch diese systematische Strukturierung können die einzelnen Architekturentscheidungen klar isoliert und jeweils hinsichtlich ihres Beitrags zur Erfüllung der zuvor definierten Zusicherungen bewertet und gegenüber

möglichen Alternativen eingeordnet werden. Der Entwurf umfasst dabei insbesondere die Struktur des Job-Stores, das Zustandsmodell, die Koordination konkurrierender Worker sowie Mechanismen für Idempotenz, Retry und Recovery.

Die resultierende Architektur wird anschließend prototypisch implementiert. Dabei werden die im Entwurf vorgesehenen Komponenten und Mechanismen in Java und Spring umgesetzt. Die Implementierung dient nicht der Entwicklung einer vollständigen fachlichen Anwendung, sondern der prototypischen Umsetzung und Untersuchung der für das Job-Processing relevanten Mechanismen. Die eigentliche Jobverarbeitung wird deshalb durch eine simulierte Fachlogik repräsentiert, während Persistenz, Zustandsverwaltung, konkurrierende Verarbeitung, Retry und Recovery entsprechend dem Architekturentwurf implementiert werden.

Die Implementierung inklusive der relevanten technischen Entscheidungen wird in Kapitel 5.2 detailliert beschrieben.

3.3 Evaluation

Die Evaluation untersucht, ob die implementierte Lösung die in Kapitel 4 definierten Zusicherungen auch unter gezielt herbeigeführten Fehlern einhält. Dazu werden die Zusicherungen zunächst in beobachtbare Eigenschaften und konkrete Prüfkriterien überführt. Auf dieser Grundlage werden geeignete Test- und Nachweismethoden definiert, Fehlersituationen gezielt erzeugt und die resultierenden Systemzustände ausgewertet.

Die Evaluationsmethodik umfasst damit die folgenden Schritte:

1. Operationalisierung
2. Definition von Tests und Nachweismethoden
3. Fehlerinjektion und Test
4. Auswertung

3.3.1 Operationalisierung der Zusicherungen

Die in Kapitel 4 formulierten Zusicherungen beschreiben zunächst abstrakte Eigenschaften des zu entwickelnden Systems. Damit diese im Rahmen der Evaluation überprüft werden können, werden sie in konkret beobachtbare Eigenschaften und daraus ableitbare Prüfkriterien überführt. Die Operationalisierung legt somit fest, anhand welcher beobachtbaren Systemzustände oder -ereignisse entschieden wird, ob eine Zusicherung unter einem definierten Fehlerszenario eingehalten wurde.

Hierzu wird für jede Zusicherung zunächst der für den Test erwartete Systemzustand bzw. das erwartete Verhalten bestimmt. Darauf aufbauend wird ein Prüfkriterium formuliert, das anhand von beobachtbaren Größen wie Zuständen im Job-Store, der Anzahl persistierter Ergebnisse, HTTP-Antworten, ausgeführten Operationen oder der Anzahl erreichter Verarbeitungsversuche überprüft werden kann. Die vollständige Zuordnung der Zusicherungen zu den jeweiligen Fehlerinjektionen und Prüfkriterien ist in Tabelle 6.1 dokumentiert.

3.3.2 Test- und Nachweismethoden

Die Auswahl der konkreten Testszenarien orientiert sich primär an der Abdeckung der zuvor definierten Zusicherungen. Entsprechend wird für jede der neun Zusicherungen systematisch mindestens ein Testszenario definiert, das gezielt evaluiert, ob die geforderte Eigenschaft unter der jeweiligen Fehlerbedingung eingehalten wird.

Die zugrunde liegenden Fehlerklassen werden in Kapitel 4.2 definiert und durch die Literatur fundiert begründet. Die vollständige Zuordnung von Systemzusicherung, simulierter Fehlerinjektion und dem dazugehörigen Prüfkriterium ist in Tabelle 6.1 dokumentiert. Die Nachweisführung einer Zusicherung kann sowohl empirische als auch analytische Anteile umfassen. Empirisch überprüfbare Eigenschaften werden durch gezielte Testszenarien untersucht. Eigenschaften, die sich aus strukturellen Merkmalen der Implementierung ableiten lassen, werden ergänzend anhand dieser Implementierungsmerkmale begründet. Die konkrete Zuordnung der jeweiligen Nachweismethode zu den einzelnen Zusicherungen erfolgt im Rahmen der Evaluation.

Zur Überprüfung der entwickelten Mechanismen werden Komponenten- und Integrationstests eingesetzt, die unterschiedliche Zwecke verfolgen. Im Rahmen der Entwicklung wurden isolierte Komponententests genutzt, um einzelne Mechanismen, wie die Entscheidungslogik der Retry-Komponenten, unabhängig von einer laufenden Datenbank oder Infrastruktur zu überprüfen. Ein erfolgreicher Komponententest zeigt dabei beispielsweise, dass eine `DataAccessException` grundsätzlich als retrybar klassifiziert wird, er zeigt jedoch nicht, ob eine reale Datenbankverbindung nach einer kurzzeitigen Störung tatsächlich wieder erfolgreich verwendet werden kann. Da sich die Evaluation auf die Fehlertoleranz des Systems unter gezielt herbeigeführten Fehlerbedingungen konzentriert, werden ausschließlich Integrationstests als Evaluationsevidenz herangezogen, die einer konkreten Zusicherung zugeordnet sind und das Zusammenspiel der beteiligten Komponenten unter den jeweiligen Fehlerbedingungen untersuchen. Komponententests wurden im Rahmen der Entwicklung eingesetzt, um einzelne Mechanismen isoliert zu überprüfen und die allgemeine Funktionsfähigkeit des Systems sicherzustellen. Dazu zählen beispielsweise Tests zur Klassifikation von Fehlern durch die Retry-Komponenten, Tests zur Überprüfung der grundsätzlichen Erreichbarkeit des Backends oder der fehlerfreien Joberstellung. Diese Tests sind jedoch nicht Bestandteil der eigentlichen Evaluation. Für die Evaluation wird entsprechend ein unter normalen Bedingungen funktionierendes System vorausgesetzt.

Integrationstests werden dabei gegen eine reale Instanz des verwendeten Datenbanksystems durchgeführt, um das Verhalten der Anwendung auch unter den tatsächlichen Laufzeitbedingungen der eingesetzten Datenbank zu untersuchen. Dadurch wird auch das Zusammenspiel der einzelnen Komponenten untereinander und mit der Datenbank getestet. Die konkrete technische Umsetzung der Testumgebung wird in Kapitel 6.1 beschrieben.

3.3.3 Fehlerinjektion und Testdurchführung

Die Überprüfung der Zusicherungen erfolgt durch gezielte Herbeiführung der für das jeweilige Testszenario relevanten Fehler- und Wettbewerbsbedingungen. Ein Testlauf beginnt dabei mit einem definierten Ausgangszustand. Anschließend wird die zu untersuchende Bedingung erzeugt und das daraus resultierende Systemverhalten beobachtet. Dieses wird abschließend mit den definierten Prüfkriterien verglichen.

Die Art der Fehlerinjektion wird dabei an das jeweils untersuchte Fehlerszenario angepasst. Fehler können beispielsweise durch die Unterbrechung von Infrastrukturverbindungen, durch das gezielte Herstellen eines für Recovery relevanten Systemzustands oder durch das Auslösen definierter Fehler innerhalb einer Verarbeitung erzeugt werden. Für die Untersuchung von Nebenläufigkeit werden konkurrierende Operationen synchronisiert und gleichzeitig ausgeführt. Dadurch können insbesondere solche Zustandsübergänge und Interaktionen untersucht werden, die bei einer rein sequenziellen Ausführung nicht auftreten.

Die konkrete technische Umsetzung der jeweiligen Fehlerinjektion und Teststeuerung wird zusammen mit den einzelnen Testszenarien in Kapitel 6 beziehungsweise dem Versuchsaufbau in Kapitel 6.1 beschrieben.

3.3.4 Reproduzierbarkeit und Auswertung

Nebenläufigkeitsabhängige Fehler können ein nicht-deterministisches Verhalten bei Mehrfachausführung desselben Tests verursachen. Insbesondere Race Conditions hängen vom konkreten Scheduling der beteiligten Threads ab. Ein einmalig erfolgreicher Testlauf stellt daher für solche Szenarien keinen hinreichenden empirischen Nachweis dar. Die Tests bezüglich konkurrierender Verarbeitung werden deshalb wiederholt unter identischen Ausgangsbedingungen ausgeführt. Für jeden Lauf werden dabei dieselben Systemparameter und dieselben Fehlerinjektionsbedingungen verwendet. Ausgewertet wird anschließend die Anzahl der bestandenen Läufe im Verhältnis zur Gesamtzahl der Durchläufe.

Die Anzahl der Wiederholungen wird dabei einheitlich auf 100 festgelegt. Diese Zahl stellt einen Kompromiss zwischen der Aussagekraft der Untersuchung und dem für die Testdurchführung erforderlichen Zeitaufwand dar. Eine höhere Wiederholungszahl erhöht die Wahrscheinlichkeit, selten auftretende Nebenläufigkeitseffekte zu beobachten, führt jedoch gleichzeitig zu einem höheren Zeitaufwand für die Testdurchführung. Ein erfolgreicher Durchlauf wird dabei nicht als Beweis für das generelle Ausbleiben eines Fehlers interpretiert. Vielmehr wird untersucht, ob die definierten Prüfkriterien über alle Wiederholungen hinweg eingehalten werden. Dieses Vorgehen bildet zugleich die methodische Grundlage für die Überprüfung von H3.

Die konkrete Durchführung, die verwendeten Parameter und die Ergebnisse der Testfälle werden in Kapitel 6 dokumentiert. Die dort dargestellten Werte bilden die konkrete Instanziierung der in diesem Kapitel beschriebenen methodischen Vorgehensweise.

4 Anforderungen

Dieses Kapitel legt die Anforderungen an das im Rahmen der Arbeit entwickelte Background-Job-Processing-System fest und schafft damit die Grundlage für den späteren Architekturentwurf sowie dessen Evaluation. Zunächst wird der Systemkontext beschrieben, um die beteiligten Komponenten, Schnittstellen und Verantwortlichkeiten einzuordnen. Anschließend wird das betrachtete Fehlermodell abgegrenzt und damit festgelegt, welche Störungen und Randbedingungen für den Entwurf relevant sind. Darauf aufbauend werden im Rahmen von TZ1 präzise Systemzusicherungen formuliert, die das gewünschte Verhalten unabhängig von konkreten Implementierungsdetails beschreiben und als verbindlicher Maßstab für die Umsetzung und die Evaluation dienen.

4.1 Systemkontext

Im Rahmen dieser Arbeit wird ein datenbankgestütztes Background-Job-Processing-System entwickelt, dessen grundlegende Systemstruktur in Abbildung 4.1 dargestellt ist. Über eine REST-Schnittstelle können Aufträge durch einen Service oder ein Frontend erzeugt werden, die zunächst im Backend validiert und anschließend in einem relationalen Job-Store persistent gespeichert werden. Unabhängig vom ursprünglichen Aufruf übernehmen mehrere Worker-Prozesse anschließend die asynchrone Verarbeitung der gespeicherten Jobs. Da der Schwerpunkt der Arbeit auf den Mechanismen des Job-

Abbildung 4.1: Systemkontext

Processings und nicht auf einer konkreten Fachanwendung liegt, wird die Fachlogik zum Zweck der Evaluation stark vereinfacht. Diese simuliert durch eine definierte Wartezeit einen lang laufenden Verarbeitungsschritt und erzeugt schließlich ein Dummy-Ergebnis. Ziel ist somit nicht die Entwicklung einer domänenspezifischen Fachanwendung, sondern einer allgemein einsetzbaren Architektur, deren Konzepte unabhängig vom gewählten Anwendungskontext auf andere Background-Job-Processing-Systeme übertragbar sind. Das betrachtete System vereint die wesentlichen Herausforderungen datenbankgestützter

Background-Job-Processing-Systeme. Hierzu zählen insbesondere die zuverlässige und verlustfreie Persistierung eingehender Aufträge, die koordinierte Verarbeitung durch mehrere parallel arbeitende Worker sowie die Wiederaufnahme unterbrochener Verarbeitungen nach Teilausfällen. Die genau betrachtete Fehlermenge wird im folgenden Kapitel dargelegt.

4.2 Abgrenzung des betrachteten Fehlermodells

Verteilte Softwaresysteme können durch eine Vielzahl unterschiedlicher Fehler beeinträchtigt werden. Hierzu zählen unter anderem Hardwaredefekte, Softwarefehler, Netzwerkausfälle, Bedienfehler oder Cyberangriffe. Wie bereits in Kapitel 2.3 erläutert wurde, kann ein System jedoch nicht sämtliche denkbaren Fehler verhindern oder behandeln. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Abwägung, welche Fehlerarten berücksichtigt und welche Risiken akzeptiert werden. Diese Entscheidung orientiert sich zum einen am Anwendungskontext und den Anforderungen an das System und zum anderen an wirtschaftlichen Randbedingungen wie Entwicklungs- und Betriebsaufwand (Kleppmann 2017, Kap. 1 Abschnitt „Reliability“).

Die im Folgenden dargelegte Teilmenge betrachteter Fehler ist eine Auswahl der von Kleppmann (2017), Newman (2026) und Bass (2021) beschriebenen typisch vorkommenden Fehlern in verteilten Systemen. Ziel dieser Arbeit ist der Entwurf einer fehlertoleranten Architektur für ein datenbankgestütztes Background-Job-Processing-System. Betrachtet werden daher ausschließlich Fehlerklassen, die während des regulären Betriebs auftreten können und deren Auswirkungen durch Mechanismen der Anwendungsarchitektur beherrscht werden können. Fehler, deren Behandlung zusätzliche Infrastruktur oder organisatorische Maßnahmen wie Backups, Replikation oder physische Redundanz erfordern, liegen außerhalb des Betrachtungsumfangs dieser Arbeit.

Die Abgrenzung der betrachteten Fehler ist in Tabelle 4.2 zusammengefasst. Innerhalb des Systementwurfs werden im Wesentlichen vier Kategorien von Fehlern behandelt. An erster Stelle stehen Ausfälle einzelner Systemkomponenten sowie transiente Infrastruktur- und Kommunikationsstörungen. In der Klassifikation nach Cristian (1991) entsprechen diese Fehler im Wesentlichen Crash- und Omission-Fehlern. Hierzu zählen insbesondere das unerwartete Beenden eines Worker-Prozesses, Container-Neustarts, JVM-Abstürze, durch das Betriebssystem terminierte Prozesse, temporäre Netzwerkunterbrechungen oder kurzfristig nicht erreichbare Datenbanken. Dadurch kann es im System zu zwei Problemen kommen. Zum einen besteht die Gefahr, dass eine laufende Verarbeitung unterbrochen wird oder vorübergehend nicht fortgesetzt werden kann. Aufgrund der Charakteristik transienter Fehler wird dabei grundsätzlich davon ausgegangen, dass die betroffene Komponente oder Ressource nach kurzer Zeit wieder verfügbar ist. Zum anderen entsteht die Gefahr, dass eine Nachricht bzw. ein Job verloren geht. Durch diese Fehler gerät das Gesamtsystem in einen unklaren Zustand darüber, wie weit die Verarbeitung bereits fortgeschritten war bzw. ob der Job persistiert wurde. Da diese Störungen in verteilten Umgebungen regelmäßig auftreten können (Newman 2026, Kap. 1 Abschnitt „Failure Is Inevitable“) und, wie bereits eingangs in Kapitel 1.2 gezeigt, gerade

in Cloud-Systemen mit zu den häufigsten Fehlern zählen (Alquraan u. a. 2018), muss die Systemarchitektur in der Lage sein, diese autonom zu überbrücken. Unter dem Aspekt der Fehlertoleranz sollen zur Behandlung transienter Fehler daher automatische Retries eingesetzt werden, um die Notwendigkeit von manuellem Eingreifen zu reduzieren.

Fehlerklasse	Relevant	Begründung
Komponenten und Infrastrukturausfälle	✓	Unterbrechen die Verarbeitung und gefährden die Zuverlässigkeit des Systems
Doppelte Requests	✓	Können zu mehrfach persistierten Jobs führen
Doppelte Ausführung	✓	Kann inkonsistente Seiteneffekte verursachen
Konkurrierender Zugriff	✓	Gefährdet die Konsistenz gemeinsamer Daten
Physischer Datenbankverlust	✗	Erfordert Backups & Redundanz
Totalausfall Infrastruktur	✗	Erfordern Disaster-Recovery Maßnahmen
Cyberangriffe	✗	Qualitätsmerkmal Sicherheit
Byzantinische Fehler	✗	Annahme: Vertrauensvolle Komponenten

Tabelle 4.2: Übersicht betrachteter Fehlerklassen

Als direkte Folge solcher Infrastrukturprobleme und der darauf reagierenden Wiederholungsmechanismen der Clients entstehen doppelte Requests. Ein Client sendet eine Anfrage erneut, weil eine Bestätigung aufgrund eines Timeouts ausblieb. Die Architektur soll im Rahmen der Zuverlässigkeit eine Exactly-Once-Semantik als Ziel haben und muss daher im beschriebenen Szenario durch Duplikaterkennung verhindern, dass identische Jobs mehrfach im Job-Store persistiert werden.

Eng damit verknüpft ist die doppelte Ausführung von Jobs. Bei der Wiederaufnahme eines zuvor abgebrochenen Jobs kann nach einem Fehler oft nicht eindeutig festgestellt werden, ob die vorherige Verarbeitung bereits Seiteneffekte erzielt hat. Die Architektur muss daher zwingend sicherstellen, dass wiederholte Ausführungen keine inkonsistenten Ergebnisse verursachen.

Des Weiteren werden konkurrierende Zugriffe betrachtet. Sobald Parallelverarbeitung mittels mehrerer, autonom agierender Worker eingesetzt wird, entsteht unweigerlich die Möglichkeit gleichzeitiger Zugriffe auf geteilte Datenstrukturen wie den zentralen Job-Store. Dies ist im klassischen Sinne zwar nicht direkt ein Fehler, jedoch können parallele Zugriffe ohne entsprechende Synchronisationsmechanismen potenziell zu Race Conditions und inkonsistenten Zuständen in der Persistenzschicht führen, weshalb deren Verhinderung wesentlicher Bestandteil des Entwurfs ist.

Dem gegenüber stehen Fehler, die bewusst außerhalb des Betrachtungsumfangs dieser Arbeit liegen. Ein physischer Datenbankverlust sowie ein Totalausfall der Infrastruktur können nicht durch die Softwarearchitektur des Job-Processing-Systems gelöst werden. Ein Verlust der Persistenzschicht oder ein Ausfall des gesamten Rechenzentrums infolge von großflächigen Stromausfällen oder Bränden am Serverstandort erfordert stattdessen infrastrukturelle Maßnahmen wie geografische Redundanz, kontinuierliche Backups und umfassende Disaster-Recovery-Konzepte (Newman 2026, Kap. 9).

Ebenso werden Cyberangriffe im Rahmen dieser Arbeit ausgeschlossen. Obwohl Angriffe wie SQL-Injections oder Denial-of-Service-Szenarien direkte Auswirkungen auf die Zuverlässigkeit eines Systems haben können, betreffen sie primär das Qualitätsmerkmal der Sicherheit. Ihre Abwehr erfordert spezialisierte Mechanismen wie strikte Eingabevalidierung, Ratenbegrenzungen, Authentifizierung und Angriffserkennungssysteme. Der Fokus dieser Arbeit liegt stattdessen auf der Gewährleistung von Fehlertoleranz bei regulärem Systembetrieb.

Schließlich werden auch byzantinische Fehler ausgeschlossen. Dies sind Fehler, bei denen kompromittierte oder defekte Komponenten willkürlich falsche oder manipulierte Informationen erzeugen und verbreiten (Kleppmann 2017, Kap. 8 Abschnitt „Byzantine Faults“). Die vorliegende Arbeit geht dahingehend von der Annahme ehrlicher Komponenten aus. Dies bedeutet, dass Komponenten zwar abstürzen oder vorübergehend nicht erreichbar sein können, im aktiven Zustand jedoch nie absichtlich falsche Informationen verbreiten.

Aus der gewählten Abgrenzung ergibt sich, dass der Schwerpunkt dieser Arbeit auf der Behandlung von Teilausfällen, konkurrierender Verarbeitung und transienten Infrastrukturfehlern liegt. Diese Fehlerszenarien besitzen unmittelbaren Einfluss auf die Zuverlässigkeit datenbankgestützter Background-Job-Processing-Systeme und bilden daher zusammen mit der Systembeschreibung aus Kapitel 4.1 die Grundlage für die im Folgenden formulierten Systemzusicherungen.

4.3 Zusicherungen

Die sich aus dem in Kapitel 4.1 beschriebenen Systemkontext und dem in Kapitel 4.2 definierten Fehlermodell ergebenden zentralen Anforderungen an das zu entwickelnde Background-Job-Processing-System sind in Tabelle 4.4 zusammengefasst. Auf Basis dieser Anforderungen lassen sich Eigenschaften ableiten, die das entwickelte Background-Job-Processing-System zu jedem Zeitpunkt gewährleisten muss, um die fehlerfreie Funktion des Systems bezogen auf die Zielsetzung der Arbeit sicherzustellen. Diese Eigenschaften werden im Folgenden als Systemzusicherungen bezeichnet. Im Gegensatz zu konkreten Architekturmechanismen beschreiben Systemzusicherungen ausschließlich das von der Architektur einzuhaltende Verhalten. Sie formulieren Invarianten des Systems, die unabhängig von der jeweiligen Implementierung oder dem aktuell betrachteten Fehlerszenario jederzeit gelten müssen. Die in Kapitel 5.1 entwickelten Architekturmechanismen dienen dazu, die Einhaltung dieser Zusicherungen sicherzustellen.

ID	Anforderung / Problemstellung
A-1	Erfassung eingehender Aufträge über die REST-Schnittstelle
A-2	Persistente Speicherung erfasster Jobs
A-3	Behandlung doppelter Client-Anfragen
A-4	Parallele Job-Verarbeitung durch mehrere Worker
A-5	Vermeidung von Race Conditions bei konkurrierendem Zugriff
A-6	Erkennung von Teilausfällen (Worker-Crashes, Container-Neustarts) und Wiederaufnahme/Wiederholung des betroffenen Verarbeitungsschrittes
A-7	Konsistente Verarbeitung trotz wiederholter Ausführung

Tabelle 4.4: Systemanforderungen

4.3.1 Joberstellung

Die Phase der Joberstellung umfasst die Entgegennahme eines Auftrags über die Schnittstelle sowie dessen dauerhafte Überführung in den zentralen Job-Store (A-1 & A-2). Unvollständige oder fehlerhafte Anfragen sollten dabei nicht zu inkonsistenten Zuständen in der Persistenzschicht führen, damit Worker-Prozesse sich darauf verlassen können, dass im Job-Store vorhandene Jobdaten fehlerfrei und vollständig verarbeitbar sind. Daraus leitet sich die erste Systemzusicherung ab:

Zusicherung Z-1 (Atomare Persistenz und Validität):

Das System garantiert, dass ein über die Schnittstelle eingehender Auftrag nur dann im Job-Store persistiert wird, wenn er die fachliche und syntaktische Validierung erfolgreich durchlaufen hat. Die Persistierung erfolgt atomar. Ein Auftrag ist für Worker-Prozesse entweder vollständig und in einem gültigen Initialzustand sichtbar oder existiert nicht.

Wie in Kapitel 2.3.2 dargelegt, können transiente Kommunikationsfehler dazu führen, dass Nachrichten erneut verschickt werden. Da im weiteren Verlauf entsprechende Wiederholungsmechanismen gefordert werden, muss die Joberstellung idempotent gestaltet sein, damit eine erneute Verarbeitung derselben Anfrage nicht zur mehrfachen Persistierung desselben Jobs führt:

Zusicherung Z-2 (Duplikaterkennung bei der Auftragserfassung):

Das System garantiert, dass identische Anfragen, die von Aufrufern erneut gesendet werden, nicht zu mehrfach angelegten Jobs im Job-Store führen (*At-Most-Once-Semantik*).

4.3.2 Jobverarbeitung

Die Phase der Jobverarbeitung beschreibt den regulären Betrieb, in dem mehrere Worker-Prozesse autonom und parallel Aufträge aus dem Job-Store beziehen und ausführen. Aus

dem Zusammenspiel paralleler Instanzen (A-4) folgt die Konkurrenz beim Zugriff auf den gemeinsamen Job-Store, wodurch das Risiko von Race Conditions entsteht. Da im Recovery-Fall eine überschneidende Bearbeitung durch zwei Worker nicht ausgeschlossen werden kann, muss das System den Zugriff auf die gemeinsamen Daten koordinieren (A-5):

Zusicherung Z-3 (Exklusive Rechte):

Das System garantiert, dass niemals mehrere Worker-Instanzen gleichzeitig Zugriff auf den gleichen Datensatz haben. Insbesondere darf immer nur eine Worker-Instanz Schreibrechte auf einen bestimmten Job haben, sodass das Schreiben eines Ergebnisses nur durch den besitzenden Job möglich ist.

Die Fachlogik soll zudem klare Übergänge bieten. Im Rahmen der konsistenten Ergebniserzeugung (A-7) darf ein Job im System keine Teilergebnisse hinterlassen:

Zusicherung Z-4 (Atomare Wirkung von Ergebnissen):

Das System garantiert, dass das Eintragen eines fachlichen Ergebnisses (oder einer Fehlermeldung) und der dazugehörige Übergang in den Folgezustand eine atomare Einheit bilden. Nach außen sichtbar ist entweder das vollständige Ergebnis inklusive der Zustandsänderung oder keinerlei Auswirkung.

Zusätzlich müssen die Zustände der Jobverarbeitung einer klaren Richtung folgen, um eine mehrfache Ergebniserzeugung zu verhindern und die Konsistenz der Daten so zu schützen (A-7):

Zusicherung Z-5 (Finalität von Endzuständen):

Das System garantiert, dass der Lebenszyklus eines Jobs einer strikt gerichteten Zustandslogik folgt. Die Übergänge in die definierten Endzustände sind final und unumkehrbar. Kein Job kann nach Erreichen eines Endzustands erneut in einen aktiven Zustand versetzt werden.

4.3.3 Retries und Recovery

Das Unterkapitel Recovery adressiert das Verhalten des Systems bei Teilausfällen, wie dem unerwarteten Absturz von Worker-Prozessen oder abgebrochenen Verbindungen. Handelt es sich bei der Ursache der abgebrochenen Verbindung um einen transienten Fehler, soll dieser nicht unmittelbar zum endgültigen Fehlschlagen der Operation führen (A-6):

Zusicherung Z-6 (Automatische Wiederholungsversuche):

Das System kann transiente und permanente Fehler unterscheiden. Erkennt es einen transienten Fehler, behandelt es diesen durch automatische Wiederholungsversuche, ohne dass ein Nutzer eingreifen muss.

Das Gesamtsystem muss außerdem in der Lage sein, abgebrochene Verarbeitungen zu erkennen und den betroffenen Job ohne manuellen Eingriff wieder im System verfügbar zu machen (A-6):

Zusicherung Z-7 (At-Least-Once-Verarbeitungsgarantie):

Das System garantiert, dass ein einmal erfolgreich persistierter Job nicht verloren geht. Wird die Verarbeitung eines Jobs durch den Absturz einer Komponente unterbrochen, stellt das System sicher, dass der Job nach Erkennung des Ausfalls automatisch wieder zur Verarbeitung freigegeben und von einer verfügbaren Instanz erneut aufgenommen wird.

Da Wiederholungsversuche und Recovery-Mechanismen dazu führen können, dass die Bearbeitung eines Jobs mehrfach gestartet wird, muss sichergestellt werden, dass das fachliche Ergebnis eindeutig bleibt (A-7):

Zusicherung Z-8 (Eindeutigkeit des Ergebnisses und Idempotenz):

Das System garantiert, dass zu jedem im System erfassten Job exakt ein finales fachliches Ergebnis oder eine finale Fehlerbeschreibung gespeichert wird. Mehrfache oder abgebrochene Ausführungsversuche verfälschen das Endergebnis nicht und erzeugen keine duplizierten Ergebnisdatensätze.

Schließlich muss das System sicherstellen, dass Jobs nicht unendlich im System verbleiben oder in endlosen Fehlerschleifen gefangen sind:

Zusicherung Z-9 (Terminierung in endlicher Zeit):

Das System garantiert, dass jeder persistierte Job nach endlicher Zeit in einen der Endzustände übergeht. Kein Job verbleibt unendlich in einem aktiven bzw. ausstehenden Status.

5 Umsetzung

Dieses Kapitel befasst sich mit dem Entwurf und der technische Umsetzung des in Kapitel 4 definierten Systems.

Der Aufbau des Kapitels gliedert sich dabei in zwei Abschnitte: Im ersten Teil wird die Architektur des Systems entwickelt und die fundamentalen Systementscheidungen begründet. Aufbauend auf diesem theoretischen Fundament dokumentiert der zweite Teil die konkrete Implementierung des Prototyps auf Basis des Java- und Spring-Ökosystems.

5.1 Architektur und Design

In diesem Kapitel wird die Architektur des entwickelten Job-Processing-Systems zur Erfüllung von TZ2 beschrieben. Das primäre Ziel besteht darin, die in Kapitel 4 erhobenen Anforderungen und Zusicherungen durch geeignete Entwurfsentscheidungen und den gezielten Einsatz von Entwurfsmustern zu erfüllen.

Zur strukturierten Darstellung der einzelnen Systemkomponenten gliedert sich das Kapitel in folgende Teilaspekte: Kapitel 5.1.1 führt zunächst in die abstrakte Gesamtsystemarchitektur ein, begründet die Wahl einer relationalen PostgreSQL-Datenbank als zentralen Job-Store und stellt die Interaktion zwischen den einzelnen Komponenten dar. Darauf aufbauend beschreibt Kapitel 5.1.2 das Zustandsmodell, welches unter anderem als Grundlage für die Koordination der Jobs und die Recovery dient. Das zugehörige Datenbankschema wird in Kapitel 5.1.3 vorgestellt, während Kapitel 5.1.4 die Mechanismen zur idempotenten Annahme neuer Jobs über REST erläutert. In Kapitel 5.1.5 wird das Transaktions- und Sperrmodell betrachtet, das eine exklusive Verarbeitung bei gleichzeitig kurzen Datenbanktransaktionen ermöglicht. Abschließend widmen sich Kapitel 5.1.6 und Kapitel 5.1.7 den Fehlertoleranzmechanismen. Während die Retry-Strategie transiente Infrastrukturfehler auf Komponentenebene kompensiert, behandelt der Recovery-Service den Ausfall von Workern und verhindert das Verbleiben verwaister Jobs. Kapitel 5.1.8 schließt den Entwurf ab, indem das *Fencing*-Prinzip zur Vermeidung von Mehrfachverarbeitungen im Fehlerfall dargelegt wird.

5.1.1 Architekturentwurf

[TODO: Hier muss noch der Weg von Datenbank -> relationale Datenbank offengelegt werden] In Kapitel 2.1.3 wurde bereits dargelegt, dass das Queue-Based Load Leveling Pattern eine mögliche und etablierte Option ist, die ausstehenden Jobs innerhalb eines Backend-Job-Processing-Systems zwischenspeichern. Eine verbreitete Variante, dieses Pattern für Nachrichten, oder wie im Fall dieser Arbeit für Jobs, umzusetzen, ist die Implementierung mithilfe dedizierter Message Broker (Kleppmann 2017, Kap. 11 Abschnitt „Message brokers“). Das im Rahmen dieser Arbeit entwickelte System nutzt hingegen bewusst eine relationale PostgreSQL-Datenbank als zentralen Job-Store. Durch die Nutzung PostgreSQL-spezifischer Mechanismen wie *SKIP LOCKED* in Kombination mit den vollständigen ACID-Transaktionsgarantien relationaler Datenbanken lässt sich

bereits ein Teil der in Kapitel 4 erhobenen Anforderungen mittels der inhärenten Datenbankfunktionen realisieren. Die Datenbank übernimmt dabei nicht ausschließlich die Rolle eines persistenten Datenspeichers, sondern fungiert gleichzeitig als Warteschlange sowie als Synchronisationsmechanismus zwischen konkurrierenden Workern. Funktional orientiert sich die Architektur somit an den Prinzipien eines klassischen Message Brokers, nutzt jedoch die Konsistenzgarantien einer relationalen Datenbank als Grundlage für die Umsetzung der definierten Zusicherungen. **[TODO: Hier nochmal überarbeiten: im folgenden ist nicht die Nutzung eines datenbankbasierten Job-Stores das Problem, sondern die Nutzung einer Datenbank die auf den Sekundärspeicher schreibt.]** Die Wahl eines datenbankbasierten Job-Stores stellt dabei einen bewussten Architektur-Trade-off dar. Im Vergleich zu In-Memory-basierten Messaging-Systemen entstehen durch den permanenten Zugriff auf einen externen persistenten Datenspeicher zusätzliche Latenzen und ein geringerer maximaler Nachrichtendurchsatz. Demgegenüber ermöglicht die relationale Datenbank jedoch eine dauerhafte Speicherung sämtlicher Jobs sowie atomare Zustandsübergänge und eine konsistente Wiederherstellung des Systemzustands nach Fehlern. Zusätzlich bietet eine relationale Datenbank mehr Sicherheiten vor verloren gegangenen Jobs, als typische In-Memory Message Broker, die dafür aufwändige Mechanismen benötigen. Die Haltung des Jobstatus in einer Datenbank hat außerdem für sehr rechenintensive Operationen den Vorteil, dass eine angefangene Operation nach einem Teilausfall sehr einfach fortgesetzt werden kann, anstatt den Job vollständig neu starten zu müssen, indem zur Wiederaufnahme der letzte bekannte Status des Jobs bei der Datenbank angefragt wird (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“). Gerade für kleine bis mittelgroße Systeme, in deren Anwendungskontext kein Jobverlust auftreten darf, stellt dies eine geeignete und praktikable Architektur dar, da auf eine aufwändige Clusterstruktur zur Absicherung von Jobverlusten verzichtet werden kann (Newman 2026, Kap. 8 Abschnitt „HowBrokers Work (In A Nutshell)“).

Die resultierende Gesamtarchitektur ist in Abbildung 5.1 dargestellt. Der Client bildet den Einstiegspunkt des Systems und erzeugt neue Jobs, die entsprechend A-1 über eine REST-Schnittstelle an das Backend übertragen werden. Im Rahmen des entwickelten Prototyps erfolgt dies lediglich direkt über einen Service, der die Kommunikation mit dem Backend-Application Programming Interface (API) kapselt. In der Praxis kann diese Komponente jedoch ebenso durch eine grafische Benutzeroberfläche oder ein beliebiges externes Anwendungssystem angesprochen werden. Die REST-API dient als Schnittstelle des Backends zur Erstellung neuer Jobs. Sie nimmt eingehende Anfragen entgegen, validiert diese (Z-1) und delegiert sie an die Servicelogik, die für die Persistierung neuer Jobs sowie die Verwaltung ihrer Zustände verantwortlich ist. Alle Jobs werden anschließend im zentralen Job-Store gespeichert (A-2), der neben den eigentlichen Jobdaten auch den aktuellen Bearbeitungszustand sowie gegebenenfalls erzeugte Ergebnisse enthält. Die eigentliche Verarbeitung erfolgt durch mehrere parallel arbeitende Worker (A-4). Jeder Worker beansprucht einen Job exklusiv aus dem Job-Store, verarbeitet dessen Nutzdaten und speichert das erzeugte Ergebnis anschließend wieder in der Datenbank. Da sämtliche Worker auf denselben Job-Store zugreifen, kann die Verarbeitung durch Synchronisation auf Datenbankebene horizontal skaliert werden, ohne dass eine direkte Kommunikation zwischen den einzelnen Workern erforderlich ist.

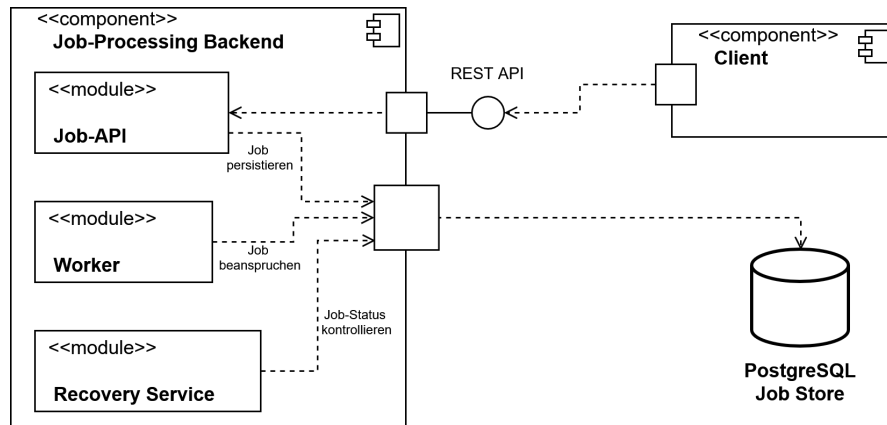


Abbildung 5.1: Abstrakte Architektur des entwickelten Systems

Ergänzt wird die Architektur durch einen Recovery-Service, der den Job-Store periodisch auf verwaiste Jobs überprüft. Kann ein Worker einen bereits beanspruchten Job aufgrund eines Fehlers oder Absturzes nicht erfolgreich abschließen, erkennt der Recovery-Service anhand des abgelaufenen Leases, dass die Bearbeitung nicht abgeschlossen wurde. Der Status des betroffenen Jobs wird, sofern der Recovery-Service eine erneute Bearbeitung für sinnvoll erachtet, zurückgesetzt und steht damit einem beliebigen Worker erneut zur Bearbeitung zur Verfügung. Auf diese Weise wird verhindert, dass Jobs dauerhaft im Zustand *Running* verbleiben (Z-9) oder infolge eines Worker-Ausfalls verloren gehen (Z-7).

5.1.2 Zustandsmodell

Zur Realisierung der in Kapitel 4 definierten Zusicherungen Z-3, Z-5, Z-7, Z-8 und Z-9 wird jeder Job durch ein explizites Zustandsmodell beschrieben. Der aktuelle Zustand eines Jobs wird dabei als Attribut des jeweiligen Datensatzes in der Datenbank gespeichert (vgl. Kapitel 5.1.3) und beschreibt den Fortschritt der Verarbeitung. Das Zustandsmodell bildet die Grundlage für sämtliche Entscheidungen bezüglich der Bearbeitung, Wiederherstellung und Beendigung eines Jobs.

Ein wesentlicher Vorteil dieses Ansatzes besteht darin, dass Datenbankzeilen nicht über den gesamten Verarbeitungszeitraum exklusiv gesperrt werden müssen. Sperren sind lediglich während kurzer atomarer Zustandsübergänge erforderlich, während die eigentliche Verarbeitung außerhalb einer Datenbanktransaktion erfolgt. Dadurch werden konkurrierende Zugriffe auf den Job-Store reduziert und Datenbankressourcen geschont, was sich positiv auf die Skalierbarkeit des Systems auswirkt.

Die in diesem System definierten Zustände sowie die zulässigen Zustandsübergänge sind in Abbildung 5.2 dargestellt. Nachdem ein neuer Job über die REST-Schnittstelle im Backend eingegangen ist, wird dieser persistent gespeichert und zunächst in den Zustand *Queued* versetzt. In diesem Zustand wartet der Job auf seine Bearbeitung und kann von einem Worker beansprucht werden. Beansprucht ein Worker den Job erfolgreich, erfolgt

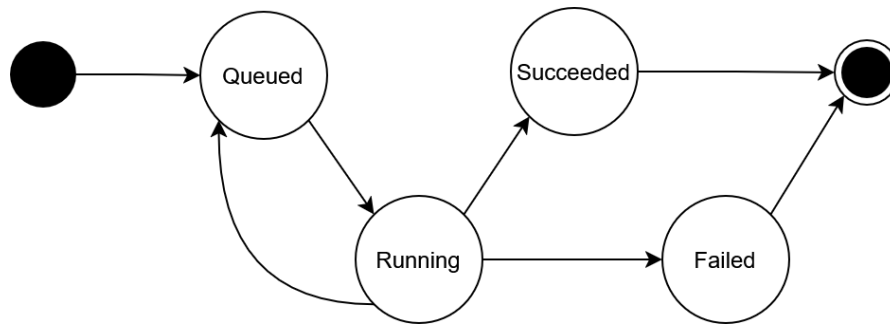


Abbildung 5.2: Zustandsmodell eines Jobs

der Zustandsübergang nach *Running*. Dadurch wird sichergestellt, dass der Job während seiner Verarbeitung keinem weiteren Worker zur Verfügung steht.

Zur Verarbeitung existieren drei mögliche Folgezustände. Kann der Worker den Job erfolgreich bearbeiten und das erzeugte Ergebnis dauerhaft in der Datenbank speichern, wird der Job in den Endzustand *Succeeded* überführt. Tritt während der Verarbeitung hingegen ein Fehler auf oder beendet der Worker seine Arbeit nicht innerhalb der vorgesehenen Lease-Dauer, übernimmt der Recovery-Service die weitere Behandlung des Jobs. Ist gemäß der definierten Recovery-Strategie ein erneuter Ausführungsversuch zulässig, wird der Job zurück in den Zustand *Queued* versetzt und kann anschließend erneut von einem beliebigen Worker übernommen werden. Wurde dagegen die maximale Anzahl zulässiger Wiederholungsversuche erreicht oder liegt ein nicht behebbarer Fehler vor, wird der Job endgültig in den Endzustand *Failed* überführt. Jobs in diesem Zustand werden vom System nicht weiter automatisch verarbeitet und erfordern das manuelle Eingreifen durch den Benutzer oder Administrator.

5.1.3 Entwurf der Datenbank

Die relationale Datenbank bildet den zentralen Job-Store des entwickelten Systems. Durch die Nutzung einer relationalen Datenbank mit ACID, wird automatisch die in Z-1 und Z-7 geforderte Dauerhaftigkeit angenommener Jobs garantiert. Sie übernimmt sowohl die Persistierung eingehender Jobs als auch die Verwaltung ihres aktuellen Bearbeitungszustands und gegebenenfalls erzeugter Verarbeitungsergebnisse. Das resultierende Datenbankschema ist in Abbildung 5.3 dargestellt. Hauptbestandteil des Schemas ist die Tabelle `jobs`. Jeder Job besitzt zunächst einen von der Datenbank erzeugten Primärschlüssel, der zur internen Identifikation dient. Zusätzlich wird für jeden Job ein vom Client bereitgestellter Idempotenzschlüssel gespeichert. Dieser stellt die in Z-2 geforderte Idempotenz in der Joberstellung sicher, auf die in Kapitel 5.1.4 näher eingegangen wird. Um das zu gewährleisten, müssen die Idempotenzschlüssel eindeutig sein. Daher ist die Spalte `idempotency_key` mit einem `UNIQUE`-Constraint belegt.

Neben den Identifikationsmerkmalen wird der eigentliche Inhalt des Jobs als Payload persistent gespeichert. Im Beispielsystem wird dieses Feld als JSON-String modelliert. Abhängig von der Art des Jobs und den Anforderungen an das System kann dies jedoch

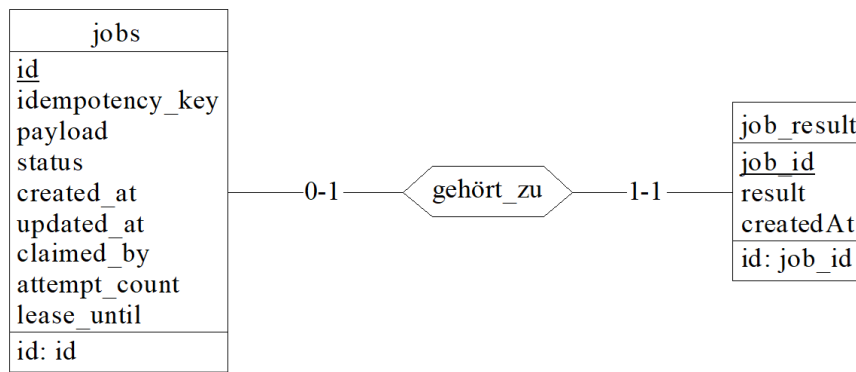


Abbildung 5.3: Datenbankschema des entwickelten Job-Processing-Systems

auch in jeglicher anderen Form umgesetzt werden, sofern die nötigen Informationen für die Durchführung des Jobs enthalten sind. Durch die Persistierung der Payload bleibt die vollständige Anfrage dauerhaft verfügbar und kann im Fehlerfall erneut verarbeitet werden, ohne dass der Client den Job nochmals übertragen muss. Das bildet die Grundlage für Z-7 und Z-8.

Zur Umsetzung des in Kapitel 5.1.2 vorgestellten Zustandsmodells besitzt jeder Job ein Statusattribut, welches den aktuellen Bearbeitungszustand beschreibt. Ergänzend hierzu werden weitere Metadaten gespeichert, die für die Umsetzung der Fehlertoleranz erforderlich sind. Das Attribut `created_at` speichert den Zeitpunkt der Joberstellung. Indem Worker neue Jobs in der Reihenfolge ihres Erstellungszeitpunkts beanspruchen, wird verhindert, dass ältere Jobs dauerhaft von neu eingehenden Jobs verdrängt werden und somit im Job-Store verbleiben. Dieses Verhalten adressiert Z-9. Während der Bearbeitung wird im Attribut `claimed_by` gespeichert, welcher Worker einen Job aktuell verarbeitet. Diese Zuordnung stellt sicher, dass ausschließlich der Worker, der einen Job erfolgreich beansprucht hat, dessen Verarbeitung abschließen und das Ergebnis persistent speichern darf. Dadurch wird die in Z-8 geforderte idempotente Ergebnisspeicherung unterstützt und verhindert, dass konkurrierende Worker im Recovery-Fall denselben Job gleichzeitig abschließen.

Für die Recovery-Strategie wird zusätzlich die Anzahl bisheriger Bearbeitungsversuche im Attribut `attempt_count` gespeichert. Dadurch kann die Anzahl automatischer Wiederholungsversuche begrenzt werden, sodass fehlerhafte Jobs nicht unbegrenzt erneut ausgeführt werden. Das Attribut `lease_until` dient der Umsetzung zeitbasierter Leases. Es gibt an, bis zu welchem Zeitpunkt ein Worker den beanspruchten Job exklusiv bearbeiten darf. Überschreitet die Bearbeitung diese Zeitspanne, kann der Recovery-Service den Job als verwaist erkennen und ihn entsprechend der definierten Recovery-Strategie erneut zur Bearbeitung freigeben. Diese beiden Attribute werden für die Umsetzung von Z-7 und Z-9 benötigt.

Die Verarbeitungsergebnisse werden in einer separaten Tabelle `job_result` gespeichert. Zwischen einem Job und seinem Ergebnis besteht dabei eine optionale Eins-zu-Eins-

Beziehung. Ein Job besitzt höchstens ein Ergebnis, da die idempotente Verarbeitung sicherstellt, dass erfolgreiche Jobs nur einmal abgeschlossen werden können. Gleichzeitig gehört jedes gespeicherte Ergebnis eindeutig zu genau einem Job. Solange ein Job noch nicht erfolgreich bearbeitet wurde, existiert dementsprechend kein zugehöriger Ergebniseintrag.

Das dargestellte Datenbankschema bildet das minimale Schema zur Umsetzung der in dieser Arbeit betrachteten Fehlertoleranzmechanismen. In realen Anwendungen kann der Job-Store beliebig um weitere fachliche Attribute oder zusätzliche Tabellen erweitert werden. Ebenso ist es möglich, eine bereits bestehende relationale Datenbank zur Umsetzung des Job-Processing-Systems zu verwenden, indem das vorgestellte Schema in die vorhandene Datenbank integriert wird.

5.1.4 Joberstellung

Die Annahme neuer Jobs gehört zur Erstellungsphase neuer Jobs und bildet somit den Einstiegspunkt in das entwickelte Job-Processing-System. Zur Umsetzung einer Exactly-Once Auslieferung, die aus den Zusicherungen Z-2 und Z-6 folgt, sind verschiedene Mechanismen nötig, die im Folgenden vorgestellt werden. Der Ablauf der Jobannahme ist in Abbildung 5.4 dargestellt. Zur Erstellung eines neuen Jobs sendet der Client eine Anfrage

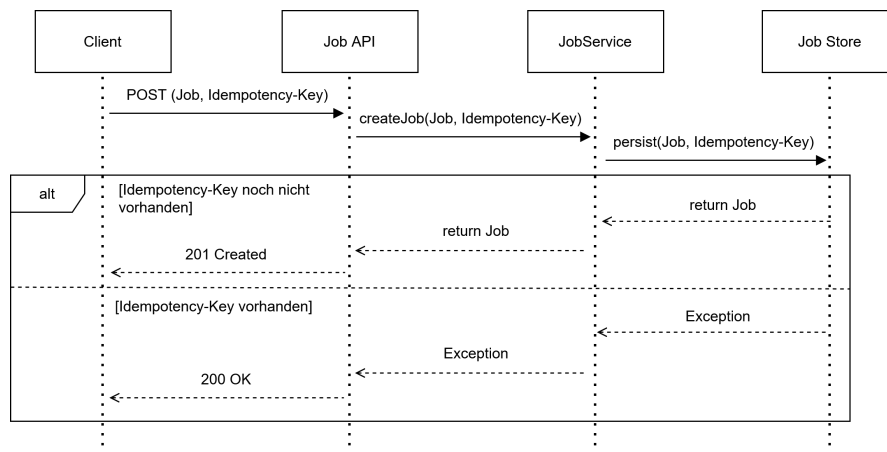


Abbildung 5.4: Ablauf der Jobannahme

über die REST-Schnittstelle an das Backend. Die Anfrage besteht aus zwei Bestandteilen: der eigentlichen Nutzlast (*Payload*), welche die durchzuführende Arbeit beschreibt, sowie einem vom Client erzeugten Idempotenzschlüssel. Dieser Schlüssel identifiziert einen Job eindeutig und ermöglicht es, mehrfach übertragene Anfragen desselben Jobs zuverlässig zu erkennen. Die Verwendung des Idempotenzschlüssels ist insbesondere für den Fall relevant, dass der Client eine Anfrage erneut sendet, obwohl der ursprüngliche Job bereits erfolgreich gespeichert wurde. Ein solches Szenario kann beispielsweise auftreten, wenn das Acknowledgement des Backends bei der Übertragung verloren geht und der Client deshalb fälschlicherweise von einer fehlgeschlagenen Joberstellung ausgeht. Durch den

Idempotenzschlüssel kann das Backend erkennen, dass der betroffene Job bereits existiert und eine doppelte Persistierung verhindern. Damit wird Zusicherung Z-2 umgesetzt. Nach Eingang der Anfrage leitet die REST-API diese an den Job-Service weiter. Dieser versucht, den Job zusammen mit dem Idempotenzschlüssel zu speichern. Existiert der Idempotenzschlüssel noch nicht, liefert die Anfrage den Job als Ergebnis. Der Service gibt den erstellten Job an die REST-API zurück, welche dem Client mit dem HTTP-Statuscode 201 **Created** als Bestätigung der erfolgreichen Joberstellung antwortet. Existiert dagegen bereits ein Job mit demselben Idempotenzschlüssel, wird ein Fehler aufgrund des **UNIQUE**-Constraints der Datenbank ausgelöst. Dieser wird durch den Service weitergegeben und schließlich vom Controller der REST-API abgefangen. Da sich der gewünschte Job bereits im System befindet und somit der gewünschte Zielzustand erreicht wurde, beantwortet die API die Anfrage mit dem HTTP-Statuscode 200 **OK**.

Kann während der Persistierung keine Verbindung zur Datenbank hergestellt werden oder tritt ein anderer transienter Fehler auf, wird der Persistierungsvorgang automatisch mehrfach mit exponentiellem Backoff wiederholt. Weitere Details zu Client- und Serverseitigen Retries werden in Kapitel 5.1.6 behandelt. Erst wenn der Job erfolgreich gespeichert wurde oder bereits existiert, wird dem Client ein erfolgreicher HTTP-Statuscode zurückgegeben. Kann der Job dagegen auch nach Ausschöpfen aller Wiederholungsversuche nicht gespeichert werden, antwortet das Backend mit dem HTTP-Statuscode 500 **Internal Server Error**. Dadurch kann der Client erkennen, dass kein erfolgreicher Abschluss der Operation garantiert werden konnte und den Erstellungsversuch seinerseits gemäß der definierten Retry-Strategie wiederholen. Dieses Zusammenspiel zwischen client- und serverseitigen Wiederholungsmechanismen stellt sicher, dass die Zusicherung Z-6 eingehalten wird und somit kein direktes manuelles Eingreifen nach dem ersten fehlgeschlagenen Versuch notwendig ist.

5.1.5 Exklusive Verarbeitung und Transaktionsmodell

Eine naheliegende Möglichkeit zur Absicherung der Hintergrundverarbeitung vor konkurrierenden Zugriffen besteht darin, eine Datenbanktransaktion über den gesamten Lebenszyklus eines Jobs geöffnet zu halten. Die Transaktion würde unmittelbar nach dem Beanspruchen eines Jobs beginnen und erst nach Abschluss der vollständigen Fachlogik sowie der Persistierung des Ergebnisses beendet werden. Wie bereits in Kapitel 2.2.2 erläutert, bringen lange Transaktionen einige Nachteile mit sich und werden daher auch für das in dieser Arbeit entworfene System nicht verwendet. Hintergrundjobs kapseln typischerweise langlaufende Berechnungen oder E/A-intensive Operationen (Microsoft Corporation (Hrsg.) 2026a). Während der gesamten Bearbeitungsdauer würden dadurch Datenbankverbindungen sowie Sperren unnötig lange gehalten werden müssen. Transaktionen sollten so lang wie nötig und so kurz wie möglich entworfen werden.

Aus diesem Grund verfolgt das entwickelte System einen anderen Ansatz. Die eigentliche Fachlogik wird vollständig außerhalb einer Datenbanktransaktion ausgeführt. Datenbanktransaktionen werden ausschließlich zur atomaren Synchronisation der Zustandsübergänge verwendet und deshalb möglichst kurz gehalten. Die Kombination aus dem in Kapitel 5.1.2 vorgestellten Zustandsmodell und kurzlaufenden Transaktionen ermöglicht

eine exklusive Bearbeitung der Jobs, ohne Datenbankressourcen während der eigentlichen Verarbeitung zu blockieren.

Der Verarbeitungsablauf gliedert sich hierzu in drei Phasen, die in Abbildung 5.5 visualisiert sind. Zunächst beansprucht ein Worker innerhalb einer Datenbanktransaktion

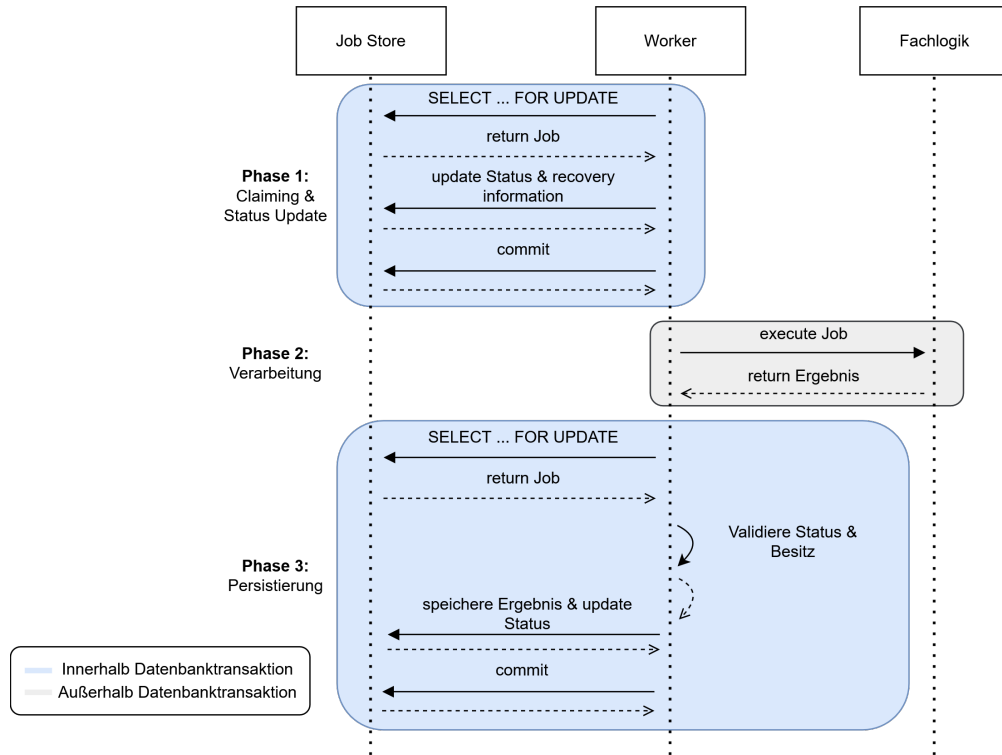


Abbildung 5.5: Transaktionsmodell

einen freien Job. Hierzu wird mittels *SELECT ... FOR UPDATE SKIP LOCKED* ein geeigneter Job ausgewählt und exklusiv gesperrt. Bereits von anderen Workern reservierte Jobs werden dadurch nicht blockierend abgewartet, sondern unmittelbar übersprungen. Zusätzlich wird der Jobstatus von *Queued* nach *Running* überführt und die Attribute für die Recovery wie *setClaimed_by*, *attempt_count* oder *lease_until* aktualisiert. Unmittelbar nach diesen Änderungen wird die Transaktion committet und die Datenbankverbindung wieder freigegeben. Anschließend erfolgt die eigentliche Verarbeitung des Jobs vollständig außerhalb einer Datenbanktransaktion. Da der Job bereits den Zustand *Running* besitzt, können andere Worker ihn währenddessen nicht erneut beanspruchen. Nach erfolgreichem Abschluss der eigentlichen Fachlogik darf ein Worker den Job nicht unmittelbar als erfolgreich markieren. Stattdessen wird zunächst eine neue Transaktion geöffnet, innerhalb der sowohl das Verarbeitungsergebnis persistent gespeichert als auch der Jobstatus von *Running* auf *Succeeded* aktualisiert wird. Beide Änderungen werden gemeinsam committet. Muss ein Job dagegen aufgrund eines Fehlers oder eines abgelaufenen Leases zurückgesetzt werden, werden innerhalb einer zusammengehörigen

Transaktion der Status auf *Queued* zurückgesetzt sowie die Attribute *claimed_by* und *lease_until* zurückgesetzt. Dadurch ist sichergestellt, dass niemals ein Zustand entstehen kann, in dem zwar bereits ein Ergebnis gespeichert wurde, der Job jedoch weiterhin als *Running* oder *Queued* gekennzeichnet ist oder umgekehrt. Die gemeinsame Speicherung von Ergebnis und Jobstatus innerhalb derselben lokalen ACID-Transaktion gewährleistet somit die in Z-4 geforderte Atomarität. Damit werden sämtliche Änderungen am Zustand eines Jobs atomar durchgeführt, während die eigentliche Fachlogik vollständig außerhalb der Transaktionsgrenzen verbleibt.

Für die Synchronisation konkurrierender Worker wird ein pessimistisches Sperrverfahren verwendet. Wie bereits in Kapitel 2.2.3 erläutert, eignen sich optimistische Synchronisationsverfahren insbesondere für Systeme, in denen konkurrierende Änderungen nur selten auftreten. Im entwickelten Job-Processing-System ist jedoch das Gegenteil der Fall. Da alle Worker neue Jobs entsprechend ihres Erstellungszeitpunkts aus derselben Warteschlange entnehmen, konkurrieren sie regelmäßig um dieselben Datensätze. Optimistische Verfahren würden in diesem Szenario zu einer hohen Anzahl abgebrochener Transaktionen und entsprechender Wiederholungsversuche führen. Kleppmann (2017, Kap. 7, Abschnitt „Pessimistic versus optimistic concurrency control“) weist darauf hin, dass sich optimistische Verfahren bei hoher Konkurrenz negativ auf den Gesamtdurchsatz auswirken, da die zusätzlich notwendigen Wiederholungen die Datenbank weiter belasten. Aus diesem Grund wird im Rahmen dieser Arbeit ein pessimistisches Sperrverfahren verwendet, bei dem konkurrierende Zugriffe bereits während der Auswahl eines Jobs verhindert werden. Durch die Nutzung expliziter Sperren wird die exklusive Bearbeitung unabhängig vom verwendeten Isolation Level gewährleistet. Aus diesem Grund genügt das in PostgreSQL standardmäßig verwendete Isolation-Level READ COMMITTED, da die notwendige Synchronisation durch die expliziten Sperren erfolgt und strengere Isolationsebenen für den beschriebenen Ablauf keinen zusätzlichen Nutzen bieten.

Die Nutzung derselben relationalen Datenbank als Job-Store und zur Speicherung der Verarbeitungsergebnisse bietet den großen Vorteil, dass sämtliche Zustandsänderungen sowie die Persistierung eines Verarbeitungsergebnisses innerhalb lokaler ACID-Transaktionen erfolgen. Die Koordination mehrerer unabhängiger Systeme und damit die Notwendigkeit verteilter Transaktionen entfallen vollständig.

5.1.6 Retry-Strategie

Auf Grundlage der in Kapitel 2.3.2 beschriebenen Prinzipien werden Wiederholungsmechanismen gezielt an denjenigen Stellen der Architektur eingesetzt, an denen temporäre Infrastrukturfehler auftreten können, ohne dass die eigentliche Fachlogik fehlerhaft ist. Ziel ist es im Rahmen von Z-6, kurzfristige Störungen durch Wiederholungsversuche zu kompensieren und dadurch unnötige Recovery-Vorgänge oder Benutzerinteraktionen zu vermeiden. Hierzu unterscheidet die Architektur zwischen der Kommunikation des Clients mit dem Backend und den internen Datenbankzugriffen durch die Worker und die Job-API. Beide Kommunikationswege stellen voneinander unabhängige Fehlerquellen dar und können daher unabhängig voneinander von transienten Infrastrukturfehlern betroffen sein.

Auf Clientseite werden Wiederholungsversuche für die Kommunikation mit der REST-Schnittstelle des Backends vorgesehen. Kann eine Anfrage aufgrund einer temporären Netzwerkstörung oder einer kurzfristigen Nichtverfügbarkeit des Backends nicht erfolgreich abgeschlossen werden bzw. bleibt die Erfolgsbestätigung aus, wird die Anfrage automatisch erneut übertragen. Da die Joberstellung durch den in Kapitel 5.1.4 beschriebenen Idempotenzmechanismus abgesichert ist, kann eine erneute Übertragung derselben Anfrage nicht zu einer mehrfachen Persistierung eines Jobs führen. Die Kombination aus Retry und Idempotenz schafft somit eine Exactly-Once-Semantik für die Joberstellung unter der Voraussetzung, dass der Fehler tatsächlich transient ist und die Parameter für Retry und Backoff passend konfiguriert sind.

Auch innerhalb des Backends werden Wiederholungsversuche für Datenbankoperationen vorgesehen, deren Fehlschlagen auf transiente Infrastrukturfehler zurückzuführen ist, beispielsweise kurzzeitige Netzwerkunterbrechungen oder die vorübergehende Nichtverfügbarkeit der Datenbank. Kann eine Operation aufgrund eines solchen Fehlers nicht abgeschlossen werden, wird sie automatisch erneut ausgeführt. Dagegen werden fachliche Fehler während der eigentlichen Jobverarbeitung bewusst nicht wiederholt, da deren Ursache nicht durch einen erneuten Ausführungsversuch beseitigt werden kann.

Zur strukturellen Entkopplung der Wiederholungslogik wird diese in einem eigenständigen Retry-Teilsystem gekapselt. Wie in Abbildung 5.6 dargestellt, nutzen die jeweiligen Komponenten, wie beispielsweise der Client im Frontend zur Erstellung der Jobs, einen zentralen RetryExecutor, der die auszuführende Operation über die Methode *execute(...)* entgegennimmt und deren Ausführung entsprechend einer konfigurierbaren Retry-Policy steuert. Der RetryExecutor enthält dabei selbst keine anwendungsspezifischen Entschei-

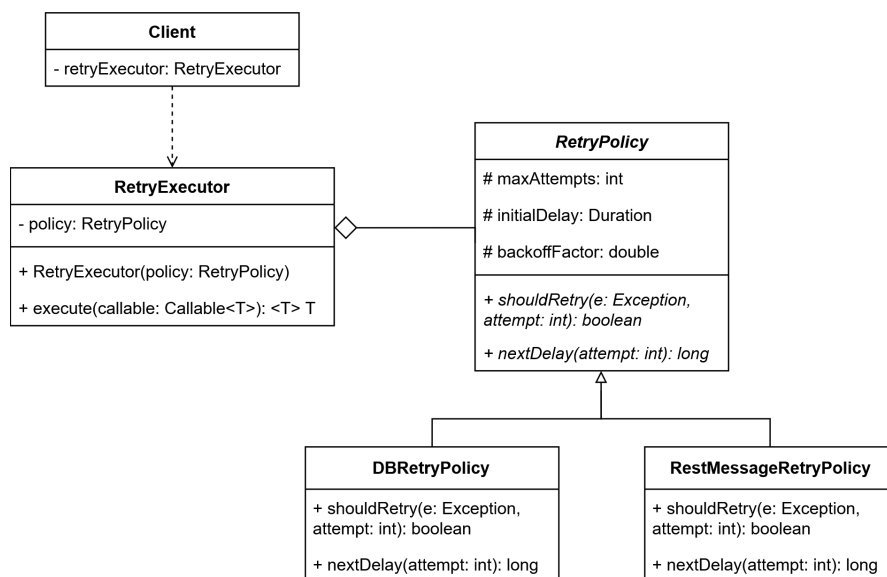


Abbildung 5.6: Klassendiagramm der Retry-Komponente

dungen darüber, wann eine Wiederholung erfolgen soll. Stattdessen fängt er Exceptions

von der auszuführenden Operation und gibt sie an *shouldRetry(...)* weiter. Die spezifische Policy entscheidet dann, ob ein Retry sinnvoll ist oder nicht, indem sie die Exception klassifiziert und einen entsprechenden Wahrheitswert zurückliefert. Die Retry-Policies implementieren dabei eigene unterschiedliche Kriterien für Wiederholungsversuche, die sich in den behandelten Fehlerarten unterscheiden. Außerdem definieren sie auch, wie das Backoff-Verhalten umgesetzt wird. Für das Job-Processing-System wird dabei ein exponentielles Backoff verwendet. Durch die zunehmende Wartezeit zwischen aufeinanderfolgenden Wiederholungsversuchen wird verhindert, dass bei transienten Infrastrukturfehlern in kurzer Zeit eine große Anzahl identischer Anfragen erzeugt wird. Dadurch wird die Belastung bereits beeinträchtigter Ressourcen, insbesondere der Datenbank oder der Netzwerkverbindung, reduziert und gleichzeitig die Wahrscheinlichkeit erhöht, dass sich die zugrunde liegende Störung bis zum nächsten Wiederholungsversuch bereits selbst behoben hat. Durch die Nutzung eines solchen Strategy-Patterns kann dasselbe Retry-Teilsystem sowohl für die Kommunikation zwischen Client und Backend als auch für interne Datenbankzugriffe der Worker verwendet werden, obwohl sich die jeweils behandelten Fehlerarten unterscheiden. Die Retry-Steuerung bleibt dadurch zentral gekapselt, während die konkrete Fehlerbehandlung an den jeweiligen Anwendungskontext angepasst werden kann. Die konkrete Implementierung vom *RetryExecutor* sowie die Konfiguration der einzelnen Retry-Policies werden in Kapitel 5.2.6 erläutert.

5.1.7 Recovery

Die in Kapitel 5.1.6 beschriebene Retry-Strategie behandelt ausschließlich transiente Infrastrukturfehler, bei denen die ausführende Komponente weiterhin aktiv ist. Fällt ein Worker dagegen während der Verarbeitung vollständig aus oder bleibt dauerhaft hängen, kann kein lokaler Retry mehr erfolgen. Zur Behandlung solcher Fehler wird die Architektur daher um einen eigenständigen Recovery-Service ergänzt. Dieser überprüft den Job-Store in regelmäßigen Abständen auf potenziell verwaiste Jobs. Hierzu werden sämtliche Jobs betrachtet, die sich weiterhin im Zustand *Running* befinden, deren Lease jedoch bereits abgelaufen ist. Ein abgelaufenes Lease wird dabei als Indikator dafür verwendet, dass die ursprüngliche Verarbeitung nicht innerhalb der vorgesehenen Zeit abgeschlossen werden konnte. Die Architektur verzichtet im Rahmen der Prototypentwicklung bewusst auf einen Heartbeat-Mechanismus und nutzt stattdessen zeitbasierte Leases, da diese ohne zusätzliche Kommunikation zwischen Worker und Job-Store auskommen und den Implementierungsaufwand reduzieren. Als Alternative ist der Einsatz eines Heartbeat-Mechanismus aber denkbar. Durch regelmäßige Verlängerungen der Leases könnte dieser verhindern, dass lang laufende, aber weiterhin aktive Verarbeitungen fälschlicherweise als ausgefallen eingestuft und erneut zur Bearbeitung freigegeben werden, wodurch der noch arbeitende Worker das Ergebnis nicht mehr speichern darf. Ein solches Verhalten könnte bei sehr aufwendigen Jobs verhindern, dass diese jemals erfolgreich beendet werden. Dem steht jedoch eine höhere Anzahl periodischer Schreibzugriffe auf den Job-Store gegenüber, wodurch die Datenbank zusätzlich belastet wird. Da der Heartbeat-Mechanismus für den im Rahmen dieser Arbeit entwickelten Prototypen und dessen Zielsetzung keinen nennenswerten Mehrwert bietet, werden lediglich zeitbasierte Leases genutzt.

Für jeden erkannten verwaisten Job entscheidet der Recovery-Service anschließend über das weitere Vorgehen. Wurde die maximal zulässige Anzahl an Ausführungsversuchen noch nicht erreicht, wird der Job nach dem in Kapitel 5.1.5 vorgestellten Transaktionschema erneut zur Verarbeitung freigegeben. Anschließend kann der Job von einem beliebigen Worker erneut beansprucht werden. Wurde die konfigurierte Obergrenze der Wiederholungsversuche bereits erreicht, wird der Job stattdessen in den Endzustand *Failed* überführt und nicht erneut verarbeitet. Dadurch wird verhindert, dass dauerhaft fehlerhafte Jobs unbegrenzt Systemressourcen belegen und nie terminieren (Z-9).

Durch diesen Mechanismus können insbesondere Worker-Abstürze, JVM-Abstürze, Container-Neustarts sowie dauerhaft blockierte Worker in Bezug auf die Jobausführung behandelt werden (Z-7). Die eigentliche Terminierung oder der Neustart fehlerhafter Worker ist dagegen nicht Bestandteil des entwickelten Systems. Diese Aufgabe wird bewusst an eine externe Prozessüberwachung oder Orchestrierungsplattform, beispielsweise Kubernetes, delegiert. Das entwickelte System stellt lediglich sicher, dass durch den Ausfall eines Workers kein Job dauerhaft verloren geht oder hängen bleibt.

Die Wiederaufnahme eines Jobs erfolgt in der entwickelten Architektur grundsätzlich nicht durch das Fortsetzen eines unterbrochenen Ausführungszustands, sondern durch eine vollständige Neuausführung auf Basis der ursprünglich persistierten Eingabedaten. Aus diesem Grund muss die Payload eines Jobs mindestens bis zum erfolgreichen Abschluss unverändert im Job-Store verbleiben. Dadurch kann jeder Wiederholungsversuch unter identischen Ausgangsbedingungen beginnen und unvollständig abgeschlossene Ausführungsversuche haben keine Auswirkungen auf den Erfolg oder das Ergebnis späterer Wiederholungen (Z-8).

Problematisch ist ein solches Vorgehen jedoch, wenn während der Verarbeitung externe Seiteneffekte entstehen. Wird beispielsweise wie in Abbildung 5.7 im Rahmen eines Jobs eine E-Mail versendet und der Worker stürzt anschließend vor dem erfolgreichen Abschluss der Verarbeitung ab, so wird der Job durch den Recovery-Service erneut zur Verarbeitung freigegeben. Die erneute Ausführung würde in diesem Fall dieselbe E-Mail ein weiteres Mal versenden. Die entwickelte Infrastruktur garantiert daher ausschließlich die idempotente Verarbeitung der internen Fachlogik des Job-Processing-Systems. Für externe Seiteneffekte kann eine Exactly-Once-Ausführung dagegen grundsätzlich nicht gewährleistet werden. Wie schon in Kapitel 2.1.2 erwähnt ist dies nur möglich, wenn das jeweils aufgerufene Zielsystem selbst eine idempotente Verarbeitung unterstützt, beispielsweise durch Nutzung von Idempotenzschlüsseln wie im Beispielsystem bei der Joberstellung (vgl. Kapitel 5.1.4). Unterstützt das Zielsystem keine entsprechende Semantik, kann eine Mehrfachausführung von Seiteneffekten infolge eines Recovery-Vorgangs nicht ausgeschlossen werden.

Ein Nebeneffekt dieser Architektur besteht darin, dass ein Job zeitweise von mehreren Workern gleichzeitig bearbeitet werden kann. Überschreitet beispielsweise ein Worker seine Lease-Dauer, wird der Job erneut freigegeben, obwohl der ursprüngliche Worker seine Verarbeitung möglicherweise noch beendet. Die beschriebene Recovery-Strategie versucht durch den Retry, die Erfolgswahrscheinlichkeit eines Jobs zu erhöhen. Sie verfolgt damit das Ziel einer At-least-Once-Ausführung von Jobs, erzeugt damit aber die Möglichkeit der Mehrfachausführung. Die daraus resultierenden Anforderungen an die idempotente

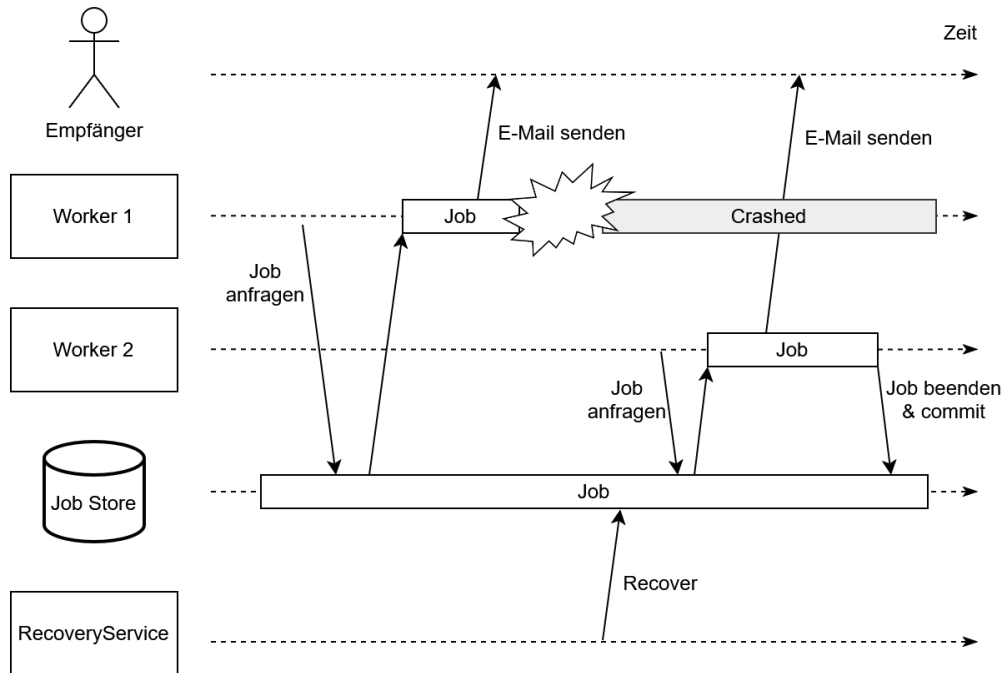


Abbildung 5.7: Mehrfachausführung mit Seiteneffekten

Verarbeitung werden im folgenden Abschnitt betrachtet.

5.1.8 Idempotente Ergebnisgenerierung

Z-8 verlangt, dass pro Job maximal ein Ergebnis erzeugt wird. Das in Kapitel 5.1.5 vorgestellte Claiming-Verfahren stellt sicher, dass ein Job im Normalfall ausschließlich von einem einzelnen Worker bearbeitet wird. Durch den in Kapitel 5.1.7 genauer betrachteten Recovery-Fall kann diese Eigenschaft jedoch nicht immer garantiert werden. Daher muss sichergestellt werden, dass das fachliche Ergebnis nur einmal gespeichert werden kann oder die Fachlogik auch bei mehrfacher Ausführung in jedem Fall immer das gleiche Ergebnis erzeugt. Da der Jobstatus sowie die Jobergebnisse in derselben Datenbank verwaltet werden, bietet sich die Umsetzung der ersten Option mithilfe von Transaktionen an. Um die idempotente Verarbeitung auf diese Weise sicherzustellen, darf ein Worker ein Verarbeitungsergebnis ausschließlich dann persistent speichern, wenn er laut Job-Store weiterhin der Besitzer des Jobs ist und für den Job noch kein Ergebnis vorliegt. Vor der Speicherung wird daher der betroffene Datensatz gesperrt und innerhalb einer transaktional geschützten Operation erneut der aktuelle Jobstatus, die Worker-ID sowie die Gültigkeit des Leases überprüft. Nur wenn der Job sich weiterhin im Zustand *Running* befindet, die gespeicherte Worker-ID mit der des ausführenden Workers übereinstimmt und das Lease noch nicht abgelaufen ist, wird das Ergebnis übernommen und der Job erfolgreich abgeschlossen. Andernfalls wird der Speichervorgang verworfen, da bereits ein anderer Worker die weitere Verarbeitung übernommen hat oder der Job bereits

abgeschlossen wurde. Dieses Prinzip wird auch „fencing“ (Kleppmann 2017, Kap. 8 Abschnitt „Fencing tokens“) genannt. Dadurch kann jeder Job höchstens einmal erfolgreich gespeichert werden, selbst wenn mehrere Worker denselben Job parallel ausgeführt haben. Das in Kapitel 5.1.7 bereits erwähnte Problem bezüglich Seiteneffekten in externen Systemen bleibt jedoch bestehen.

5.2 Implementierung

Die sich aus TZ3 ergebende Implementierung des Systems orientiert sich unmittelbar an dem in Kapitel 5.1 erarbeiteten Systementwurf zur Umsetzung der definierten Zusicherungen. Da einzelne Garantien, wie die atomare Verarbeitung oder die Idempotenz, erst durch das Zusammenwirken mehrerer Komponenten realisiert werden, erfolgt die Darstellung der Implementierung anhand der zentralen Bausteine und Mechanismen des Systems.

Das Kapitel gliedert sich wie folgt: Kapitel 5.2.1 begründet zunächst die Auswahl der verwendeten Technologien. In Kapitel 5.2.2 wird die datenbankseitige Abbildung der Entitäten sowie die Kapselung der SQL-Zugriffe im Repository behandelt. Kapitel 5.2.3 erläutert den Aufbau und Ablauf der Joberstellung, woraufhin Kapitel 5.2.4 die Komponenten zur Verarbeitung der Jobs beschreibt. Der atomare Abschluss von Jobs und die Absicherung gegen fehlerhafte Doppelverarbeitungen mittels *Fencing* wird in Kapitel 5.2.5 dargelegt. Schließlich widmen sich Kapitel 5.2.6 der konkreten Umsetzung des Retry-Mechanismus und Kapitel 5.2.7 der Wiederherstellung verwaister Jobs.

5.2.1 Auswahl verwendeter Technologien

Die Implementierung des entwickelten Background-Job-Processing-Systems erfolgt vollständig in Java. Als Entwicklungsplattform wird das Spring-Framework eingesetzt, da dieses eine umfangreiche Infrastruktur für den Aufbau transaktionsbasierter Client-Server-Anwendungen bereitstellt und gleichzeitig viele der im Architekturentwurf vorgesehenen Mechanismen direkt unterstützt. Darüber hinaus zählt Spring zu einem weit verbreiteten Frameworks für die Entwicklung von Unternehmensanwendungen und findet breite Anwendung in der industriellen Praxis. Dadurch entsteht eine realitätsnahe Entwicklungsumgebung, die gleichzeitig einen hohen Grad an Automatisierung bietet.

Für die Bereitstellung der REST-Schnittstelle wird Spring Web verwendet. Die Abbildung der HTTP-Endpunkte sowie die Serialisierung und Deserialisierung von JSON-Nachrichten erfolgen dadurch deklarativ über Annotationen. Die Persistenzschicht basiert auf Spring Data JPA. Durch die objektorientierte Abbildung relationaler Datenbanken sowie die Abstraktion reduziert JPA den Implementierungsaufwand für Standardoperationen erheblich. Gleichzeitig bleibt über native Queries die Möglichkeit erhalten, Operationen feingranular zu steuern sowie gezielt auf datenbankspezifische Funktionen und optimierte SQL-Anweisungen zurückzugreifen. Die benötigten transaktionalen Eigenschaften werden mithilfe von Spring Transactions über `@Transactional` umgesetzt.

Als Job-Store wird PostgreSQL eingesetzt. Die Datenbank bietet die für das entwickelte System benötigten Mechanismen zur transaktionalen Verarbeitung und Synchronisation

konkurrierender Zugriffe. Insbesondere die Unterstützung von `SELECT ... FOR UPDATE SKIP LOCKED` ermöglicht für einen pull-basierten Ansatz eine effiziente Koordination mehrerer Worker in einer Job Queue, ohne dass Worker sich gegenseitig blockieren und Wartesituationen entstehen.

Die Kommunikation zwischen Frontend und Backend erfolgt über den von Spring bereitgestellten `RestClient`. Die Verarbeitung der Jobs wird innerhalb des Backends durch einen konfigurierbaren Thread-Pool realisiert, der auf dem `ExecutorService` der Java-Concurrency-Bibliothek basiert. Dadurch kann die Anzahl gleichzeitig aktiver Worker unabhängig von der übrigen Systemarchitektur konfiguriert werden.

Die Auswahl der Technologien orientiert sich somit an den Anforderungen des entwickelten Job-Processing-Systems. Die bereitgestellten Framework-Funktionen reduzieren den Implementierungsaufwand für Infrastrukturaufgaben und ermöglichen es, den Fokus auf die Entwicklung und Evaluation der im Architekturentwurf beschriebenen Mechanismen zu legen.

5.2.2 Persistenz und Datenbankzugriff

Für den Zugriff auf die relationale Datenbank wird Spring Data JPA verwendet. Die Abbildung der persistenten Daten erfolgt durch die Entitätsklassen `Job` und `JobResult`, die jeweils mit der Annotation `@Entity` versehen sind.

Der Status eines Jobs wird als Java-Enumeration modelliert und über JPA persistent gespeichert. Damit wird sichergestellt, dass ausschließlich die im Zustandsmodell definierten Zustände verwendet werden können. Die Nutzdaten eines Jobs werden als JSON in der Datenbank persistiert. Dadurch kann die *Payload* eine Menge an strukturierten Informationen für die Jobspezifikation beinhalten. Alternativ könnte die *Payload* auch in Abhängigkeit von der Domain und dem Systementwurf als eigene Entität modelliert werden mit einer 1:1-Beziehung zum Job.

Eine besondere Bedeutung besitzt der *Idempotency-Key*, der zusätzlich zur technischen Identifikation eines Jobs gespeichert wird. Dieser Schlüssel wird vom Client als UUID erzeugt und dient zur Erkennung mehrfach übermittelter Anfragen. Auf Datenbankebene wird daher ein `UNIQUE`-Constraint für das entsprechende Attribut definiert. Dadurch kann die Datenbank selbst sicherstellen, dass die Idempotency-Keys eindeutig sind. In Kombination mit der Kennzeichnung *nullable=false* kann so die Eindeutigkeit erstellter Jobs auf Datenbankebene sichergestellt werden (Z-2).

Die Verarbeitungsergebnisse werden durch die Entitätsklasse `JobResult` repräsentiert. Umgesetzt wird die 1:1-Beziehung zwischen `JobResult` und `Job` über die `@OneToOne`-Annotation. Dabei wird die Fremdschlüsselbeziehung über `@JoinColumn` in der Tabelle des Ergebnisses umgesetzt. Die Beschränkung auf höchstens ein Ergebnis pro Job ist insbesondere für die idempotente Ergebnisspeicherung relevant (Z-8). Daher wird auch der Fremdschlüssel des `JobResult` mit einem `UNIQUE`-Constraint belegt.

Der Datenbankzugriff wird über ein Repository-Interface gekapselt. Neben den von Spring Data JPA bereitgestellten Standardoperationen werden für das Job-Processing-System spezifische Abfragen definiert, die unmittelbar die Anforderungen an die konkurrierende Verarbeitung und die Recovery unterstützen. Für die Auswahl eines Jobs

zur Verarbeitung wird beispielsweise die in listing 5.1 dargestellte native SQL-Abfrage definiert.

```
1 SELECT *
2 FROM jobs
3 WHERE status = :status
4 ORDER BY created_at
5 LIMIT :limit
6 FOR UPDATE SKIP LOCKED
```

Listing 5.1: Claiming eines Jobs im JobRepository

FOR UPDATE sperrt dabei den ausgewählten Datensatz für die laufende Transaktion zum Zwecke des exklusiven Zugriffs (Z-3), während SKIP LOCKED bereits durch andere Transaktionen gesperrte Datensätze ignoriert. Dadurch können mehrere Worker gleichzeitig nach verfügbaren Jobs suchen, ohne sich gegenseitig durch gesperrte Datensätze zu blockieren. Die Sortierung nach dem Erstellungszeitpunkt stellt dabei sicher, dass ältere Jobs bevorzugt verarbeitet werden (Z-9).

Für den Abschluss einer Verarbeitung und Recovery-Szenarien wird der in listing 5.2 gezeigte Befehl zum gezielten Zugriff auf einzelne Jobs bereitgestellt, der mit der Auswahl eine Sperre für diesen Datensatz setzt.

```
1 SELECT *
2 FROM jobs
3 WHERE id = :jobId
4 FOR UPDATE
```

Listing 5.2: Auswählen eines Jobs zur Bearbeitung

Auch dieser Befehl dient dazu, konkurrierende Änderungen an einem konkreten Job zu verhindern (Z-3).

Zusätzlich stellt das Repository eine Abfrage bereit, mit der Jobs anhand der Kombination ihres Status und des Ablaufs ihres Leases identifiziert werden können. Damit bildet die Abfrage aus listing 5.3 die Grundlage für die Erkennung verwaister Jobs durch den Recovery-Mechanismus.

```
1 SELECT j.id
2 FROM Job j
3 WHERE j.status = :status
4 AND j.lease_until < :timestamp
```

Listing 5.3: Finden verwaister Jobs

Wird der gesuchte Status auf *Running* und der `timestamp` auf den aktuellen Zeitpunkt gesetzt, liefert die Anfrage alle Jobs, die noch nicht beendet wurden und deren Lease gleichzeitig abgelaufen ist.

Die Transaktionskapselung wird durch `@Transactional` erreicht. Alle Datenbankoperationen in einer mit `@Transactional` gekennzeichneten Methode werden im Rahmen einer gemeinsamen Transaktion ausgeführt. Die konkrete Nutzung zur Umsetzung der in

Kapitel 5.1 definierten Transaktionen wird bei den jeweiligen Verarbeitungsmechanismen in den folgenden Abschnitten erläutert.

Die implementierte Persistenzschicht stellt damit neben der dauerhaften Speicherung der Jobs und Ergebnisse (Z-7) auch die für die Fehlertoleranz erforderlichen Mechanismen zur Sicherstellung von Konsistenz (Z-1, Z-2, Z-4, Z-8) und zur Synchronisation konkurrierender Zugriffe (Z-3) bereit.

5.2.3 Joberstellung

Die Erstellung eines Jobs beginnt auf Clientseite mit der Zusammenstellung der für die Verarbeitung benötigten Nutzdaten. Diese werden als *Payload* in Form eines JSON-Objekts repräsentiert. Zusätzlich erzeugt der Client einen UUID-basierten *Idempotency-Key*, der gemeinsam mit der Payload in dem Data Transfer Object (DTO) `CreateJobRequest` zusammengefasst wird.

Listing 5.4 zeigt, wie das Request-Objekt anschließend über den im Frontend implementierten `BackendRestClient` an die REST-Schnittstelle des Backends übertragen wird. Die Anfrage wird als HTTP-POST-Request an den Endpunkt `/job` gesendet und enthält das zuvor erzeugte Request-Objekt im HTTP-Body.

```
1 public ResponseEntity<CreateJobResponse> createJob(CreateJobRequest request) {  
2     return restClient.post()  
3         .uri("/job")  
4         .body(request)  
5         .retrieve()  
6         .toEntity(CreateJobResponse.class);  
7 }
```

Listing 5.4: POST eines Jobs

Im Backend wird die Anfrage durch den mit Spring Web implementierten `JobController` entgegengenommen. Der entsprechende Endpunkt ist über `@PostMapping("/job")` definiert und in listing 5.5 dargestellt. Der HTTP-Body wird durch `@RequestBody` automatisch in ein `CreateJobRequest`-Objekt überführt und in zwei Schritten validiert. Zunächst prüft Spring auf Ebene der REST-Schnittstelle mithilfe von `@Valid`, ob die durch Bean-Validation-Annotationen wie `@NotNull` oder `@NotBlank` definierten syntaktischen Anforderungen erfüllt sind. Anschließend erfolgt eine fachliche Validierung der Eingangsdaten durch einen separaten Validator. Schlägt eine der beiden Validierungen fehl, beantwortet der Controller die Anfrage mit dem HTTP-Status 400 `Bad Request`. Anschließend delegiert der Controller die eigentliche Joberstellung an den `JobService`. Nach erfolgreicher Erstellung gibt der Controller den HTTP-Status 201 `Created` zurück. Schlägt die Persistierung dagegen aufgrund einer Verletzung des `UNIQUE`-Constraints für den *Idempotency-Key* fehl, wird die daraus resultierende `DataIntegrityViolationException` abgefangen. Der Controller prüft, ob tatsächlich eine Verletzung des `UNIQUE`-Constraints vorliegt. Ist dies der Fall, wird der bereits vorhandene Job anhand seines Idempotency-Keys aus dem Job-Store geladen und gemeinsam mit dem HTTP-Status 200 `OK` an den Client zurückgegeben. Andere Datenbankfehler werden dagegen nicht als Idempotenzfall interpretiert und führen zu einer Antwort mit 500 `Internal Server Error`. Für den Client

ist damit unabhängig davon, ob ein Job erstmals angelegt oder eine bereits erfolgreich ausgeführte Anfrage erneut übermittelt wurde, eindeutig erkennbar, dass der gewünschte Job im System vorhanden ist.

```
1 @PostMapping("/job")
2 public ResponseEntity<CreateJobResponse> createJob(@Valid @RequestBody
3     CreateJobRequest request) {
4     if (!validator.isValid(request.getPayload()))
5         return ResponseEntity.badRequest().build();
6     try {
7         Job job = jobService.createJob(request);
8         return ResponseEntity
9             .status(HttpStatus.CREATED)
10            .body(new CreateJobResponse(job));
11    } catch (DataIntegrityViolationException e){
12        if (!isConstraintViolation(e))
13            throw e;
14        Job job = jobService.getJobById(request.getIdempotencyKey())
15            .orElseThrow(IllegalStateException::new);
16        return ResponseEntity.ok(new CreateJobResponse(job));
17    } catch (Exception e) {
18        return ResponseEntity.internalServerError().build();
19    }
```

Listing 5.5: POST Endpoint

Die eigentliche Erstellung und Persistierung des Jobs übernimmt der `JobService`. Hierzu wird zunächst über eine Factory aus dem Request-DTO eine neue `Job`-Entität erzeugt. Anschließend wird diese innerhalb einer durch `@Transactional` gekapselten Serviceoperation atomar persistiert und an den Controller zurückgegeben.

5.2.4 Claiming und Verarbeitung

Die parallele Verarbeitung der Jobs wird durch den `WorkerManager` gesteuert, dessen Verhalten in listing 5.6 dargestellt ist. Dieser erzeugt beim Start der Anwendung einen Thread-Pool und übergibt jedem darin ausgeführten Thread eine eigene `Worker`-Instanz. Die Anzahl der parallel arbeitenden Worker wird über die Anwendungskonfiguration festgelegt und ist somit an den Systemkontext anpassbar. Jeder Worker erhält dabei Zugriff auf den `WorkerService` sowie auf eine Implementierung des `Processor`-Interfaces. Die Initialisierung des Thread-Pools und das Starten der Worker erfolgt daher nach Erzeugung des Spring-Kontexts über die mit `@PostConstruct` annotierte Methode `start()`, um zu garantieren, dass der `WorkerService` und der `Processor` bereits durch Spring injiziert wurden.

```
1 public WorkerManager(WorkerService workerService, Processor jobProcessor,
2     @Value("${worker.count}") int workerCount) {
3     this.workerService = workerService;
4     this.jobProcessor = jobProcessor;
```

```

5     this.workerCount = workerCount;
6     this.executorService = Executors.newFixedThreadPool(workerCount);
7 }
8
9 @PostConstruct
10 void start() {
11     for (int i = 0; i < workerCount; i++) {
12         executorService.submit(new Worker(workerService, jobProcessor));
13     }
14 }

```

Listing 5.6: Initialisierung des Worker-Pools

Die Steuerung der eigentlichen Verarbeitung erfolgt innerhalb der **Worker**-Instanzen. Jeder Worker besitzt eine eigene UUID, die bei seiner Erzeugung generiert wird. Diese wird beim Beanspruchen eines Jobs als *claimed_by* im Job-Store hinterlegt und ermöglicht dadurch die eindeutige Zuordnung eines laufenden Jobs zu demjenigen Worker, der ihn aktuell verarbeitet. Der **ExecutorService** ruft automatisch die in listing 5.7 dargestellte *run()*-Methode der übergebenen **Worker**-Instanz auf. Innerhalb der Methode wird für die Verarbeitung der Jobs aus dem Job-Store gesorgt. Hierzu versucht der **Worker** zunächst, über die Methode *claimNextJob(UUID id)* des **WorkerService** einen verfügbaren Job zu beanspruchen. Wird kein Job gefunden, ist das **Optional**-Objekt leer. Der Worker wird daraufhin für eine kurze Zeit pausiert und führt anschließend eine erneute Abfrage durch. Beendet wird die Schleife erst, wenn der Thread unterbrochen wird.

```

1 @Override
2 public void run() {
3     while (!Thread.currentThread().isInterrupted()) {
4         try {
5             Optional<Job> job = workerService.claimNextJob(id);
6             if (job.isEmpty()) {
7                 if (!sleep(2000)) {
8                     break;
9                 }
10                continue;
11            }
12            JobResult result = processor.process(job.get());
13            workerService.finishJob(job.get().getId(), id, result);
14        } catch (Exception e) {
15            e.printStackTrace();
16        }
17    }
18 }

```

Listing 5.7: Ausführungsschleife eines Workers

Findet *claimNextJob* hingegen einen Job, sperrt die Methode diesen und gibt ihn an die **Worker**-Instanz zurück. Anschließend wird der Job über den **Processor** verarbeitet und das Ergebnis gespeichert.

Die Implementierung von *claimNextJob* ist in listing 5.8 veranschaulicht. Sie ist als transaktionale Operation implementiert und umfasst sowohl die Auswahl eines freien Jobs als auch dessen Überführung in den Zustand *Running*. Hierfür greift das Repository auf die in Kapitel 5.2.2 beschriebene Abfrage mit `FOR UPDATE SKIP LOCKED` zurück. Die Methode fordert dabei zunächst einen Job im Zustand *Queued* an. Existiert kein geeigneter Datensatz, wird ein leeres `Optional` zurückgegeben. Andernfalls wird der Datensatz durch das `JobRepository` gesperrt, die für die laufende Verarbeitung relevanten Attribute (vgl. Kapitel 5.1.5) werden innerhalb einer gemeinsamen Transaktion aktualisiert und der Job wird zurückgegeben. Besonders wichtig ist neben dem Status die Aktualisierung von *lease_until* für die Recovery, indem eine festgelegte Dauer auf den über `Instant` angefragten aktuellen Zeitpunkt addiert wird.

```
1 @Transactional
2 public Optional<Job> claimNextJob(UUID workerId) {
3     Optional<Job> result = jobRepository.findByStatus(JobStatus.QUEUED, 1);
4     if (result.isEmpty()) {
5         return Optional.empty();
6     }
7     Job job = result.get();
8     job.setStatus(JobStatus.RUNNING);
9     job.setClaimed_by(workerId);
10    job.setAttempt_count(job.getAttempt_count() + 1);
11    job.setLease_until(
12        Instant.now().plusSeconds(maximumJobDuration));
13    job.setUpdatedAt(Instant.now());
14
15    return Optional.of(job);
16 }
```

Listing 5.8: Beanspruchen eines Jobs

Durch die transaktionale Kapselung werden Auswahl und Statusänderung atomar durchgeführt. Nach Abschluss der Methode wird die Transaktion beendet und der Job befindet sich im Job-Store im Zustand *Running*. Die eigentliche Verarbeitung findet somit nicht innerhalb dieser Transaktion statt. Dadurch bleibt die Datenbankverbindung nur für die zur Reservierung erforderliche Zeit belegt.

Nach erfolgreichem Claiming übergibt der Worker den Job zur Verarbeitung an seine `Processor`-Instanz. Die eigentliche Fachlogik wird dadurch von der allgemeinen Worker-Infrastruktur getrennt. Das `Processor`-Interface definiert hierzu lediglich die Methode `process(Job)`, die einen Job entgegennimmt und ein `JobResult` zurückliefert. Der Worker selbst enthält damit keine jobspezifische Verarbeitungslogik. Stattdessen kann dieselbe Worker-Infrastruktur für unterschiedliche Arten von Hintergrundjobs verwendet werden, indem eine entsprechende `Processor`-Implementierung bereitgestellt wird. Im konkreten System übernimmt die Klasse `JobProcessor` diese Aufgabe. Die Verarbeitung eines Jobs findet vollständig außerhalb einer offenen Datenbanktransaktion statt. Der `Processor` kann daher auch länger laufende Berechnungen oder externe E/A-Operationen durchführen, ohne dabei eine Datenbankverbindung oder eine Sperre auf dem Job zu

halten. Erzeugt die Fachlogik erfolgreich ein `JobResult`, wird dieses anschließend von der `Worker`-Instanz an den `WorkerService` zur Persistierung übergeben. Die dabei verwendete Transaktion sowie die Prüfung der Berechtigung des Workers zur Ergebnisspeicherung werden in Kapitel 5.2.5 behandelt.

Tritt während der eigentlichen Verarbeitung eine Exception auf, bricht die Verarbeitung ab und es wird kein Ergebnis gespeichert. Der Job verbleibt zunächst im Zustand *Running*. Die weitere Behandlung erfolgt in diesem Fall über den Recovery-Mechanismus, dessen Umsetzung in Kapitel 5.2.7 behandelt wird.

5.2.5 Abschluss einer Jobverarbeitung

Nach erfolgreicher Ausführung der Fachlogik übergibt der Worker das erzeugte `JobResult` zusammen mit der Job-ID und seiner eigenen Worker-ID an den `WorkerService`. Die in listing 5.9 dargestellte Methode `finishJob(...)` übernimmt dabei die atomare Persistierung des Verarbeitungsergebnisses sowie den Abschluss der Jobverarbeitung.

Wie bereits bei der Reservierung eines Jobs erfolgt auch dieser Schritt entsprechend dem Vorgehen aus Kapitel 5.1.5 atomar innerhalb einer Datenbanktransaktion mittels `@Transactional`.

```
1 @Transactional
2 public Job finishJob(UUID jobId, UUID workerId, JobResult jobResult)
3     throws JobNotFoundException {
4
5     Job job = jobRepository.findByIdForUpdate(jobId)
6         .orElseThrow(JobNotFoundException::new);
7
8     if (job.getStatus() != JobStatus.RUNNING)
9         throw new IllegalStateException();
10
11     if (!workerId.equals(job.getClaimed_by()))
12         throw new IllegalStateException();
13
14     if (job.getLease_until().isBefore(Instant.now()))
15         throw new LeaseExpiredException();
16
17     job.setStatus(JobStatus.SUCCEEDED);
18     resultRepository.save(jobResult);
19     job.setResult(jobResult);
20     job.setUpdatedAt(Instant.now());
21
22     return job;
23 }
```

Listing 5.9: Abschluss einer Jobverarbeitung

Zu Beginn der Methode wird der betreffende Job über `findByIdForUpdate(...)` geladen. Die zugrunde liegende SQL-Abfrage verwendet `FOR UPDATE` und sperrt den Datensatz dadurch exklusiv bis zum Abschluss der Transaktion. Dadurch wird verhindert, dass

Recovery-Prozesse oder andere konkurrierende Transaktionen den Job während seiner Finalisierung verändern können. Das Verhalten ist essenziell, um Z-8 garantieren zu können. Anschließend wird überprüft, ob sich der Job noch im Zustand *Running* befindet. Dadurch wird ausgeschlossen, dass bereits abgeschlossene oder zurückgesetzte Jobs erneut oder unberechtigt beendet werden. Danach erfolgt die Prüfung, ob die übergebene Worker-ID mit dem im Job gespeicherten Attribut *claimed_by* übereinstimmt. Nur der Worker, der den Job zuvor erfolgreich beansprucht hat, ist berechtigt, dessen Verarbeitung abzuschließen. Dieser Vergleich bildet die technische Umsetzung des im Architekturkapitel beschriebenen Fencing-Mechanismus. Hat der Job zwischenzeitlich einen anderen Besitzer erhalten, beispielsweise infolge eines Recovery-Vorgangs, wird die Ergebnisspeicherung abgebrochen. Zusätzlich wird geprüft, ob das zum Job gehörende Lease zum Zeitpunkt der Ergebnisspeicherung noch gültig ist. Ist das Lease bereits abgelaufen, wird eine *LeaseExpiredException* ausgelöst. Auch in diesem Fall wird kein Ergebnis persistiert, da nicht mehr sichergestellt werden kann, dass der Worker den Job weiterhin exklusiv besitzt. Die weitere Behandlung erfolgt anschließend durch den Recovery-Mechanismus. Erst nachdem sämtliche Prüfungen erfolgreich abgeschlossen wurden, erfolgt die eigentliche Persistierung des Verarbeitungsergebnisses. Hierzu wird zunächst der Jobstatus auf *Succeeded* gesetzt. Anschließend wird das *JobResult* gespeichert und dem entsprechenden Job zugeordnet. Schließlich wird noch der Zeitstempel der letzten Änderung aktualisiert. Da sämtliche Änderungen innerhalb derselben Datenbanktransaktion erfolgen, wird verhindert, dass ein Zustand entsteht, in dem zwar bereits ein Verarbeitungsergebnis existiert, der Job jedoch weiterhin als *Running* gekennzeichnet ist oder umgekehrt. Erst mit dem erfolgreichen Commit der Transaktion gilt die Verarbeitung eines Jobs als vollständig abgeschlossen. Schlägt eine der Konsistenzprüfungen fehl oder tritt während der Persistierung eine Exception auf, wird die laufende Transaktion automatisch zurückgerollt. Bereits durchgeführte Änderungen werden dadurch verworfen, sodass der Job-Store auch im Fehlerfall einen konsistenten Zustand beibehält (Z-4 & Z-8).

5.2.6 Retries

Kern des in Kapitel 5.1.6 beschriebenen Retry-Teilsystems ist die Klasse *RetryExecutor*, die die allgemeine Steuerung von Wiederholungsversuchen übernimmt. Der *RetryExecutor* besitzt lediglich eine öffentliche Methode *execute()*, der eine beliebige Operation in Form eines *Callable*-Objekts übergeben wird. Dadurch kann derselbe Mechanismus unabhängig davon verwendet werden, ob beispielsweise ein REST-Aufruf, ein Datenbankzugriff oder eine andere Operation wiederholt werden soll. Listing 5.10 zeigt den wesentlichen Aufbau der Implementierung.

```
1 public <T> T execute(Callable<T> callable) throws Exception {
2     int attempt = 1;
3     while (true) {
4         try {
5             return callable.call();
6         } catch (Exception e) {
7             if (!policy.shouldRetry(e, attempt)) {
```

```

8         throw e;
9     }
10    Thread.sleep(policy.nextDelay(attempt));
11    attempt++;
12 }
13 }
14 }

```

Listing 5.10: Generischer RetryExecutor

Die Methode führt die übergebene Operation zunächst unmittelbar aus. Tritt dabei keine Exception auf, wird das Ergebnis direkt an den Aufrufer zurückgegeben. Wird dagegen eine Exception ausgelöst, entscheidet die spezifische **RetryPolicy**, ob ein weiterer Ausführungsversuch sinnvoll ist. Hierzu erhält sie sowohl die aufgetretene Exception als auch die aktuelle Versuchsnummer. Lehnt die Policy einen weiteren Versuch ab, wird die Exception unverändert an den Aufrufer weitergereicht. Andernfalls berechnet die Policy zunächst die Dauer des nächsten Backoff-Intervalls und pausiert den Worker anschließend für diese Zeit, bevor die Operation erneut ausgeführt wird.

Die konkrete Retry-Strategie wird über das Interface **RetryPolicy** beschrieben. Dieses definiert die beiden Methoden **shouldRetry(...)** und **nextDelay(...)**. Während **shouldRetry(...)** entscheidet, ob ein Fehler erneut ausgeführt werden soll, bestimmt **nextDelay(...)** die Wartezeit bis zum nächsten Versuch. Dadurch werden Fehlerklassifikation und die Steuerung der Wartezeit vollständig vom eigentlichen Retry-Mechanismus getrennt.

Für die Berechnung des Backoff-Intervalls wird entsprechend dem Architekturentwurf ein exponentieller Backoff verwendet. Die Verzögerung ergibt sich aus

$$d_n = d_0 \cdot b^{n-1},$$

wobei d_n die Wartezeit des aktuellen Wiederholungsversuchs, d_0 die initiale Wartezeit, b den Backoff-Faktor und n die Nummer des aktuellen Wiederholungsversuchs bezeichnet. Mit jedem weiteren Fehlversuch vergrößert sich somit das Warteintervall exponentiell.

Für die unterschiedlichen Einsatzzwecke werden jeweils eigene Implementierungen der **RetryPolicy** verwendet. Listing 5.11 zeigt exemplarisch die Implementierung der für Datenbankzugriffe verwendeten Policy.

```

1  @Override
2  public boolean shouldRetry(Exception e, int attempt) {
3      if (attempt >= maxAttempts
4          || e instanceof DataIntegrityViolationException) {
5          return false;
6      }
7      return e instanceof DataAccessException;
8  }
9
10 @Override
11 public long nextDelay(int attempt) {
12     return (long) (initialDelay.toMillis()

```



```

13         * Math.pow(backoffFactor, attempt - 1));
14     }

```

Listing 5.11: Beispiel einer RetryPolicy

Die Datenbank-Policy klassifiziert ausschließlich Datenbankzugriffsfehler (`DataAccessException`), wozu unter anderem die Nichterreichbarkeit der Datenbank zählt, als potenziell transient und begrenzt gleichzeitig die maximale Anzahl der Wiederholungsversuche. Die `DataIntegrityViolationException` wird dabei explizit ausgeschlossen, obwohl sie eine Unterklasse der `DataAccessException` ist. Sie entsteht bei der Verletzung von Datenbank-Constraints und stellt daher keinen transienten, sondern einen permanenten Fehler dar, der durch einen erneuten Ausführungsversuch nicht behoben werden kann. Darüber hinaus muss diese Ausnahme unverändert an den `JobController` weitergegeben werden, da sie dort zur Erkennung einer Verletzung des `UNIQUE`-Constraints des Idempotency-Keys verwendet wird (vgl. Kapitel 5.2.3). Würde sie bereits vom `RetryExecutor` behandelt und nach Ausschöpfen der Wiederholungsversuche durch eine andere Ausnahme ersetzt, könnte der Controller nicht mehr zwischen einer erfolgreichen Neuerstellung und einer bereits zuvor erfolgten Joberstellung unterscheiden. Für die Kommunikation zwischen Client und Backend wird dagegen eine separate Policy eingesetzt, die insbesondere Netzwerkfehler, Verbindungsabbrüche sowie als temporär klassifizierte HTTP-Fehler berücksichtigt. Obwohl beide Policies unterschiedliche Fehlerarten behandeln, nutzen sie denselben generischen `RetryExecutor`.

Die Einbindung des Retry-Mechanismus erfolgt unmittelbar an den Stellen, an denen temporäre Infrastrukturfehler auftreten können. Die jeweilige Operation wird hierzu als Lambda-Ausdruck an den `RetryExecutor` übergeben. Listing 5.12 zeigt dies exemplarisch für einen Datenbankzugriff im Backend.

```

1 Job job = retryExecutor.execute(
2     () -> jobService.createJob(request)
3 );

```

Listing 5.12: Nutzung des RetryExecutors

Wird Zeile 6 aus listing 5.5 mit diesem Ausdruck ersetzt und die Datenbank ist beispielsweise nicht erreichbar, wird der Aufruf von `creatJob(request)` automatisch erneut versucht. Neben der Joberstellung wird derselbe Mechanismus ebenfalls beim Claiming neuer Jobs sowie beim Abschluss einer Verarbeitung eingesetzt. Im Frontend kapselt der `RetryExecutor` darüber hinaus sämtliche REST-Aufrufe an das Backend. Durch diese einheitliche Verwendung entsteht ein konsistentes Wiederholungsverhalten über alle Komponenten hinweg, ohne dass Retry-Logik mehrfach implementiert werden muss. Änderungen an Wiederholungsstrategien beschränken sich dadurch auf die jeweilige `RetryPolicy`.

5.2.7 Recovery

Die Wiederherstellung verwaister Jobs wird durch die beiden Komponenten `RecoveryService` und `RecoveryExecutor` umgesetzt. Während der `RecoveryService` die periodische Suche

nach wiederherzustellenden Jobs übernimmt, implementiert der `RecoveryExecutor` die eigentliche Recovery-Logik innerhalb einer Datenbanktransaktion.

Die in listing 5.13 dargestellte zentrale Methode des `RecoveryService` wird mithilfe der Spring-Annotation `@Scheduled` in festen Zeitabständen ausgeführt. Im entwickelten System erfolgt ein Recovery-Zyklus alle 30 Sekunden. Dabei werden zunächst sämtliche Jobs ermittelt, deren Lease bereits abgelaufen ist und deren Status weiterhin *Running* lautet. Hierzu wird die bereits in Kapitel 5.2.2 vorgestellte Repository-Abfrage verwendet. Die Abfrage liefert ausschließlich die Identifikatoren der betroffenen Jobs zurück und verwendet bewusst keine Sperren. Dadurch werden während der Suche keine Datenbankzeilen unnötig blockiert und konkurrierende Worker können ihre Verarbeitung gegebenenfalls noch beenden. Für jede gefundene Job-ID wird anschließend der `RecoveryExecutor` aufgerufen.

```
1 @Scheduled(fixedDelay = 30, timeUnit = TimeUnit.SECONDS)
2 public void recoverCycle() {
3     List<UUID> ids =
4         repository.findByStatusAndLease_untilBefore(
5             JobStatus.RUNNING, Instant.now());
6
7     for (UUID id : ids) {
8         recoveryExecutor.recoverJob(id);
9     }
10 }
```

Listing 5.13: Periodischer Recovery-Zyklus

Die eigentliche Wiederherstellung erfolgt innerhalb der Methode `recoverJob()`, die mit `@Transactional` gekennzeichnet ist. Die Details zeigt listing 5.14. Zu Beginn der Transaktion wird der betreffende Job erneut über seine ID geladen und dabei mittels `SELECT FOR UPDATE` exklusiv gesperrt. Die Sperre stellt sicher, dass Recovery und konkurrierende Worker den Zustand desselben Jobs nicht gleichzeitig verändern können.

Da zwischen der Suche nach abgelaufenen Leases und dem Beginn der Recovery-Transaktion bereits Änderungen am Job erfolgt sein können, wird dessen aktueller Zustand erneut überprüft. Insbesondere kann ein Worker den Job zwischenzeitlich noch erfolgreich abgeschlossen haben. Befindet sich der Job daher nicht mehr im Zustand *Running*, wird die Recovery ohne weitere Änderungen beendet. Die erneute Prüfung verhindert, dass bereits erfolgreich abgeschlossene Jobs aufgrund einer zwischenzeitlichen Zustandsänderung fälschlicherweise erneut in die Warteschlange aufgenommen werden.

Anschließend wird anhand des `attempt_count` entschieden, ob der Job erneut ausgeführt werden darf. Wurde die konfigurierte maximale Anzahl an Ausführungsversuchen noch nicht erreicht, wird der Job wieder in den Zustand *Queued* überführt. Andernfalls wird er als *Failed* markiert. Dadurch werden endlose Wiederholungszyklen dauerhaft fehlerhafter Jobs verhindert.

Unabhängig von der aktuellen Versuchsanzahl werden die Attribute `claimed_by` und `lease_until` zurückgesetzt sowie der Änderungszeitpunkt aktualisiert.

```
1 @Transactional
```

```

2 public void recoverJob(UUID id) {
3     Job job = repository.findByIdForUpdate(id).orElse(null);
4
5     if (job == null || job.getStatus() != JobStatus.RUNNING) {
6         return;
7     }
8     if (job.getAttempt_count() < maxAttempts) {
9         job.setStatus(JobStatus.QUEUED);
10    } else {
11        job.setStatus(JobStatus.FAILED);
12    }
13
14    job.setClaimed_by(null);
15    job.setLease_until(null);
16    job.setUpdatedAt(Instant.now());
17 }

```

Listing 5.14: Transaktionale Wiederherstellung eines Jobs

Da der Recovery-Mechanismus ausschließlich Jobs mit dem Status `RUNNING` betrachtet und alle anderen Zustände ignoriert, können über den Recovery-Prozess keine Zustandsänderungen bereits abgeschlossener Jobs erfolgen. Zusammen mit der Einschränkung, dass Worker ausschließlich Jobs im Zustand `QUEUED` übernehmen können, ergibt sich, dass Endzustände nach ihrem Erreichen nicht mehr verlassen werden können. Damit wird die in Z-5 formulierte Finalität der Endzustände sichergestellt.

6 Evaluation

Dieses Kapitel untersucht im Rahmen von TZ4, ob das implementierte Job-Processing-System die in Kapitel 4 definierten Zusicherungen auch unter den jeweils betrachteten Fehlerszenarien einhält. Hierzu wird zunächst der konkrete Versuchsaufbau beschrieben. Anschließend werden die für die einzelnen Zusicherungen durchgeführten Fehlerszenarien und deren Prüfkriterien dargestellt. Darauf aufbauend wird die konkrete Umsetzung der Tests inklusive Testparameter beschrieben und anschließend die beobachteten Ergebnisse ausgewertet. Den Abschluss bildet die Bewertung der formulierten Hypothesen.

6.1 Versuchsaufbau

Sofern für die einzelnen Tests nicht anders angegeben, wird die Anwendung für die Tests mittels `@SpringBootTest` vollständig mit allen ihren Komponenten gestartet. Dadurch kann bei den entsprechenden Tests nicht nur das Verhalten einzelner isolierter Komponenten untersucht werden, sondern auch das Zusammenspiel von REST-Schnittstelle, Service-Schicht, Retry- und Recovery-Mechanismen sowie der Datenbank. Die Datenbank läuft dabei lokal über Docker auf einem Postgres 17 image, um die benötigte Datenbank reproduzierbar und unter kontrollierten Bedingungen bereitstellen zu können. Vor den jeweiligen Testdurchläufen wird außerdem sichergestellt, dass die relevanten Datenbanktabellen leer sind, sodass die einzelnen Testläufe unabhängig voneinander sind und sich nicht gegenseitig beeinflussen.

Die Anzahl an Wiederholungen der nebenläufigkeitsabhängigen Tests wird auf 100 Wiederholungen festgelegt. Die Wahl von 100 Wiederholungen stellt dabei einen Kompromiss zwischen der Testabdeckung unwahrscheinlicher Grenzfälle und dem für die Durchführung erforderlichen Zeitaufwand dar. Ein erfolgreich abgeschlossener Testlauf wird dabei nicht als Beweis für das generelle Ausbleiben eines Fehlers interpretiert. Vielmehr wird überprüft, ob die für das jeweilige Szenario definierten Prüfkriterien über sämtliche Wiederholungen hinweg eingehalten werden. Bei Tests, bei denen unterschiedliche Ausgänge des Rennens möglich sind, wird zusätzlich überprüft, dass die vorgesehenen konkurrierenden Ausgänge tatsächlich im Verlauf der Wiederholungen auftreten.

Die verwendeten Systemparameter sind für die getesteten Szenarios angepasst. Das Ziel dabei war die einzelnen Komponenten so aufeinander abzustimmen, dass die betrachteten Fehlerinjektionen zuverlässig und reproduzierbar durchgeführt werden können und dass insbesondere das Auftreten der Konkurrenzsituationen maximiert wird. Konkret wurde die Anzahl der Worker auf 1 gesetzt, die Anzahl der maximalen Verarbeitungsversuche für Jobs auf 5, die Lease-Dauer auf 10 Sekunden, die Verarbeitungszeit der Jobs auf 5 Sekunden und das Recovery-Intervall ebenfalls auf 5 Sekunden. Das Retry- und Backoff-Verhalten ist mit 200ms und einem Faktor von 2^n gewählt, um die Laufzeit der Tests zu begrenzen. Findet ein Worker keinen freien Job, pausiert er und wartet 2 Sekunden, bis er den Job-Bestand erneut prüft. Einzelne Parameter können für die unterschiedlichen Tests angepasst werden, so beispielsweise die Anzahl Worker für die nebenläufigkeitsabhängigen Tests. Diese Änderungen werden bei der jeweiligen Testbeschreibung dargelegt.

6.2 Fehlerszenarien und Operationalisierung

Ausgangspunkt der folgenden Testszenarien sind die bereits in Kapitel 4 definierten Zusicherungen. Für jede Zusicherung wurde untersucht, welches konkret beobachtbare Systemverhalten ihre Verletzung anzeigen würde. Die resultierenden Prüfkriterien und Fehlerinjektionen sind in Tabelle 6.1 zusammengefasst. Ziel dabei war, die Szenarien so zu entwerfen, dass möglichst realistische Ausfallbedingungen simuliert werden, wie beispielsweise ein echter Netzwerkfehler oder ein Ausfall einer einzelnen Komponente. Für jede Zusicherung wird mindestens ein Szenario durchgeführt, anhand dessen die jeweils relevanten Prüfkriterien ausgewertet werden können. Die Auswahl inklusive Begründung der betrachteten Fehlerklassen wurde bereits in Kapitel 4.2 beschrieben.

Z	Fehlerinjektion	Umsetzung	Prüfkriterium / Metrik
Z-1	Ungültige Payload	Fachlich und syntaktisch invalide Requests senden	HTTP 400, kein DB-Eintrag; Zeilenzahl vor/nach unverändert
Z-2	Acknowledge geht verloren	Abfangen und verwerfen des Acknowledge, stattdessen Fehler werfen	Client startet automatischen Retry. Die API wird insgesamt 2 Mal angesprochen. Es wird genau ein Job im Backend gespeichert und der Statuscode der vom Client erhaltenen Antwort ist 200 OK
Z-3	(a) Viele Worker konkurrieren um einen Job; (b) Zwei Worker versuchen gleichzeitig Ergebnis zu speichern	(a) K Worker-Instanzen versuchen den gleichen Job zu beanspruchen, loggen des jeweils beanspruchten Jobs; (b) Synchronisation der Worker vor anschließendem parallelen Speicherversuch	(a) Von K gleichzeitigen Versuchen darf genau einer erfolgreich sein; (b) Es gibt genau einen Speichervorgang zu dem Job. Verspäteter Aufruf wird per Fencing abgelehnt
Z-4	Erzwungener Fehler zwischen Save und Commit	Test-Hook wirft Exception nach setStatus(), vor resultRepository.save()	Nach Fehler: kein Ergebnis $\wedge$ Running
Z-5	Recovery-Scan parallel zu finishJob	Recovery-Scan und finishJob zum gleichen Job über Barrier synchronisieren und parallel durch verschiedene Threads ausführen	Kein Übergang aus Succeeded/Failed zurück in Queued

Fortsetzung auf nächster Seite

Z	Fehlerinjektion	Umsetzung	Prüfkriterium / Metrik
Z-6	(a) Transiente DB-Störung; (b) permanenter Fachfehler	(a) DB-Verbindung kurzzeitig kappen; (b) Anfrage mit existierendem Idempotenzschlüssel als permanenter Fehler	(a) Erfolg nach Wiederherstellung, Retry-Anzahl im Log; (b) sofortiger Abbruch ohne Retry-Schleife
Z-7, Z-8, Z-9	Harter Worker-Ausfall während Running	Worker-Instanz während Bearbeitung hart beenden	Job wird nach Recovery von einem neuen Worker aufgenommen und erreicht Succeeded , gespeichertes Ergebnis entspricht dem erfolgreichen (zweiten) Ausführungsversuch
Z-8	(a) Recovery eines Jobs im Status <i>Running</i> ; (b) Lease abgelaufen ohne vorherige Recovery	(a) Manuelle Recovery eines aktiven Jobs. Speicherversuch durch nicht-besitzenden Worker vor und nach Abschluss durch besitzenden Worker; (b) <code>lease_until</code> auf Vergangenheit setzen	(a) Zugriff wird abgelehnt, es wird genau ein Ergebnis vom besitzenden Worker gespeichert; (b) Zugriff wird abgelehnt, kein Ergebnis wird gespeichert
Z-9	Job schlägt dauerhaft fachlich fehl	Processor-Mock wirft für bestimmte Job-ID bei jedem Versuch eine Exception	<code>attempt_count</code> erreicht Maximum, Job geht in Failed über; Zeit bis Failed $\leq \text{maxAttempts} \times (\text{Lease} + \text{Recovery-Intervall} + \text{Worker idle-Timeout})$

Tabelle 6.1: Testmatrix: Zusicherungen, Fehlerinjektion und Prüfkriterien

Eine Zuordnung der in Tabelle 6.1 beschriebenen Fehlerszenarien zu den konkret implementierten Testklassen und -methoden befindet sich im Anhang in Tabelle 8.1.

6.3 Durchführung

Dieser Abschnitt beschreibt, wie die in Tabelle 6.1 definierten Fehlerinjektionen konkret technisch umgesetzt wurden.

6.3.1 Joberstellung und Idempotenz (Z-1, Z-2)

Die Überprüfung von Z-1 erfolgt, indem eine syntaktisch sowie eine fachlich ungültige Anfrage über den Client an das laufende Backend gesendet werden. Vor und nach beiden

Anfragen wird die Anzahl der im Job-Store vorhandenen Jobs über denselben Client abgefragt und verglichen.

Für Z-2 wird das Verlorengelangen einer Erfolgsantwort simuliert. Hierzu wird am `BackendRestClient` ein `ClientHttpRequestInterceptor` registriert, der beim ersten Aufruf die tatsächliche Serverantwort abwartet, anschließend jedoch eine `IOException` auslöst, bevor die Antwort an den Client weiter gereicht wird. Dadurch verarbeitet das Backend die Anfrage und persistiert den Job, ohne dass diese Information beim Client ankommt. Der clientseitige `RetryExecutor` sollte die Exception folglich als transienten Kommunikationsfehler interpretieren und dieselbe Anfrage samt identischem Idempotency-Key erneut übertragen. Schließlich wird geprüft, ob tatsächlich nur ein Job persistiert wurde. Auf diese Weise wird das in Kapitel 2.1.2 beschriebene Szenario eines verlorenen Acknowledgements durch einen transienten Netzwerkfehler realitätsnah nachgebildet. Zusätzlich prüft dieser Test die Durchführung eines automatischen clientseitigen Retries bei transienten Fehlern (vgl. Kapitel 6.4).

6.3.2 Retry-Verhalten bei Infrastrukturfehlern (Z-6)

Für die Untersuchung transienter Datenbankfehler wird die über Docker bereitgestellte PostgreSQL-Datenbank in einem Testcontainer ausgeführt. Die Datenbankanbindung der Anwendung wird dabei über einen von Toxiproxy bereitgestellten Proxy geführt. Die Fehlerinjektion erfolgt durch das gezielte Unterbrechen der über den Proxy geführten Verbindung, während der Datenbank-Container selbst weiterläuft. Ein Neustart der Datenbank selbst hätte bezogen auf den Test der Retry-Funktionalität gegenüber der gewählten Fehlerinjektion keinen wesentlichen Mehrwert, da sowohl ein Neustart als auch ein hier simulierter Verlust der Datenbankverbindung aus Sicht der Anwendung zunächst in einer nicht verfügbaren Datenbank resultieren. Auch wenn sich die zugrunde liegenden Ursachen unterscheiden, ist für den hier betrachteten Retry-Fall entscheidend, dass die Anwendung den Datenbankzugriff vorübergehend nicht erfolgreich durchführen kann. Auch für die Dauerhaftigkeit und Konsistenz bereits erfolgreich persistierter Daten hätte ein Testfall, der den Crash oder Neustart der Datenbank simuliert, keinen Mehrwert, da diese Eigenschaften nicht durch die entwickelte Architektur, sondern durch die ACID-Eigenschaften von PostgreSQL sichergestellt werden. Daher wird im Rahmen der Evaluation auf einen solchen Test verzichtet. Da die Worker und der Recovery-Service für die folgenden Tests nicht benötigt werden, aber die Logs verunreinigen würden, werden beide Komponenten für die in diesem Kapitel beschriebenen Tests deaktiviert.

Da eine bereits offene, im Verbindungspool vorgehaltene Datenbankverbindung von der Unterbrechung des Proxys nicht zwangsläufig betroffen ist, wird der Verbindungspool nach der Unterbrechung zusätzlich gezielt geleert, sodass der nächste Datenbankzugriff tatsächlich einen neuen, über den unterbrochenen Proxy geführten Verbindungsversuch erzwingt. In Testfall (a) wird die Verbindung für eine festgelegte Dauer unterbrochen und anschließend automatisiert wiederhergestellt. Als Variante hierzu wird zusätzlich untersucht, wie sich das System verhält, wenn die Unterbrechung dauerhaft bestehen bleibt und sämtliche Wiederholungsversuche erschöpft werden, bevor die Datenbankverbindung wiederhergestellt wird. Für Testfall (b) wird eine Anfrage mit einem bereits verwendeten

Idempotency-Key gesendet, um stellvertretend die Behandlung eines nicht retrybaren Datenbankfehlers zu prüfen.

Die Retry-Logik wird dabei ausschließlich anhand der Joberstellung überprüft. Sowohl das Beanspruchen als auch der Abschluss eines Jobs verwenden dieselbe `RetryExecutor/DBRetryPolicy`-Komponente. Eine gesonderte Untersuchung an diesen Stellen liefert daher keinen zusätzlichen Erkenntnisgewinn über die bereits geprüfte Klassifikations- und Wiederholungslogik hinaus. Der für Z-2 durchgeführte Test belegt zudem das clientseitige Retry-Verhalten bei Kommunikationsfehlern zwischen Client und Backend. Eine vollständige, zu (a) und (b) symmetrische Untersuchung der clientseitigen `RestMessageRetryPolicy` wird nicht zusätzlich durchgeführt, da der zugrunde liegende `RetryExecutor`-Mechanismus bereits anhand der `DBRetryPolicy` vollständig untersucht wurde und sich beide Policies lediglich in der konkreten Fehlerklassifikation unterscheiden.

6.3.3 Atomarität der Ergebnisspeicherung (Z-4)

Um einen Fehler gezielt zwischen der Statusänderung und der Persistierung des Ergebnisses auszulösen, wird ein Test-Hook in die Implementierung von `finishJob` eingefügt, der im produktiven Betrieb keine Wirkung entfaltet und im Testfall über Mockito so konfiguriert wird, dass er nach der Statusänderung, aber vor dem Speichern des Ergebnisses eine Exception auslöst. Anschließend wird geprüft, dass die Änderungen verworfen wurden und kein Zwischenzustand im Job-Store verblieben ist.

6.3.4 Nebenläufigkeit beim Claiming und Abschluss (Z-3, Z-5)

Die Testszenarien zu Z-3 und Z-5 hängen vom zeitlichen Zusammenspiel mehrerer konkurrierender Operationen ab. Um die konkurrierenden Aufrufe möglichst zeitgleich zu starten, werden die beteiligten Threads über eine `CyclicBarrier` synchronisiert, sodass keiner der Aufrufe beginnt, bevor nicht alle beteiligten Threads bereitstehen. Um die Worker synchronisieren zu können, wird im Rahmen der Tests ein separater Threadpool genutzt, dem die jeweiligen Operationen übergeben werden. Damit die Test-Jobs garantiert von den Operationen des Threadpools verarbeitet werden, wird die Anzahl der Worker in diesen Tests auf 0 gesetzt.

Da die getesteten Eigenschaften hier Zugriffs- und Schreibrechte der Worker auf spezifische Datensätze betreffen, ist eine notwendige Voraussetzung für die Aussagekraft dieser Tests, dass die konkurrierenden Zugriffe tatsächlich auf Datenbankebene aufeinandertreffen und nicht bereits vorher durch den Verbindungspool serialisiert werden. Die verwendete maximale Poolgröße muss daher mindestens der Anzahl gleichzeitig zugreifender Worker entsprechen. Um diese Systemeigenschaft sicherzustellen, wird dies vor Ausführung der betroffenen Testklasse automatisiert überprüft und die Testausführung andernfalls abgebrochen.

Für Z-3 (a) wird ein einzelner Job von mehreren Worker-Threads gleichzeitig beansprucht. Anschließend wird geprüft, wie viele Worker dabei Erfolg hatten. Da für den Test genau ein Job im Job-Store persistiert ist, sollte nur ein Worker erfolgreich sein und alle anderen ein leeres `Optional`-Objekt erhalten. Z-3 (b) testet demgegenüber nach

dem gleichen Prinzip das Speichern desselben Jobs zur selben Zeit. Ein besitzender und ein nicht besitzender Job versuchen über eine Barrier synchronisiert ein Ergebnis zu speichern. Zum Test von Z-5 werden `finishJob` und der direkte Aufruf von `recoverJob` auf denselben Job angewendet, wobei der Job absichtlich mit bereits erreichter maximaler Versuchsanzahl präpariert wird, sodass je nach Ausgang des Wettlaufs entweder der erfolgreiche Abschluss oder der endgültige Fehlschlag eintritt. Der Verlierer des Rennens darf dann keine Änderung mehr vornehmen, da der Gewinner den Job bereits in einen finalen Zustand überführt hat.

Für Z-5 wird zusätzlich zur mehrfachen Ausführung protokolliert, wie viele Durchläufe auf den jeweiligen Ausgang entfallen, um sicherzustellen, dass das Rennen tatsächlich beide möglichen Ausgänge erzeugt und nicht durch eine feste Reihenfolge vorentschieden ist.

6.3.5 Recovery nach Worker-Ausfall (Z-7, Z-8, Z-9)

Zum Nachweis von Z-7 wird ein tatsächlicher Prozessabbruch herbeigeführt. Dazu wird der SpringBoot Kontext geladen, aber die Anzahl laufender Worker auf 0 gesetzt. Anschließend wird das System als eine zweite eigenständige Instanz als Prozess gegen dieselbe Datenbank gestartet und ausgeführt. Dieser Prozess verfügt über einen laufenden Worker. Aus dem Test heraus wird ein Job erzeugt und bis zu dessen Übernahme durch den Worker-Prozess abgewartet. Sobald der Job in Bearbeitung ist, wird der Prozess hart beendet, ohne dass ein geordnetes Herunterfahren stattfindet. Daraufhin wird geprüft, ob der Worker Prozess den Job tatsächlich nicht beenden konnte und ein zweiter, unabhängiger Prozess mit laufendem Worker wird gestartet. Der `RecoveryService` sollte den Job im nächsten Durchlauf als verwaist erkennen und erneut zur Bearbeitung ausstellen. Der neue Worker sollte den verwaisten Job daraufhin übernehmen. Zusätzlich zum Erreichen des Zustands `Succeeded` wird für Z-8 geprüft, dass der gespeicherte Ergebnisinhalt dem tatsächlich erfolgreichen, zweiten Verarbeitungsversuch entspricht und keine Reste des abgebrochenen ersten Versuchs enthält. Funktioniert die Recovery wird außerdem gezeigt, dass ein Job nach Ablauf des Leases aus dem aktiven Zustand befreit wird, sodass er dort nicht hängen bleibt (Z-9).

Der Test zu Z-8 (a) prüft, dass die Berechtigungen eines Workers nach der Recovery erlöschen. Dazu wird nach Ablauf des Leases ein Recovery-Durchlauf aktiviert und gewartet, bis der Job durch einen anderen Worker übernommen wird. Daraufhin versucht der ursprüngliche Besitzer ein Ergebnis zu speichern. Dieser Versuch wird nach Fertigstellung durch den neuen Besitzer wiederholt. Zusätzlich untersucht Z-8 (b), ob der Worker die Berechtigung nach Ablauf des Leases auch ohne vorherigen Recovery-Durchgang verliert. Zur Überprüfung wird ein Job mit Status *Running* und abgelaufenem Lease im Job-Store persistiert. Anschließend versucht der eingetragene Besitzer den Job über `finishJob` zu beenden.

Für Z-9 wird die Fachlogik eines einzelnen, gezielt ausgewählten Jobs über einen `JobProcessor-Spy` mittels `@MockitoSpyBean` so konfiguriert, dass jeder Verarbeitungsversuch mit einer Exception fehlschlägt. Vor der eigentlichen Untersuchung wird zunächst ein Kontrolljob regulär verarbeitet, um sicherzustellen, dass der konfigurierte Spy die

grundsätzliche Funktionsfähigkeit des Workers nicht beeinträchtigt. Anschließend wird geprüft, dass der fehlschlagende Job nach genau der konfigurierten Maximalanzahl der Wiederholungsversuche in den Zustand *Failed* überführt wird und kein weiterer Verarbeitungsversuch mehr stattfindet. Da Z-9 den Übergang in einen Endzustand nach endlicher Zeit fordert, wird für die maximale Zeit bis zur endgültigen Beendigung eines dauerhaft fehlschlagenden Jobs eine obere Schranke definiert. Diese ergibt sich aus der maximalen Anzahl der Verarbeitungsversuche sowie der für einen einzelnen Verarbeitungszyklus maximal zu erwartenden Wartezeit:

$$T_{\max} = A_{\max} \cdot (T_{\text{lease}} + T_{\text{recovery}} + T_{\text{idle}})$$

Dabei bezeichnet $A_{\max}$ die maximal zulässige Anzahl von Verarbeitungsversuchen, bevor ein Job in *Failed* überführt wird, T_{lease} die konfigurierte Lease-Dauer bis ein Job als verwaist gilt, T_{recovery} das Recovery-Intervall und T_{idle} das Warteintervall eines verfügbaren Workers zwischen zwei Übernahmeversuchen. Um die Testlaufzeit zu optimieren, werden für ausschließlich diesen Test das Recovery-Intervall und die Lease-Dauer auf 1s reduziert. In Kombination mit den in Kapitel 6.1 beschriebenen Parametern ergibt sich daraus für den Test folgende Konfiguration:

$$\begin{aligned} A_{\max} &= 5 \\ T_{\text{lease}} &= 1 \text{ s}, \\ T_{\text{recovery}} &= 1 \text{ s}, \\ T_{\text{idle}} &= 2 \text{ s}. \end{aligned}$$

Daraus folgt, dass für die Zeit bis zum Übergang in einen finalen Status die obere Schranke von $T_{\max} = 20 \text{ s}$ gilt. Diese Schranke gilt unter der Voraussetzung, dass während des gesamten Testzeitraums die Anzahl der verfügbaren Worker gleich groß wie die der ausstehenden Jobs ist. Da die Anzahl Worker und Jobs in diesem Test beide eins beträgt, ist diese Bedingung erfüllt.

6.4 Ergebnisse

Die im Folgenden dargestellten Beobachtungen stützen sich auf die tatsächlich protokollierten Testläufe. Gekürzte, für die jeweilige Beobachtung repräsentative Auszüge der Logs finden sich im Anhang unter Kapitel 8.2. Die vollständigen Protokolle aller Durchläufe sind der Arbeit unter Testlogs.md als Begleitdatei beigefügt.

Atomare Persistenz und Validität (Z-1) Beide Requests wurden vom Backend mit dem HTTP-Status 400 *Bad Request* abgelehnt, der erste aufgrund der syntaktischen, der zweite aufgrund der fachlichen Validierung. Die Anzahl der im Job-Store vorhandenen Jobs war vor und nach beiden Anfragen identisch, folglich kam es für keine der beiden ungültigen Requests zu einer Persistierung des Jobs.

Damit wurde das für Z-1 definierte Prüfkriterium erfüllt. Die Atomarität der eigentlichen Persistierung wurde durch den Test nicht direkt überprüft, sondern ergibt sich konstruktiv

aus der Verwendung von `@Transactional` mit PostgreSQL, da die Datenbank selbst ACID sicherstellt.

Duplikaterkennung bei der Auftragserfassung (Z-2) Im ersten Ausführungsversuch wurde der Job erfolgreich persistiert, bevor die simulierte `IOException` die Antwort abfing. Der clientseitige `RetryExecutor` übertrug daraufhin dieselbe Anfrage samt identischem Idempotency-Key erneut. Das Backend erkannte dabei die Verletzung des entsprechenden `UNIQUE`-Constraints und behandelte den zweiten Aufruf als Idempotenzfall, ohne diesen seinerseits erneut zu wiederholen. Der clientseitige Retry wurde also tatsächlich ausgeführt, während der resultierende Datenbankfehler auf Backendseite korrekt als nicht retrybar eingestuft wurde. Der zweite Client-Aufruf lieferte den HTTP-Status `200 OK` und denselben, bereits zuvor erzeugten Job zurück. Während des gesamten Tests erhöhte sich die Anzahl der Jobs im Job-Store lediglich von null auf eins.

Das für Z-2 definierte Prüfkriterium ist damit erfüllt: Eine erneute Übermittlung derselben Anfrage wie hier beispielsweise nach einem verlorenen Response führt nicht zu einer zweiten Persistierung, sondern liefert den bereits vorhandenen Job zurück.

Exklusive Rechte (Z-3) Bei der Untersuchung des Claimings (Testfall a) konnte in allen Durchläufen jeweils genau ein Worker den Job erfolgreich übernehmen, erkennbar am einmaligen Zustandsübergang von *Queued* nach *Running*. Die übrigen konkurrierenden Worker erhielten in keinem der Durchläufe einen Job. Folglich wurde ein mehrfacher Übergang desselben Jobs nach *Running* in keinem Durchlauf beobachtet, was bedeutet, dass das Claiming exklusiv ablief.

Bei der Untersuchung der Ergebnisspeicherung (Testfall b) konnte in allen Durchläufen jeweils nur der aktuell berechnete Worker den Job erfolgreich abschließen. Der konkurrierende Aufruf mit abweichender Worker-ID wurde je nach Scheduling der Anfragen entweder abgewiesen, da der Job schon nicht mehr im Zustand *Running* war, oder weil die Worker-ID nicht mit dem gespeicherten `claimed_by`-Wert übereinstimmte. Nach jedem Testlauf existierte genau ein persistiertes `JobResult`, dessen Inhalt dem vom berechtigten Worker übermittelten Ergebnis entspricht.

Damit wurde das für Z-3 definierte Prüfkriterium in beiden Testfällen erfüllt: Ein verfügbarer Job wird höchstens einem Worker zur Verarbeitung zugeordnet und von zwei konkurrierenden Abschlussversuchen kann höchstens einer das Ergebnis persistieren.

Atomare Wirkung von Ergebnissen (Z-4) Nach dem durch den Test-Hook erzwungenen Fehlschlagen des Aufrufs befand sich der Job weiterhin im Zustand *Running* und ein `JobResult` wurde nicht persistiert. Die zuvor im Persistenzkontext vorgenommene Statusänderung zu *Succeeded* wurde vollständig verworfen.

Das für Z-4 definierte Prüfkriterium ist damit erfüllt: Tritt innerhalb der gemeinsamen Transaktion ein Fehler wie beispielsweise zwischen Statusänderung und Ergebnispersistierung auf, bleiben keine Teiländerungen in der Datenbank bestehen. Die beobachteten Zustände bestätigen damit die durch die ACID-Garantien und das Transaktionsmodell umgesetzte atomare Wirkung.

Finalität von Endzuständen (Z-5) In 47 der 100 Durchläufe gewann `finishJob`. Der Job erreichte den Zustand *Succeeded*, die Ergebnisdaten wurden persistiert und der konkurrierende `recoverJob`-Aufruf wurde aufgrund des bereits erreichten Endzustands folgenlos beendet. In den übrigen 53 Durchläufen gewann `recoverJob`. Da die maximale Anzahl an Verarbeitungsversuchen bereits erreicht war, wurde der Job in den Zustand *Failed* überführt und der konkurrierende `finishJob`-Aufruf wurde abgewiesen.

Damit traten beide möglichen Ausgänge des konkurrierenden Zugriffs in den wiederholten Testläufen tatsächlich auf. Anhand der protokollierten Zustandsübergänge jedes Durchlaufs wurde zusätzlich sichergestellt, dass in keinem der 100 Durchläufe ein Übergang zurück aus einem bereits erreichten Endzustand nach *Queued* oder *Running* erfolgte. Die beobachteten Übergänge beschränkten sich durchgehend auf *Queued* nach *Running* sowie anschließend entweder auf *Running* nach *Succeeded* oder *Running* nach *Failed*.

Die Testergebnisse zeigen das durch die Implementierung umgesetzte Verhalten. Im System existieren genau drei Stellen, an denen der Status eines Jobs verändert wird: `claimNextJob`, `finishJob` und `recoverJob`. Für alle drei Stellen bestehen Vorbedingungen, die eine Änderung ausschließen, sobald ein Job bereits *Succeeded* oder *Failed* erreicht hat: `claimNextJob` berücksichtigt ausschließlich Jobs im Zustand *Queued*, während `finishJob` und `recoverJob` ihre Verarbeitung abbrechen, wenn der Job nicht den Zustand *Running* besitzt. Da der relevante Datensatz bei allen drei Operationen mit `FOR UPDATE` innerhalb einer Transaktion gesperrt wird, werden Lesen des Status, Prüfung der Vorbedingung und Zustandsänderung isoliert und atomar gegenüber konkurrierenden Zugriffen ausgeführt. Eine zweite Transaktion mit altem Status kann die Prüfung daher nicht umgehen. Da sämtliche statusverändernden Operationen durch diese Vorbedingungen eingeschränkt sind, folgt daraus, dass ein einmal erreichter Endzustand nicht mehr verlassen werden kann. Die empirischen Ergebnisse bestätigen dieses Verhalten unter den untersuchten konkurrierenden Zugriffen.

Automatische Wiederholungsversuche (Z-6) In Testfall (a) schlug der erste Zugriffsversuch aufgrund der nicht verfügbaren Datenbankverbindung fehl. Der `RetryExecutor` klassifizierte den Fehler als retrybar und führte drei weitere Ausführungsversuche mit Wartezeiten von 200 ms, 400 ms und 800 ms durch. Nach Wiederherstellung der Datenbankverbindung schloss der vierte Versuch erfolgreich ab, sodass die Anfrage mit dem HTTP-Status 201 `Created` beantwortet wurde und der zurückgegebene Idempotency-Key dem des ursprünglichen Requests entsprach. Ergänzend wurde als Gegenprobe geprüft, wie sich das System bei einem anhaltenden als transient klassifizierten Fehler verhält. Die Anfrage wurde nach Erschöpfung aller konfigurierten Wiederholungsversuche mit dem HTTP-Status 500 `Internal Server Error` beantwortet und es wurde kein erneuter Retry gestartet.

Im zweiten Testfall führte die erneute Verwendung eines bereits vorhandenen Idempotency-Keys zu einer `DataIntegrityViolationException`. Der `RetryExecutor` stufte diese als nicht retrybar ein, sodass kein weiterer Ausführungsversuch erfolgte. Der Controller behandelte den Fehler als Idempotenzfall und beantwortete die Anfrage mit 200 `OK`.

Das für Z-6 definierte Verhalten – Fehlerklassifikation, automatische Wiederholung bei

transienten und unmittelbarer Abbruch bei permanenten Fehlern sowie eine definierte Terminierung bei dauerhaftem Ausfall – wurde damit unter den betrachteten Bedingungen beobachtet.

Worker-Ausfall und Wiederaufnahme (Z-7, Z-8, Z-9) Der gestartete Worker-Prozess übernahm den erzeugten Job und setzte dessen Status auf *Running*. Nach dem gezielten, harten Abbruch des Prozesses befand sich der Job weiterhin im Status *Running*. Der zunächst gestartete zweite Worker-Prozess übernahm den Job nicht sofort, sondern erst nachdem der Recovery-Service den verwaisten Job beim nächsten Zyklus anhand des abgelaufenen Leases erkannte und ihn in den Zustand *Queued* zurückführte. Der zweite Worker übernahm den Job daraufhin und verarbeitete ihn erfolgreich. Der Job erreichte den Status *Succeeded* mit einem `attempt_count` von zwei, was die ursprüngliche Verarbeitung durch den ersten und die erneute Verarbeitung durch den zweiten Worker bestätigt. Der gespeicherte Ergebnisinhalt entsprach dabei nachweislich dem tatsächlich erfolgreichen, zweiten Verarbeitungsversuch und enthielt keine Reste des unterbrochenen ersten Versuchs.

Damit wurde das für Z-7 definierte Verhalten beobachtet: Der Ausfall eines Workers führt nicht zum Verlust eines bereits persistierten Jobs, sondern zur automatischen Wiederfreigabe durch den Recovery-Service und der anschließenden erneuten Verarbeitung. Die Persistenz des Jobs über den Prozessabbruch hinweg wird dabei die Durability-Eigenschaft von PostgreSQL abgesichert. Während die Recovery dafür sorgt, dass der Job nicht in der Verarbeitung hängen bleibt, sorgt ACID dafür, dass die Jobdaten selbst nicht verloren gehen. Dass der Job zudem mit inhaltlich korrektem, nicht durch den abgebrochenen ersten Versuch verfälschtem Ergebnis abschließt, liefert zugleich einen Teilbeleg für Z-8. Der beobachtete Übergang aus *Running* zurück nach *Queued* nach Ablauf des Leases zeigt darüber hinaus einen Teilaspekt von Z-9, nämlich dass ein Job nicht dauerhaft in einem aktiven Zustand verbleibt.

Eindeutigkeit des Ergebnisses bei abgelaufenem Lease (Z-8) Im Testszenario (a) wurde der Job nach explizit ausgelöstem Recovery-Vorgang in den Zustand *Queued* überführt und von einem anderen Worker übernommen. Der zuvor berechnete, jetzt aber überholte Worker wurde bei seinem anschließenden `finishJob`-Aufruf abgewiesen. Der neue Worker schloss den Job dagegen erfolgreich ab. Auch der `finishJob`-Aufruf des überholten Workers nach der Fertigstellung wurde abgelehnt. Insgesamt wurde genau ein `JobResult` für den Job persistiert.

In Testszenario (b) wurde der Aufruf von `finishJob` durch den weiterhin eingetragenen Worker aufgrund des abgelaufenen Leases mit einer `LeaseExpiredException` abgewiesen und kein Ergebnis persistiert.

Damit ist sowohl der Fall eines bereits durch Recovery übernommenen als auch der Fall eines abgelaufenen, aber noch nicht durch die Recovery wiederhergestellten Jobs abgedeckt. In beiden Fällen konnte ein verspäteter Worker kein Ergebnis mehr speichern, während der gültige Verarbeitungsversuch aus Testfall (a) genau ein Ergebnis erzeugte. Anknüpfend an die im vorangegangenen Abschnitt beobachtete inhaltliche Korrektheit des

gespeicherten Ergebnisses nach einem echten Prozessabbruch, zeigt dies den verbliebenen zu zeigenden Teil von Z-8.

Das für Z-8 definierte Prüfkriterium wird damit erfüllt: Für einen Job existiert höchstens ein persistiertes Ergebnis und ein verspäteter Worker wird an der Ergebnispersistierung gehindert. Das beobachtete Verhalten realisiert damit die At-Most-Once-Eigenschaft der Ergebnisspeicherung. Zusammen mit der bei Z-7 überprüften erneuten Verarbeitung nach einem Worker-Ausfall wird die angestrebte At-Least-Once-Verarbeitung mit eindeutiger Ergebniswirkung angenähert.

Terminierung in endlicher Zeit (Z-9) Der zunächst verarbeitete Kontrolljob erreichte erwartungsgemäß den Status *Succeeded* und bestätigte damit, dass der konfigurierte JobProcessor-Spy die grundsätzliche Funktionsfähigkeit des Workers nicht beeinträchtigt. Für den anschließend erzeugten Testjob trat bei jedem Übernahmeversuch der simulierte permanente Fachfehler auf. Nach jedem fehlgeschlagenen Versuch lief das Lease ab, woraufhin der Recovery-Service den Job erneut in den Zustand *Queued* überführte. Dieser Ablauf wiederholte sich, bis die konfigurierte maximale Anzahl an Verarbeitungsversuchen erreicht war.

Nach dem letzten fehlgeschlagenen Versuch überführte der Recovery-Service den Job nicht erneut nach *Queued*, sondern in den Endzustand *Failed*. Der abschließende `attempt_count` entsprach dabei genau dem konfigurierten Wert `maxAttempts`.

Das für Z-9 definierte Kriterium ist damit erfüllt: Der dauerhaft fehlschlagende Job verbleibt nicht unbegrenzt im Wechsel zwischen *Running* und *Queued*, sondern wird nach Erreichen der maximalen Anzahl von Verarbeitungsversuchen endgültig in den Zustand *Failed* überführt. Die hierfür benötigte Zeit betrug 12,179 s und lag damit unterhalb der aus den konfigurierten Zeitparametern abgeleiteten oberen Schranke von 20 s.

Für die Terminierung ist neben der Begrenzung der Anzahl der Verarbeitungsversuche auch die Reihenfolge der Jobauswahl relevant. Die Auswahl erfolgt wie in Kapitel 5.2.2 beschrieben anhand des Erstellungszeitpunkts durch die Sortierung der Jobs innerhalb der Datenbankabfrage nach dem Attribut `created_at`. Dadurch kann es nicht passieren, dass ein Job dauerhaft durch später erstellte Jobs verdrängt wird.

6.5 Bewertung der Hypothesen

[TODO: H1, H2 und H3 jeweils explizit bewerten: Welche Tests stützen die Hypothese? Welche Ergebnisse wurden beobachtet? Wird die Hypothese bestätigt, teilweise gestützt oder nicht gestützt?]

7 Diskussion und Fazit

[TODO: To be done. Hier sind bis jetzt nur Notizen]

7.1 Gewonnene Erkenntnisse

7.2 Einschränkungen und Grenzen der Arbeit

Ausschließlich Notizen

Diese Arbeit betrachtet die Fehlertoleranz ab dem Zeitpunkt, zu dem ein Job persistent im Job-Store vorliegt; die zuverlässige Auftragserzeugung/Client-Kommunikation (z.B. Outbox, Messaging) ist nicht Gegenstand. Consumers may crash at any time, so it could happen that a broker delivers a message to a consumer but the consumer never processes it, or only partially processes it before crashing. In order to ensure that the message is not lost, message brokers use *acknowledgments*: a client must explicitly tell the broker when it has finished processing a message so that the broker can remove it from the queue." (Kleppmann 2017, Kap.11 -> Acknowledgments and redelivery bzw. Message brokers)

REST hat keinen Puffer und ist synchron -> Client und Server muss während der ganzen Kommunikation laufen, damit Kommunikation funktioniert. Weitere Nachteile in Referenc: (Richardson 2018, Kap.3.2.2)

Kein Fokus auf Kommunikation im allgemeinen, weder wie oben genannt die Erstellung von Jobs, als auch die Benachrichtigung bei Fertigstellung. Dafür bräuchte es Transaktionales messaging bspw. mittels transactional outbox pattern (Richardson 2018, Kap.3.3.7) oder asynchrones Messaging (Richardson 2018, Kap.3.4.2)

Es gibt keine perfekte reliability (Kleppmann 2017, Kap. 1, Abschnitt „Reliability“)

Load Leveling umgesetzt?

Performance von DB gegenüber Message Queues

Keine Fehlertoleranz in Bezug auf Datenbeständigkeit (z.B. Festplatte des Job-Stores geht kaputt), Naturkatastrophen (Ganzer Standort fällt aus z.B. wegen Überschwemmung) (Newman 2026, Kap. 9)

Es gibt keine Möglichkeit absolute Garantien zu geben, nur viele Möglichkeiten der Risikoreduktion (Kleppmann 2017, Kap. 7 Abschnitt „The Meaning of ACID“-> Ende)

um performance zu verbessern könnte man verschiedene checkpoints in die verarbeitung einbringen, sodass bei einem Abbrechen einer Task nicht die gesamte Task neu gestartet wird sondern ab einem bestimmten punkt wieder angefangen wird (Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability considerations“).

vorsicht bei dinge bei denen kein rollback gemacht werden kann: versendet ein worker eine email, stürzt ab und die operation wird wiederholt wird potenziell noch eine email versendet. das muss gesondert abgefangen werden

Es können Probleme entstehen, wenn zu viele Anfragen rein kommen, die die DB nicht gleichzeitig verarbeiten kann (vgl. Exmaple (Microsoft Corporation (Hrsg.) 2026b)) Dafür

bräuchte es separates Queue-Based Load Leveling für das Messaging an sich, das System konzentriert sich auf Queue-Based Load Leveling bzgl der Jobverarbeitung. Es werden zwar Clientseitige Retries durchgeführt, das ist aber keine Garantie, dass die Nachricht trotz hoher Last ankommt

bietet sich vor allem an, wenn man sowieso schon eine db nutzt, man braucht keine zusätzliche Infrastruktur (im Vergleich zum Message Broker, der als eigener Server läuft)

Bei sehr großen Systemen kann die Datenbank aufgrund einer hohen Anzahl Zugriffsversuche jedoch zum limitierenden Faktor werden, sodass dedizierte Messaging-Systeme hinsichtlich des erreichbaren Durchsatzes Vorteile bieten (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“).

Vermeidung komplexität verteilter Transaktionen durch Nutzung lokaler ACID-Transaktionen da alles in derselben Datenbank liegt

Einschränkung Z-7 nur bis zu einem bestimmten Grad -> Kein Job kann verloren gehen, aber ein job wird nicht unendlich oft recovered da sonst Konflikt mit Z-9

- viel fine tuning möglich, was zum einen die Konfiguration der Parameter für Retries angeht (backoff factor, initial delay, maxAttempts) und zum anderen die genaue Klassifikation der Fehler in transient und permanent, hier vor allem nur zu Vorführungszwecken exemplarisch implementiert

Die vorgestellte Architektur garantiert die Konsistenz der Jobverwaltung und der Ergebnispersistenz. Seiteneffekte innerhalb der eigentlichen Fachlogik liegen dagegen außerhalb des Verantwortungsbereichs des Frameworks. Soll die Verarbeitung externer Ressourcen ebenfalls fehlertolerant erfolgen, muss die Fachlogik selbst geeignete Mechanismen wie Transaktionen, Kompensationsoperationen oder idempotente Operationen einsetzen.

bekanntes Zitat von Dijkstra, sinngemäß: *Testen kann die Anwesenheit von Fehlern zeigen, niemals ihre Abwesenheit.* -> Testing liefert hier grundsätzlich nur notwendige, keine hinreichende Evidenz

Einschränkungen:

- Prototyp statt produktives System
- PostgreSQL als konkretes Datenbanksystem
- begrenzte Anzahl an Workern
- künstlich injizierte Fehler
- keine byzantinischen Fehler
- kein Heartbeat
- simulierte Fachlogik
- kontrollierte Testumgebung
- Race Conditions werden durch künstliche Verzögerungen wahrscheinlicher gemacht

- Ergebnisse können nicht unmittelbar auf beliebige verteilte Systeme übertragen werden

Ausblick: -Performance-Test -> ist das System in der Praxis brauchbar? -Test, wie das System skaliert -> wie performt es bei vielen Anfragen auf die Datenbank? Literatur weist auf hohe Latenzen hin

Literaturverzeichnis

- Alquraan, Ahmed u. a. (Okt. 2018). „An Analysis of Network-Partitioning Failures in Cloud Systems“. In:
- Apache Software Foundation (Hrsg.) (22. Mai 2026). *Apache Kafka Documentation 4.3.X*. AK 4.3.X. URL: <https://kafka.apache.org/43/> (besucht am 26.06.2026).
- Bass, Len (Aug. 2021). *Software architecture in practice*. Unter Mitarb. von Paul Clements und Rick Kazman. Fourth edition. SEI series in software engineering. Boston: Addison-Wesley.
- Broadcom Inc. (Hrsg.) (15. Juni 2026a). *RabbitMQ Documentation / RabbitMQ*. URL: <https://www.rabbitmq.com/docs> (besucht am 26.06.2026).
- (24. Mai 2026b). *Spring Boot*. Spring Boot. URL: <https://spring.io/projects/spring-boot> (besucht am 24.05.2026).
- Cristian, Flavin (Feb. 1991). „Understanding fault-tolerant distributed systems“. In: *Communications of the ACM* 34.2, S. 56–78. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/102792.102801. URL: <https://dl.acm.org/doi/10.1145/102792.102801> (besucht am 10.08.2026).
- Hangfire OÜ (Hrsg.) (26. Juni 2026). *Hangfire – Background jobs and workers for .NET and .NET Core*. URL: <https://www.hangfire.io> (besucht am 26.06.2026).
- Hohpe, Gregor und Bobby Woolf (2005). *Enterprise integration patterns designing, building, and deploying messaging solutions*. 5. print. Boston [u.a.: Addison-Wesley.
- Kleppmann, Martin (2017). *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. First edition. Beijing, [China: O’Reilly.
- Kudraß, Thomas (5. März 2015). *Taschenbuch Datenbanken*. Hrsg. von Kudraß Thomas. Carl Hanser Verlag GmbH & Co. KG. ISBN: 978-3-446-43508-7. DOI: 10.3139/9783446440265.fm. URL: <https://www.hanser-elibrary.com/doi/10.3139/9783446440265.fm> (besucht am 21.06.2026).
- Microsoft Corporation (Hrsg.) (31. März 2026a). *Best Practices for Background Jobs - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/background-jobs> (besucht am 17.06.2026).
- (12. Juni 2026b). *Queue-Based Load Leveling Pattern - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling> (besucht am 17.06.2026).
- Newman, Sam (Sep. 2026). *Building Resilient Distributed Systems*. O’Reilly. URL: <https://learning.oreilly.com/library/view/building-resilient-distributed/9781098163532/> (besucht am 24.05.2026).
- Richardson, Chris (2018). *Microservices patterns*. Software development. Shelter Island: Manning.
- Rosoco BV (Hrsg.) (24. Mai 2026). *JobRunr Documentation*. URL: <https://www.jobrunr.io/en/documentation/> (besucht am 24.05.2026).
- Rupp, Chris (2021). *Requirements-Engineering und -Management*. Unter Mitarb. von SOPHIST-Gesellschaft für Innovatives Software-Engineering. 7. Auflage. München: Carl Hanser Verlag GmbH & Co. KG. ISBN: 978-3-446-45587-0.

- Saake, Gunter, Andreas Heuer und Kai-Uwe Sattler (Juni 2018). *Datenbanken – Konzepte und Sprachen*. mitp Verlags GmbH & Co KG. ISBN: 978-3-95845-778-2. URL: <https://learning.oreilly.com/library/view/datenbanken-konzepte/9783958457782/> (besucht am 19.06.2026).
- Starke, Gernot (2024). *Effektive Softwarearchitekturen ein praktischer Leitfaden*. 10., überarbeitete Auflage. München: Carl Hanser Verlag GmbH & Co. KG. ISBN: 978-3-446-47672-1.
- Temporal Technologies Inc. (24. Mai 2026). *Temporal Docs*. URL: <https://docs.temporal.io/> (besucht am 24.05.2026).
- Terracotta Inc. (Hrsg.) (24. Mai 2026). *Quartz Scheduler Documentation*. URL: <https://www.quartz-scheduler.org/documentation/quartz-2.5.x/> (besucht am 24.05.2026).
- The PostgreSQL Global Development Group (Hrsg.) (14. Mai 2026). *PostgreSQL 18.4 Documentation*. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/18/index.html> (besucht am 21.06.2026).

8 Anhang

8.1 Testzuordnung

Fehlerszenario	Testmethode
Z-1	<code>JoberstellungTest.invalidPayloadTest()</code>
Z-2	<code>JoberstellungTest.duplicateRequestTest()</code>
Z-3 (a)	<code>ConcurrencyTest.onlyOneWorkerClaimsContendedJob()</code>
Z-3 (b)	<code>ConcurrencyTest.concurrentFinishAttemptsYieldExactlyOneStoredResult()</code>
Z-4	<code>WorkerCrashRecoveryTest.atomicSaveTest()</code>
Z-5	<code>JobStatusModellTest.finishJobAndRecoveryRaceNeverReversesTerminalState()</code>
Z-6 (a)	<code>DatabaseRetryTest.jobCreationRecoversAfterTransientDbOutage()</code>
Z-6 (a Gegenprobe)	<code>DatabaseRetryTest.jobCreationFailsAfterExhaustedRetriesOnPermanentOutage()</code>
Z-6 (b)	<code>DatabaseRetryTest.testRetryNotAppliedOnConstraintViolation()</code>
Z-7, Z-8, Z-9	<code>JobRecoveryForWorkerCrashTest.jobSurvivesRealWorkerProcessCrash()</code>
Z-8 (a)	<code>WorkerCrashRecoveryTest.recoveryComponentTest()</code>
Z-8 (b)	<code>WorkerCrashRecoveryTest.finishRejectedWhenLeaseExpiredButNotYetRecovered()</code>
Z-9	<code>JobTerminationTest.terminationAfterTooManyFailuresTest()</code>

Tabelle 8.1: Zuordnung von Zusicherungen zu den jeweiligen Testmethoden

8.2 Testlogs

`JoberstellungTest.invalidPayloadTest()`

`JoberstellungTest.duplicateRequestTest()`

`ConcurrencyTest.onlyOneWorkerClaimsContendedJob()`

`ConcurrencyTest.concurrentFinishAttemptsYieldExactlyOneStoredResult()`

`WorkerCrashRecoveryTest.atomicSaveTest()`

JobStatusModelTest.finishJobAndRecoveryRaceNeverReversesTerminalState()

DatabaseRetryTest.jobCreationRecoversAfterTransientDbOutage()

DatabaseRetryTest.jobCreationFailsAfterExhaustedRetriesOnPermanentOutage()

DatabaseRetryTest.testRetryNotAppliedOnConstraintViolation()

JobRecoveryForWorkerCrashTest.jobSurvivesRealWorkerProcessCrash()

WorkerCrashRecoveryTest.recoveryComponentTest()

WorkerCrashRecoveryTest.finishRejectedWhenLeaseExpiredButNotYetRecovered()

JobTerminationTest.terminationAfterTooManyFailuresTest()